

Complete Python Course - Module List (End-to-End, Full Coverage)

MODULE 1 - Python Basics & Getting Started

- Introduction to Python
- History, Features, Applications
- Installing Python
- Running Python (REPL + Script)
- Basic Syntax
- Indentation
- Comments
- Print/Input
- Writing First Program

MODULE 2 - Variables, Data Types & Operators

- Variables & Assignment
- Dynamic Typing
- Keywords
- Basic Data Types
- Type Conversion
- Arithmetic Operators
- Comparison Operators
- Logical Operator
- Assignment Operator

Input/Output Formatting

MODULE 3 - Control Flow Statements

if, elif, else

Nested Conditions

Loops (for, while)

Loop Control: break, continue, pass

range() Function

Patterns Using Loops

Infinite Loops (Mistakes & Handling)

MODULE 4 - Functions, Modules & Packages

Defining Functions

Parameters & Return Values

Default & Keyword Arguments

Variable-Length Arguments (*args, **kwargs)

Lambda Expressions

map(), filter(), reduce()

Creating Custom Modules

import, from-import, alias

Standard Modules Overview (math, random, datetime)

MODULE 5 - Python Data Structures

Strings

Indexing, Slicing

String Methods

Mutable vs Immutable Strings

Lists

CRUD operations

List methods

List slicing

List comprehensions

Tuples

Immutable sequences

Tuple packing & unpacking

Sets

Set operations

Useful set functions

Dictionaries

Key-Value Pairs

CRUD operations

dict methods

Iteration patterns

MODULE 6 - Object-Oriented Programming (OOPS)

Classes & Objects

init() and Constructors

Instance & Class Variables

Instance, Class & Static Methods

Encapsulation (_private variables)

Inheritance

Method Overriding

Polymorphism

Built-in Magic Method

MODULE 7 - File Handling & Exception Handling

File Handling

Working with files (read, write, append)

File modes

Working with text files

Working with binary files

CSV handling (intro)

MODULE 8- Exception Handling

try, except, finally

Multiple exceptions

Raising exception

Module 1: Introduction to Python Programming(theory)

1. Introduction to Programming

Programming is the process of writing instructions that a computer can understand and execute. These instructions are written using programming languages, which allow humans to communicate with machines in a structured and meaningful way.

Computers themselves cannot understand human language. They operate in binary — sequences of 0s and 1s. Programming languages act as a bridge, enabling developers to express ideas, logic, and operations through readable syntax that can be translated into machine instructions.

Programming is essential in almost every field today: engineering, business, education, healthcare, artificial intelligence, automation, research, and more. Whether it is building a simple calculator or designing a large-scale distributed system, programming forms the foundation.

Programming involves several core concepts:

1.1 Logic Building

Every program is essentially a sequence of logical steps. A programmer must know how to break a problem into smaller, manageable components and convert them into steps that can be written in code.

1.2 Algorithms

An algorithm is a step-by-step logical procedure to solve a problem. For example, an algorithm to add two numbers would involve:

- Take two inputs
- Add them
- Display the result

1.3 Syntax

Syntax refers to the rules of a programming language. Just like grammar in English, every language has its rules. Violating these rules causes syntax errors.

1.4 Semantics

Semantics refers to the meaning of the code. Even if the syntax is correct, the program must make logical sense.

2. What is Python?

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and released in 1991. It focuses on readability, simplicity, and productivity.

The name "Python" comes from the comedy series *Monty Python's Flying Circus*, not from the snake.

Python is known for its clean and minimal syntax, making it extremely beginner-friendly compared to other languages like Java, C, or C++. It is widely used across domains such as web development, artificial intelligence, machine learning, data science, automation, scripting, and game development.

Python is based on several design philosophies:

- Code readability
- Simple structure
- Minimalistic syntax
- Ease of learning
- Flexibility
- Portability

3. Features of Python

Python stands out due to several powerful features:

3.1 Simple and Easy to Learn

Python's syntax resembles English. For example, printing something in Python is as simple as:

```
print("Hello World")
```

There are no semicolons, no curly braces, and minimal boilerplate code.

3.2 Interpreted Language

Python is not compiled. It executes code line by line. This makes debugging easy and reduces development time.

3.3 Dynamically Typed

You do not need to declare data types. Python automatically detects the type:

```
x = 10          # x is integer
x = "Hello"     # now x is string
```

3.4 Cross-Platform Compatibility

Python works on:

- Windows
- macOS
- Linux
- Unix

Write once, run anywhere.

3.5 Large Standard Library

Python comes with a vast built-in library that includes modules for:

- File handling
- Mathematics
- Networking
- Date and time
- Regular expressions
- System operations
- JSON
- Unit testing

3.6 Extensive Ecosystem

Python's third-party libraries like NumPy, Pandas, TensorFlow, Flask, Django, and PyTorch allow developers to build anything from websites to AI systems.

3.7 Open Source

Python is free to download, modify, and distribute.

3.8 Embeddable

You can embed Python inside C/C++ programs.

3.9 Object-Oriented & Functional

Python supports:

- Functions
- Classes
- Objects
- Inheritance
- Polymorphism
- Higher-order functions

It is multiparadigm and flexible.

4. Applications of Python

Python is used everywhere today.

4.1 Web Development

Frameworks like Django, Flask, and FastAPI help build full-scale web applications.

4.2 Data Science

Libraries:

- NumPy
- Pandas
- Matplotlib
- Scikit-Learn
- Seaborn

Python is the #1 language for data science.

4.3 Machine Learning & AI

ML libraries like TensorFlow, PyTorch, and Keras rely heavily on Python.

4.4 Automation & Scripting

Python can automate tasks such as:

- File handling
- Web scraping
- Sending emails
- API automation

4.5 Cybersecurity

Tools like Scapy, Nmap, and penetration testing scripts use Python.

4.6 Game Development

You can create games using Pygame.

4.7 Mobile Applications

Kivy enables mobile app development.

4.8 GUI Applications

Tkinter is used for desktop GUI development.

Python is universal and extremely useful for both beginners and professionals.

5. Installing Python

Python installation varies by platform, but the process is simple.

5.1 Download Python

Visit: <https://www.python.org/downloads/>

Click the download button for your operating system.

5.2 Installation (Windows)

1. Run the installer
2. Check "Add Python to PATH"
3. Click Install Now
4. Wait for the installation to finish

5.3 Verify Installation

Open CMD or terminal:

```
python --version
```

You should see:

```
Python 3.x.x
```

5.4 Installing on Linux/Mac

Execute:

```
sudo apt install python3
```

or

```
brew install python
```

6. Running Python Code

Python can be executed in several ways.

6.1 Using the Python Shell (REPL)

REPL stands for:

- Read
- Evaluate
- Print
- Loop

Start REPL:

```
python
```

Output:

```
>>>
```

Run code directly:

```
>>> print("Hello")
Hello
```

6.2 Running Python Script File

Create a file: `hello.py`

Write:

```
print("Hello from script!")
```

Execute:

```
python hello.py
```

6.3 Using an IDE

Popular IDEs:

- VS Code
- PyCharm
- Jupyter Notebook
- Thonny
- Spyder

VS Code recommended for beginners.

7. Python Syntax

Python syntax refers to the structure of how the language is written.

7.1 Indentation

Python uses indentation to define blocks.

Correct:

```
if True:
    print("Inside block")
```

Wrong:

```
if True:
print("Error")
```

Indentation usually uses 4 spaces.

7.2 Statements

A statement is a single line of code performing an action:

```
x = 5
print(x)
```

Multi-line statements can use \ but are usually written using parentheses.

7.3 Comments

Comments help describe the code.

Single-line comment:

```
# This is a comment
```

Multi-line comment:

```
"""
This is
multi-line
comment
"""
```

8. Keywords in Python

Keywords are reserved words and cannot be used as variable names.

Common keywords: if, else, elif, for, while, class, def, return, try, except, import, from, with, global, lambda, True, False, None

You can list all keywords:

```
import keyword
print(keyword.kwlist)
```

9. Variables in Python

Variables are named locations used to store data. Python variables are created the moment you assign a value.

Example:

```
x = 10
name = "Hemanth"
```

9.1 Dynamic Typing

Python allows variables to change type:

```
x = 10          # int
x = "Hello"     # string
```

9.2 Variable Naming Rules

- Must not start with a number
- Cannot use keywords
- No spaces
- Underscore allowed

Valid: my_name, age2, country_code

Invalid: 2age, my name, class

10. Data Types (Overview)

Python supports multiple built-in data types.

10.1 Numeric Types

- int → Whole numbers
- float → Decimal numbers
- complex → e.g., 4 + 2j

10.2 Boolean Type

- `True`
- `False`

10.3 String Type

Sequences of characters:

```
"Hello"  
'Python'
```

10.4 Sequence Types

- `List`
- `Tuple`
- `Range`

10.5 Mapping Type

- `Dictionary {key: value}`

10.6 Set Types

- `Set`
- `Frozen set`

Modules 2 and 5 explain these in detail.

11. Input and Output

11.1 Output: `print()`

```
print("Welcome")
```

Printing multiple values:

```
print("Name:", "Hemanth")
```

11.2 Input: `input()`

```
name = input("Enter your name: ")
```

Everything returned from `input()` is a string.

12. Errors in Python

Python has several types of errors:

12.1 SyntaxError

Caused by incorrect syntax.

```
print("Hello"
```

12.2 NameError

Using undefined variables.

```
print(age)
```

12.3 TypeError

Invalid operation on incompatible types.

```
print(10 + "abc")
```

12.4 IndentationError

Incorrect indentation.

```
if x > 5:  
print("No")
```

12.5 ValueError

Invalid value passed to a function.

```
int("Hello")
```

13. The Python Interpreter

The Python interpreter processes the code.

Steps:

1. Code is written in .py file
2. Interpreter reads line by line
3. Bytecode is generated
4. Python Virtual Machine (PVM) executes it

Because Python is interpreted, it is slower than compiled languages — but easier to debug.

14. Python Comments, Docstrings & Documentation

14.1 Comments

Used to explain code.

14.2 Docstrings

Used for function/class documentation.

Example:

```
def add(a, b):  
    """This function returns the sum of two numbers"""  
    return a + b
```

View docstring:

```
print(add.__doc__)
```

15. Python Execution Model

Python executes code using the following flow:

15.1 Source Code (.py file)

Written by the programmer.

15.2 Interpreter Parses Code

Checks syntax.

15.3 Bytecode Generation

Interpreter converts code into bytecode (.pyc).

15.4 Execution on PVM

Python Virtual Machine executes bytecode.

16. Hello World Explained in Depth

Here is the simplest Python program:

```
print("Hello World")
```

Explanation:

- `print()` is a built-in function
- `"Hello World"` is a string literal
- The interpreter executes it line by line
- Output is displayed on the screen

This single line is the starting point for every Python programmer.

17. Python File Structure

A typical Python script includes:

Shebang line (optional):

```
#!/usr/bin/python3
```

Imports:

```
import math
```

Functions / Classes:

```
def example():  
    pass
```

Executable Code:

```
print("Program started")
```

18. Built-in Functions (Brief Overview)

Python includes 140+ built-in functions like:

- `print()`
- `len()`
- `type()`
- `input()`
- `range()`

- `help()`
- `abs()`
- `sum()`

These functions help perform basic operations without importing any library.

19. Python IDE Setup

19.1 VS Code

- Install Python extension
- Use integrated terminal
- Auto-formatting
- Linting

19.2 PyCharm

- Professional-grade features
- Advanced debugging
- Project management

19.3 Jupyter Notebook

- Popular for data science
 - Run code in cells
 - Inline visualizations
-

20. Best Practices for Beginners

- Always write readable code
 - Use proper indentation
 - Use meaningful variable names
 - Comment your code
 - Test code frequently
 - Avoid writing everything in one file
-

21. Summary of Module 1 (Theory)

- Python is a beginner-friendly, high-level programming language
- It is simple, readable, and extremely powerful
- Installation is straightforward on all platforms
- Python offers multiple ways to run code
- Syntax is clean and minimal

- Indentation is used instead of braces
- Comments and docstrings improve code clarity
- Variables do not require type declarations
- Python has many built-in data types
- Errors help identify mistakes in code
- Python has a simple execution model
- "Hello World" is the first program every Python programmer writes
- IDEs like VS Code and PyCharm help write code efficiently

MODULE-1(SNIPPETS):

MODULE 1 — Examples & Code Snippets

These examples help beginners understand Python's basics such as printing, input, comments, indentation, variables, keywords, and the structure of a Python program.

★ 1. Hello World Program

Example 1: Basic Print Statement

```
print("Hello, World!")
```

Explanation:

- The `print()` function outputs text to the screen.
- `"Hello, World!"` is a string literal.

Output:

```
Hello, World!
```

★ 2. Printing Multiple Values

Example 2: Print With Multiple Arguments

```
print("My name is", "Hemanth")
```

Explanation:

- Python automatically inserts a space between values.
- This is different from concatenation.

Output:

My name is Hemanth

★ 3. Indentation Example

Example 3: Correct Indentation

```
if True:
    print("This is indented correctly")
```

Output:

```
This is indented correctly
```

Wrong Indentation:

```
if True:
print("Indentation Error")
```

This raises:

```
IndentationError: expected an indented block
```

★ 4. Comments in Python

Example 4: Single-Line Comment

```
# This is a single-line comment
print("Comments are ignored by Python!")
```

Output:

```
Comments are ignored by Python!
```

Example 5: Multi-Line Comment

```
"""
This is a multi-line comment
used to explain code in detail
"""
print("Python supports multi-line comments using triple quotes!")
```

★ 5. Using Input From the User

Example 6: Taking User Input

```
name = input("Enter your name: ")
print("Hello,", name)
```

Sample Output:

```
Enter your name: Hemanth
Hello, Hemanth
```

Example 7: Taking Numerical Input

```
age = int(input("Enter your age: "))
print("You are", age, "years old.")
```

Sample Output:

```
Enter your age: 20
You are 20 years old.
```

★ 6. Keywords Example

Example 8: Display All Python Keywords

```
import keyword
print(keyword.kwlist)
```

Output:

A list like:

```
['False', 'None', 'True', 'and', 'as', 'assert', 'break', ...]
```

★ 7. Variables & Dynamic Typing

Example 9: Assigning Variables

```
x = 10
name = "Python"
pi = 3.14

print(x, name, pi)
```

Output:

```
10 Python 3.14
```

Example 10: Dynamic Typing

```
x = 10
print(x)

x = "Now I am a String!"
print(x)
```

Output:

```
10
Now I am a String!
```

★ 8. Basic Python Program Structure

Example 11: Simple Program With Comments

```
# Program to show basic structure

# Input Section
name = input("Enter your name: ")

# Processing Section
greeting = "Hello, " + name

# Output Section
print(greeting)
```

Sample Output:

```
Enter your name: Ravi
Hello, Ravi
```

★ 9. Using Escape Characters

Example 12: Escape Sequences

```
print("Hello\nPython")    # New line
print("Hello\tWorld")     # Tab space
print("He said \"Python is great\"") # Quotes
```

Output:

```
Hello
Python
Hello  World
He said "Python is great"
```

★ 10. Working With print() Parameters

Example 13: Using sep Parameter

```
print("Python", "Java", "C++", sep=" | ")
```

Output:

```
Python | Java | C++
```

Example 14: Using end Parameter

```
print("Hello", end=" ")  
print("World")
```

Output:

```
Hello World
```

★ 11. Type Checking and Conversion

Example 15: Checking Type

```
num = 10  
print(type(num))
```

Output:

```
<class 'int'>
```

Example 16: Type Conversion

```
x = "100"  
y = int(x)  
print(y + 50)
```

Output:

```
150
```

★ 12. Using Built-in help() Function

Example 17: Getting Help

```
help(print)
```

This shows documentation about the `print()` function.

★ 13. Arithmetic Operations (Very Basic Preview)

Example 18: Simple Arithmetic

```
a = 10
```

```
b = 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

Output:

```
13
7
30
3.3333333333333335
```

★ 14. String Concatenation

Example 19: Using + Operator

```
first = "Python"
second = "Programming"
print(first + " " + second)
```

Output:

```
Python Programming
```

★ 15. Using Variables Inside Strings

Example 20: f-Strings (Preview for Beginners)

```
name = "Hemanth"
age = 21
print(f"My name is {name} and I am {age} years old.")
```

Output:

```
My name is Hemanth and I am 21 years old.
```

★ 16. Errors Demonstration

Example 21: SyntaxError Example

```
print("Hello"
```

Raises:

```
SyntaxError: unexpected EOF while parsing
```

Example 22: NameError Example

```
print(username)
```

Error:

```
NameError: name 'username' is not defined
```

Example 23: TypeError Example

```
print("Age: " + 20)
```

Error:

```
TypeError: can only concatenate str (not "int") to str
```

★ 17. First Mini-Project: Introduction Program

Example 24: Basic Introduction Script

```
print("=== Welcome to Python Introduction Program ===")

name = input("Enter your name: ")
college = input("Enter your college name: ")

print("\n--- Your Details ---")
print("Name:", name)
print("College:", college)
print("Happy Learning!")
```

Sample Output:

```
=== Welcome to Python Introduction Program ===
Enter your name: Hemanth
Enter your college name: MLRIT

--- Your Details ---
Name: Hemanth
College: MLRIT
Happy Learning!
```

★ 18. Using Variables to Store Messages

Example 25: Message Storing

```
message = "Python is simple and powerful!"
print(message)
```

Output:

```
Python is simple and powerful!
```

★ 19. Multi-Line Output

Example 26: Multi-Line Print

```
print("""Python is awesome.  
It is easy to learn.  
It is used in AI, ML, Web Development, and more.""")
```

★ 20. Using print with Formatting

Example 27: Using format()

```
name = "Hemanth"  
branch = "Data Science"  
  
print("My name is {} and I study {}".format(name, branch))
```

★ 21. Getting Basic System Information

Example 28: import sys

```
import sys  
print("Python version:", sys.version)
```

★ 22. Simple Menu Program (Beginner Level)

Example 29: Menu Display

```
print("=== Simple Menu ===")  
print("1. Python Basics")  
print("2. Variables")  
print("3. Exit")
```

★ 23. Using Operators for Simple Tasks

Example 30: Small Calculator

```
a = 5  
b = 2  
  
print("Addition:", a + b)  
print("Subtraction:", a - b)  
print("Multiplication:", a * b)
```

```
print("Division:", a / b)
```

★ 24. Demonstrating String Multiplication

```
print("Hello " * 3)
```

Output:

```
Hello Hello Hello
```

★ 25. Simple Pattern Printing

Example 31: Star Pattern

```
print("")
print("***")
print("*****")
```

Output:

```
*
**
***
```

Summary

This module covered:

- ✓ Basic print statements and output formatting
- ✓ Python indentation rules
- ✓ Comments (single-line and multi-line)
- ✓ User input with `input()` function
- ✓ Variables and dynamic typing
- ✓ Keywords in Python
- ✓ Type checking and conversion
- ✓ Escape characters and string manipulation
- ✓ Common errors and debugging
- ✓ Basic arithmetic operations
- ✓ Simple mini-projects for practice

MODULE 1 — LECTURES (Complete & Detailed)

Python Basics & Getting Started

This module contains 5 full lectures, covering everything from Python introduction to writing the first program.

★ Lecture 1: Introduction to Python

Topic

"What is Python? Features, History, and Applications"

Definition (detailed)

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum in 1991.

It emphasizes code readability, simple syntax, and faultless design philosophy.

Python supports multiple paradigms: procedural, object-oriented, and functional programming.

Python programs are executed line-by-line by an interpreter, making it easy to debug and develop.

Python is one of the most widely used languages today in Web Development, AI, Machine Learning, Data Science, Automation, and more.

Python's syntax is designed to be clean, expressive, and human-readable, making it ideal for beginners.

Syntax (Intro Example)

```
print("Welcome to Python Programming")
```

This simple line shows how easy Python is—no semicolons, no braces, no complex structure.

Examples

Example 1: Simple Print

```
print("Python is easy to learn!")
```

Example 2: Case Sensitivity

```
Name = "Hemanth"  
name = "Python"
```

```
print(Name)
```

```
print(name)
```

Explanation: Python is case-sensitive. `Name` and `name` are different variables.

Key Takeaways

- Python is beginner-friendly and extremely powerful.
- It is an interpreted language (executes line by line).
- Used in Data Science, Web Dev, AI, ML, Cybersecurity, Automation.
- Syntax is simple and close to natural English.
- Python is open-source and has massive community support.

Practice Section

ready_to_practice: You are ready to understand Python's basic environment, history, and features.

description: Practice writing simple print statements and explore what Python can be used for.

mcqs: Attempt MCQs 1–3 from Module 1.

coding_challenges: Try a simple task: *"Print three lines describing why you want to learn Python."*

★ Lecture 2: Installing Python & Setting Up Environment

Topic

"Installing Python on Windows/Mac/Linux, Understanding REPL, and Setting Up IDEs"

Definition (detailed)

Python installation involves downloading the official package from python.org.

A Python environment consists of the interpreter, standard library, and PATH configuration.

REPL (Read–Evaluate–Print Loop) is Python's interactive shell where each line executes immediately.

IDEs like VS Code, PyCharm, and Jupyter Notebook simplify writing, debugging, and running code.

The environment setup ensures Python scripts run globally from your terminal.

Syntax — Running Python in REPL

```
>>> print("Hello from REPL!")
```

Examples

Example 1: Checking Python Version

```
python --version
```

Output:

```
Python 3.12.2
```

Example 2: Running a Python Script

File: `hello.py`

```
print("Hello from Python script!")
```

Run:

```
python hello.py
```

Example 3: VS Code Setup

1. Install VS Code
2. Install Python extension
3. Create file → `app.py`
4. Run using "Run Code" button or integrated terminal

Key Takeaways

- Always check "Add Python to PATH" during installation.
- Python REPL allows instant execution.
- Python scripts end with `.py`.
- Use VS Code or PyCharm for better productivity.

Practice Section

ready_to_practice: You can now install Python and run basic programs.

description: Write and run your first `.py` file.

mcqs: Attempt MCQs 3–5.

coding_challenges: Task: Create a script named `welcome.py` printing your name and college.

★ Lecture 3: Python Syntax & Indentation

Topic

"Understanding Python Syntax, Indentation Rules, and Code Blocks"

Definition (detailed)

Python syntax refers to the structure and rules that define how Python code must be written.

Python uses indentation instead of braces `{ }` to define code blocks.

Improper indentation leads to `IndentationError`.

A code block is a group of statements executed together (inside loops, functions, conditionals).

Python enforces whitespace sensitivity—spaces and tabs must not mix.

Syntax Example

```
if True:
    print("This block is indented correctly")
```

Examples

Example 1: Correct Indentation

```
if 5 > 2:
    print("Five is greater than two!")
```

Example 2: Wrong Indentation

```
if 5 > 2:
print("Error due to indentation")
```

This raises:

```
IndentationError: expected an indented block
```

Example 3: Multi-Level Indentation

```
if True:
    print("Level 1")
    if True:
        print("Level 2")
```

Key Takeaways

- Indentation determines code structure in Python.
- Standard indentation = 4 spaces.
- Mixing tabs and spaces is not allowed.
- Code blocks are created using indentation, not brackets.

Practice Section

ready_to_practice: You can now write correct Python indentation.

description: Practice nested indentation and simple conditional blocks.

mcqs: Attempt MCQs 5–7.

coding_challenges: Task: Write a program with two nested if statements.

★ Lecture 4: Comments, Documentation & Basic Input/Output

Topic

"Using Comments, Docstrings, print(), input(), and Writing Clean Code"

Definition (detailed)

Comments are lines ignored by the Python interpreter; used for explanation.

Python supports single-line and multi-line comments.

Docstrings are multi-line strings used for documenting functions, classes, or modules.

`print()` displays output; `input()` takes input from the user.

Clean code uses comments, docstrings, indentation, and meaningful variable names.

Syntax

Single-line comment:

```
# This is a comment
```

Multi-line comment:

```
"""  
This is a multi-line comment
```

```
"""
```

print():

```
print("Hello")
```

input():

```
name = input("Enter your name: ")
```

Examples

Example 1: Comments Demo

```
# This prints a greeting
print("Hello, Python!")
```

Example 2: Docstring Example

```
def greet():
    """This function prints a welcome message."""
    print("Welcome!")
```

Example 3: User Input

```
city = input("Enter your city: ")
print("You live in", city)
```

Example 4: Printing with sep and end

```
print("Python", "Java", "C++", sep=" | ")
print("Hello", end=" ")
print("World")
```

Key Takeaways

- Comments improve readability.
- Docstrings document code behavior.
- `input()` always returns a string.
- `print()` is highly flexible with parameters like `sep` and `end`.

Practice Section

ready_to_practice: You can now use comments, docstrings, and input/output.

description: Document your code properly and use input/output functions.

mcqs: Attempt MCQs 7–9.

coding_challenges: Task: Write a program that:

- takes name and age as input

- prints them in a clean formatted way

★ Lecture 5: First Python Program (Hello World Program & Execution Model)

Topic

"Writing Your First Program — Hello World, Execution Flow, Program Structure"

Definition (detailed)

Python programs consist of statements executed sequentially unless directed otherwise.

The first program typically introduces beginners to syntax and environment.

Programs usually contain:

- imports
- variable declarations
- logic
- output

Execution flow: Parse → Compile to Bytecode → Execute on Python Virtual Machine (PVM).

Python files use the `.py` extension and are run via the interpreter.

Syntax - Hello World

```
print("Hello, World!")
```

Examples

Example 1: Basic Program

```
print("Hello from Python!")
```

Example 2: Adding User Interaction

```
name = input("Enter your name: ")
print("Welcome to Python,", name)
```

Example 3: Mini Program Structure

```
# Import section
```

```
import math

# Input section
radius = float(input("Enter radius: "))

# Processing section
area = math.pi * radius * radius

# Output section
print("Area of circle is:", area)
```

Key Takeaways

- Hello World is the first step in any programming language.
- Python programs run top to bottom.
- Python uses `.py` extension for script files.
- Execution involves interpreter, bytecode, and PVM.
- Structuring code improves readability.

Practice Section

ready_to_practice: You can now create your own Python files with basic logic.

description: Build simple programs using input, print, and calculations.

mcqs: Attempt MCQs 10–12.

coding_challenges: Task: Make a program that:

- takes two numbers
- adds them
- prints the total

Module 1 Lecture Summary

What You've Learned:

- ✓ **Lecture 1:** Introduction to Python - Features, history, and applications
- ✓ **Lecture 2:** Installing Python and setting up development environment
- ✓ **Lecture 3:** Python syntax and indentation rules
- ✓ **Lecture 4:** Comments, documentation, and input/output operations
- ✓ **Lecture 5:** First Python program and execution model

Skills Acquired:

- Understanding Python's purpose and capabilities
- Installing and configuring Python environment
- Writing properly indented Python code
- Using comments and docstrings effectively

- Creating interactive programs with input/output
- Understanding Python's execution model

Next Steps:

- Complete all practice sections for each lecture
- Attempt the MCQs (1-12) to test your understanding
- Work through all coding challenges
- Move on to Module 2: Data Types and Operators

Happy Learning!

MODULE 1 — MCQs (15 Questions)

(With options, correct answers & short explanations)

MCQ 1. Python is a _____ language.

- a) Compiled
- b) Interpreted
- c) Machine
- d) Assembly

Answer: b) Interpreted

Explanation: Python executes code line by line using an interpreter, which translates the source code into bytecode at runtime rather than compiling it ahead of time.

MCQ 2. Who created Python?

- a) James Gosling
- b) Guido van Rossum
- c) Dennis Ritchie
- d) Bjarne Stroustrup

Answer: b) Guido van Rossum

Explanation: Python was created by Guido van Rossum in 1991. James Gosling created Java, Dennis Ritchie created C, and Bjarne Stroustrup created C++.

MCQ 3. What is the correct file extension for Python files?

- a) .pt
- b) .py
- c) .pyt
- d) .python

Answer: b) .py

Explanation: Python source files use the `.py` extension. This is the standard convention recognized by all Python interpreters and IDEs.

MCQ 4. Which of the following is used to print output in Python?

- a) `echo()`
- b) `printf()`
- c) `print()`
- d) `output()`

Answer: c) `print()`

Explanation: The `print()` function is Python's built-in function for displaying output to the console. Other options are functions from different programming languages.

MCQ 5. Which symbol is used for single-line comments in Python?

- a) `//`
- b) `#`
- c) `**`
- d) `/* */`

Answer: b) `#`

Explanation: Python uses the hash symbol `#` for single-line comments. Everything after `#` on that line is ignored by the interpreter.

MCQ 6. Python uses _____ to define code blocks.

- a) Parentheses
- b) Curly braces
- c) Angle brackets
- d) Indentation

Answer: d) Indentation

Explanation: Unlike most programming languages that use braces {}, Python uses indentation (typically 4 spaces) to define code blocks, making the code more readable.

MCQ 7. What will be the output of the following code?

```
print("Python" + "3")
```

- a) Python 3
- b) Python+3
- c) Python3
- d) Error

Answer: c) Python3

Explanation: The + operator concatenates strings in Python. "Python" and "3" are both strings, so they join together without spaces to form "Python3".

MCQ 8. Which function is used to take input from the user?

- a) input()
- b) scan()
- c) read()
- d) take()

Answer: a) input()

Explanation: The `input()` function is Python's built-in function for receiving user input from the console. It always returns the input as a string.

MCQ 9. What type does `input()` return?

- a) int
- b) float
- c) boolean
- d) string

Answer: d) string

Explanation: The `input()` function always returns a string, regardless of what the user types. To use it as a number, you must convert it using `int()` or `float()`.

MCQ 10. What will be the output?

```
print("Hello\nWorld")
```

- a) Hello World
- b) HelloWorld
- c) Hello
World
- d) Error

Answer: c) Hello (on one line) World (on the next line)

Explanation: `\n` is an escape sequence that creates a new line. The output will display "Hello" on the first line and "World" on the second line.

MCQ 11. Python programming language was first released in the year _____.

- a) 1990
- b) 1991
- c) 1995
- d) 2000

Answer: b) 1991

Explanation: Python was first released by Guido van Rossum in 1991. It has since become one of the most popular programming languages in the world.

MCQ 12. Which of the following is NOT a Python keyword?

- a) for
- b) while
- c) def
- d) main

Answer: d) main

Explanation: `main` is not a reserved keyword in Python, though it's commonly used as a function name. `for`, `while`, and `def` are all Python keywords with specific meanings.

MCQ 13. What will the following code output?

```
print("Hello", end=" ")
print("Python")
```

- a) HelloPython
- b) Hello Python
- c) Hello
Python
- d) Error

Answer: b) Hello Python

Explanation: The `end=" "` parameter changes the default line ending from a newline to a space, so both print statements output on the same line with a space between them.

MCQ 14. What is the result of this code?

```
x = 5
x = "Python"
print(x)
```

- a) 5
- b) Python
- c) Error
- d) None

Answer: b) Python

Explanation: Python supports dynamic typing, meaning variables can change types. The variable `x` first holds an integer, then gets reassigned to hold a string. The final value is "Python".

MCQ 15. Which module gives the list of all Python keywords?

- a) sys
- b) os
- c) keyword
- d) random

Answer: c) keyword

Explanation: The `keyword` module provides the `kwlist` attribute containing all Python keywords. Use `import keyword` then `print(keyword.kwlist)` to see them.

MODULE 1 — CODING QUESTIONS (10 Beginner Tasks)

Perfect for absolute beginners, simple and easy.

Coding Task 1: Print Your Name

Problem: Write a Python program to print your full name.

Example Solution:

```
print("Hemanth Kumar")
```

Explanation: Use the `print()` function to display text. Replace "Hemanth Kumar" with your actual name. This is the simplest Python program to get started.

Coding Task 2: Simple Greeting Program

Problem: Take the user's name as input and print a greeting message like:


```
Hello, <name>!
```

Example Solution:

```
name = input("Enter your name: ")
print("Hello,", name + "!")
```

Explanation: The `input()` function prompts the user and stores their response in the `name` variable. We then use `print()` to display a personalized greeting. The `+` concatenates the name with the exclamation mark.

Alternative Solution:

```
name = input("Enter your name: ")
print(f"Hello, {name}!")
```

Coding Task 3: Print Three Lines

Problem: Display this exact output using three `print()` statements:

```
Python
is
fun
```

Example Solution:

```
print("Python")
print("is")
print("fun")
```

Explanation: Each `print()` statement automatically adds a newline at the end, so calling `print()` three times displays the words on separate lines.

Coding Task 4: Input & Age Program

Problem: Take age from the user and print a message:

```
You are <age> years old.
```

Example Solution:

```
age = input("Enter your age: ")
print("You are", age, "years old.")
```

Explanation: The `input()` function captures the user's age as a string. We then use `print()` with multiple arguments separated by commas, which automatically adds spaces between them.

Alternative with f-string:

```
age = input("Enter your age: ")
print(f"You are {age} years old.")
```

Coding Task 5: Simple Introduction Program

Problem: Ask the user for:

- name
- branch
- college

Print them in a clean formatted manner.

Example Solution:

```
name = input("Enter your name: ")
branch = input("Enter your branch: ")
college = input("Enter your college: ")

print("\n--- Your Details ---")
print("Name:", name)
print("Branch:", branch)
print("College:", college)
```

Explanation: We collect three pieces of information using `input()`, then display them in a formatted way. The `\n` at the start of the first print creates a blank line for better readability.

Alternative with f-strings:

```
name = input("Enter your name: ")
branch = input("Enter your branch: ")
college = input("Enter your college: ")

print(f"\n--- Your Details ---")
print(f"Name: {name}")
print(f"Branch: {branch}")
print(f"College: {college}")
```

Coding Task 6: Add Two Numbers

Problem: Write a program that:

- takes two numbers as input
- adds them
- prints the result

Example Solution:

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

result = num1 + num2

print("The sum is:", result)
```

Explanation: Since `input()` returns strings, we use `int()` to convert them to integers so we can perform mathematical addition. The result is then calculated and displayed.

Alternative:

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

print("The sum is:", num1 + num2)
```

Coding Task 7: Print Multiplication Pattern

Problem: Print the following using `print()`:

```
*
**
***
```

Example Solution:

```
print("*")
print("**")
print("***")
```

Explanation: Each `print()` statement displays a different number of asterisks. Each call automatically moves to a new line.

Alternative using multiplication:

```
print("*" * 1)
print("*" * 2)
print("*" * 3)
```

Coding Task 8: Escape Sequence Practice

Problem: Print this exact output:

```
Hello
Python    World
"He said Python is amazing"
```

Use `\n`, `\t`, and escape quotes.

Example Solution:

```
print("Hello\nPython\tWorld\n\"He said Python is amazing\"")
```

Explanation:

- `\n` creates a new line
- `\t` creates a tab space
- `\` escapes the quotation marks so they appear in the output

Alternative (multiple prints):

```
print("Hello")
print("Python\tWorld")
print("\"He said Python is amazing\"")
```

Coding Task 9: Using sep and end Parameters

Problem: Print this line using ONE `print()` with `sep=" | "`:

```
Python | Java | C++
```

Then print:

```
Hello World
```

using two `print()` statements but no newline between them (use `end=" "`).

Example Solution:

```
# Part 1: Using sep parameter
print("Python", "Java", "C++", sep=" | ")

# Part 2: Using end parameter
print("Hello", end=" ")
print("World")
```

Explanation:

- The `sep` parameter changes what goes between multiple arguments (default is a space)
 - The `end` parameter changes what comes at the end of the print (default is `\n` for newline)
 - Using `end=" "` makes the next print continue on the same line
-

Coding Task 10: Menu Display Program

Problem: Display a menu like:

```
=== MENU ===
1. Python Basics
```

2. Variables and Data Types
3. Exit

Using only `print()` statements.

Example Solution:

```
print("=== MENU ===")
print("1. Python Basics")
print("2. Variables and Data Types")
print("3. Exit")
```

Explanation: Each `print()` statement displays one line of the menu. Python automatically moves to the next line after each print, creating a well-formatted menu display.

Alternative (single print with multi-line string):

```
print("""=== MENU ===
1. Python Basics
2. Variables and Data Types
3. Exit""")
```

Practice Tips

1. **Test your code:** Run each program and verify the output matches expectations
2. **Experiment:** Try modifying the code to see what changes
3. **Debug:** If you get errors, read the error message carefully—it tells you what went wrong
4. **Type it out:** Don't just copy-paste; typing helps you learn
5. **Add comments:** Practice adding `#` comments to explain your code

Good luck with your practice!

MODULE 2 THEORY — Variables, Data Types & Operators

(3000+ words, master-level explanation)

★ 1. Introduction to Variables and Data Handling in Python

One of the most fundamental concepts in programming is the ability to store, retrieve, and manipulate data. A program is built around data: numbers, text, boolean flags, lists of values, user info, and much more. All these are handled using variables and data types.

Python makes data handling extremely simple due to its dynamic typing, automatic memory management, and flexible operators.

This module covers variables, data types, type conversion, operators, and expressions in extensive detail — forming the backbone for every other Python module.

★ 2. Understanding Variables

A variable is a name that refers to a value stored in memory. In Python, variables are created automatically when a value is assigned.

Example:

```
x = 10
name = "Hemanth"
pi = 3.14
```

Each variable refers to an object in memory.

★ 2.1 Variable Characteristics

✱ Python variables are:

1. Dynamically Typed

Unlike C, C++, or Java:

```
int x = 10;
```

In Python:

```
x = 10
```

Python decides the type based on value.

2. No Need for Declaration

You never declare a variable before using it:

```
age = 21
```

Automatic creation happens during assignment.

3. Variables Are References (Not Boxes)

Python variables do not store the value. They store references to objects in memory.

Example:

```
x = 10  
y = x
```

Here, both `x` and `y` refer to the same object in memory.

★ 2.2 Rules for Naming Variables

Python variable names must follow these rules:

✓ **Allowed:**

- Letters (A–Z, a–z)
- Digits (0–9)
- Underscore (`_`)

✓ **Must NOT:**

- start with digits
- use keywords
- contain spaces
- use special characters: `@`, `#`, `%`, `&`, `*`

✓ **Examples:**

Valid Variable Names	Invalid Variable Names
----------------------	------------------------

Valid Variable Names	Invalid Variable Names
name	lname (starts with digit)
student_name	first-name (hyphen)
age2	class (keyword)
_count	my value (space)

★ 2.3 Variable Assignment

Python uses the equal sign (=) for assignment.

✓ Single Assignment

```
x = 100
```

✓ Multiple Assignment

```
a = b = c = 5
```

✓ Multi-value Assignment

```
x, y, z = 1, 2, 3
```

✓ Swapping Values (Pythonic)

```
a, b = b, a
```

No need for a third variable.

★ 2.4 Constants in Python

Python doesn't have built-in constant types, but programmers use UPPERCASE names:

```
PI = 3.14159  
MAX_STUDENTS = 50
```

It's just a naming convention — not enforced by Python.

★ 3. Understanding Data Types

A data type determines:

- what kind of value an object holds
- what operations can be performed on that value
- how memory is allocated

Python has built-in data types, grouped into categories:

★ 3.1 Numeric Data Types

1. int (Integer)

Whole numbers with unlimited precision.

```
x = 10
y = -50
z = 12345678901234567890
```

Python can handle very large integers automatically.

2. float (Floating-Point)

Numbers with decimals.

```
pi = 3.14
temp = -2.5
```

Floating points follow IEEE-754 double precision format.

3. complex (Complex Numbers)

Used in mathematics and engineering.

```
c = 4 + 3j
```

Parts:

- Real → `c.real`
- Imaginary → `c.imag`

★ 3.2 Boolean Data Type

```
is_valid = True
is_logged_in = False
```

Booleans are used in:

- comparisons

- loops
- conditions
- flags

Boolean values are actually subclasses of integers:

```
int(True) # 1
int(False) # 0
```

★ 3.3 String Data Type

Text or sequences of characters.

```
name = "Python"
message = 'Hello'
paragraph = """This is a multi-line string."""
```

Strings are:

- immutable
 - iterable
 - indexable
-

★ 3.4 Sequence Types

1. List

Ordered, mutable collection.

```
numbers = [1, 2, 3]
```

2. Tuple

Ordered, immutable.

```
coords = (10, 20)
```

3. Range

Used in loops.

```
range(1, 11)
```

★ 3.5 Mapping Type

dict (Dictionary)

Key-value pairs.

```
student = {"name": "Hemanth", "age": 21}
```

★ 3.6 Set Types

set

Unique values.

```
s = {1, 2, 3}
```

frozenset

Immutable set.

★ 3.7 NoneType

Represents the absence of a value.

```
x = None
```

★ 4. Checking Data Types

Use `type()`:

```
print(type(10))          # <class 'int'>
print(type(3.14))        # <class 'float'>
print(type("Python"))    # <class 'str'>
print(type(True))        # <class 'bool'>
```

★ 5. Type Conversion (Casting)

Python performs:

1. Implicit Conversion

Automatic by interpreter.

```
x = 10    # int
y = 2.5    # float
```

```
z = x + y
```

z becomes float.

2. Explicit Conversion

Manual conversion using constructors:

- ✓ `int()`
- ✓ `float()`
- ✓ `str()`
- ✓ `bool()`

5.1 Converting String to Integer

```
x = int("50")
```

Invalid:

```
int("hello") # ValueError
```

5.2 Converting Integer to String

```
s = str(100)
```

5.3 Converting Float to Integer

```
int(3.99) # 3 (flooring)
```

5.4 Converting Boolean

```
int(True) # 1
int(False) # 0

bool(10) # True
bool(0) # False
```

★ 6. Operators in Python

Operators perform operations on values and variables.

★ 6.1 Types of Operators

- ✓ Arithmetic Operators
- ✓ Comparison Operators
- ✓ Logical Operators

- ✓ Assignment Operators
- ✓ Bitwise Operators
- ✓ Membership Operators
- ✓ Identity Operators

Let's explore all in detail.

★ 6.2 Arithmetic Operators

Operator	Meaning	Example	Output
+	Addition	5 + 3	8
-	Subtraction	10 - 4	6
*	Multiplication	4 * 3	12
/	Division	10 / 3	3.333...
//	Floor Division	10 // 3	3
%	Modulo	10 % 3	1
**	Exponent	2 ** 3	8

★ 6.3 Comparison (Relational) Operators

Operator	Meaning
==	Equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal
5 == 5	# True
4 != 3	# True
10 > 20	# False

Returns Boolean values.

★ 6.4 Logical Operators

Operator	Meaning
and	Both conditions must be true

Operator	Meaning
or	At least one must be true
not	Reverses boolean value
True and False	# False
True or False	# True
not True	# False

★ 6.5 Assignment Operators

Operator	Meaning
=	Simple assignment
+=	a = a + b
-=	a = a - b
*=	a = a * b
/=	a = a / b
%=	a = a % b
**=	a = a ** b
//=	a = a // b

Example:

```
x = 5
x += 3 # x = 8
```

★ 6.6 Bitwise Operators

Operate on binary numbers.

Operator	Meaning
&	AND
~	~
^	XOR
~	NOT
<<	Left Shift
>>	Right Shift

Example:

```
5 & 3 # 1
```

★ 6.7 Membership Operators

Operator	Meaning
in	Checks presence
not in	Checks absence
"p" in "python"	# True
5 in [1, 2, 3]	# False

★ 6.8 Identity Operators

Operator	Meaning
is	Same object?
is not	Not same?
x = [1, 2, 3]	
y = [1, 2, 3]	
x is y	# False (different memory)
x == y	# True (values same)

★ 7. Expressions in Python

An expression is any combination of:

- variables
- values
- operators
- function calls

Example:

```
result = (10 + 5) * 2
```

★ 8. Operator Precedence

Order in which operations are performed.

Highest → Lowest:

1. () Parentheses
2. ** Exponent
3. * / // %
4. + -
5. Comparison operators
6. Logical not

- 7. Logical `and`
- 8. Logical `or`

Example:

```
x = 10 + 3 * 2
```

Result = 16, not 26.

★ 9. `eval()`, `repr()`, `str()`

`eval()`

Executes a string as Python code.

```
eval("10 + 5") # 15
```

`repr()`

Represents object as string with quotes.

`str()`

Converts object to human-readable string.

★ 10. Number System Conversions

Binary

```
bin(10) # '0b1010'
```

Hex

```
hex(255) # '0xff'
```

Octal

```
oct(8) # '0o10'
```

★ 11. Memory Management in Python

Python uses:

- ✓ Automatic Garbage Collection
- ✓ Reference Counting
- ✓ Memory Allocator
- ✓ Heap Storage

Example of reference count:

```
x = 100
y = x
```

Now the object 100 has at least 2 references.

★ 12. Mutable vs Immutable Types

Immutable Types

- int
- float
- bool
- string
- tuple
- frozenset

Any modification creates a new object.

Mutable Types

- list
- dict
- set

Modification happens in place.

★ 13. Errors Related to Data Types

1. TypeError

```
print(10 + "hello")
```

2. ValueError

```
int("abc") # invalid literal
```

3. OverflowError

Very rare in Python due to unlimited integer precision.

★ 14. Input & Output Formatting

Basic `print()`

```
print("Hello", "Python")
```

`sep` argument

```
print("Python", "Java", "C++", sep=" | ")
```

`end` argument

```
print("Hello", end=" ")  
print("World")
```

Formatted Strings (f-Strings)

```
name = "Hemanth"  
print(f"Hello {name}")
```

★ 15. Summary of Module 2

- Variables store references to objects
- Python is dynamically typed
- Data types: numeric, boolean, string, list, tuple, dict, set
- Type conversion (implicit & explicit)
- Operators: arithmetic, comparison, logical, assignment, bitwise, identity, membership
- Operator precedence defines evaluation order
- Python internally manages memory
- Mutable vs immutable types
- `print()`, `input()`, f-strings for I/O

MODULE 2 — Examples & Code Snippets (15 Examples)

★ Example 1: Basic Variable Assignment

```
x = 10
```

```
name = "Python"
pi = 3.14

print(x, name, pi)
```

Output:

```
10 Python 3.14
```

Explanation:

- Variables store data in memory
 - Python assigns the type automatically based on the value
 - No need to declare the type explicitly (unlike C, Java)
 - Each variable can hold different data types
-

★ Example 2: Multiple Assignment

```
a, b, c = 1, 2, 3
print(a, b, c)
```

Output:

```
1 2 3
```

Explanation:

- Assigning multiple variables in one line improves readability
 - This is called "tuple unpacking" or "parallel assignment"
 - All assignments happen simultaneously
 - Very useful for initializing related variables
-

★ Example 3: Swapping Variables (Pythonic Way)

```
x = 5
y = 10

x, y = y, x
print(x, y)
```

Output:

```
10 5
```

Explanation:

- No need for a third temporary variable
 - Python evaluates the right side first, then assigns to the left
 - This is a unique Python feature that makes code cleaner
 - Traditional swap in other languages requires: `temp = x; x = y; y = temp`
-

★ Example 4: Dynamic Typing Demonstration

```
x = 100
print(type(x))

x = "Python"
print(type(x))
```

Output:

```
<class 'int'>
<class 'str'>
```

Explanation:

- Python allows reassigning different data types to the same variable
 - This is called "dynamic typing"
 - The variable type changes based on the assigned value
 - In statically-typed languages like Java/C++, this would cause an error
-

★ Example 5: Checking Variable Type

```
value = 3.14
print(type(value))    # <class 'float'>
```

Output:

```
<class 'float'>
```

Explanation:

- `type()` is a built-in function that returns the object's data type
 - Useful for debugging and understanding what type of data you're working with
 - Returns the class/type of the object
 - Can be used with any Python object
-

★ Example 6: String, Integer & Float Input

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
```

```
height = float(input("Enter your height: "))

print(name, age, height)
```

Sample Input:

```
Enter your name: Hemanth
Enter your age: 21
Enter your height: 5.9
```

Output:

```
Hemanth 21 5.9
```

Explanation:

- `input()` always returns a string, regardless of what the user types
 - Convert to `int` or `float` using type constructors
 - Without conversion, mathematical operations would fail
 - Always validate user input in production code to handle invalid entries
-

★ Example 7: Implicit Type Conversion

```
x = 5          # int
y = 2.5        # float

result = x + y
print(result, type(result))
```

Output:

```
7.5 <class 'float'>
```

Explanation:

- Python automatically promotes `int` → `float` to prevent data loss
 - This is called "implicit type conversion" or "type coercion"
 - Python always converts to the more precise type
 - The integer 5 becomes 5.0 before addition
-

★ Example 8: Explicit Type Conversion

```
x = "100"
num = int(x)
print(num + 50)
```

Output:

Explanation:

- Manual conversion using type constructors is called "explicit conversion" or "type casting"
 - The string "100" is converted to integer 100
 - Without conversion, "100" + 50 would cause a `TypeError`
 - Only valid numeric strings can be converted (e.g., `int("abc")` raises `ValueError`)
-

★ Example 9: Arithmetic Operators

```
a = 10
b = 3

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)
print("Modulo:", a % b)
print("Power:", a ** b)
```

Output:

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Floor Division: 3
Modulo: 1
Power: 1000
```

Explanation:

- **Addition (+):** Sum of two numbers
 - **Subtraction (-):** Difference between numbers
 - **Multiplication (*):** Product of numbers
 - **Division (/):** Always returns a float, even if result is whole number
 - **Floor Division (//):** Returns only the integer part, discards decimal
 - **Modulo (%):** Returns the remainder after division
 - **Power (**):** Raises first number to the power of second ($10^3 = 1000$)
-

★ Example 10: Comparison Operators

```
print(10 > 3)
print(5 == 5)
print(7 != 2)
```

Output:

```
True
True
True
```

Explanation:

- Comparison operators return boolean values (True/False)
 - > checks if left is greater than right
 - == checks for equality (value comparison)
 - != checks for inequality (not equal)
 - These are fundamental for conditional statements and loops
-

★ Example 11: Logical Operators

```
x = True
y = False

print(x and y)
print(x or y)
print(not x)
```

Output:

```
False
True
False
```

Explanation:

- **and**: Returns True only if both conditions are True
 - **or**: Returns True if at least one condition is True
 - **not**: Reverses the boolean value (True → False, False → True)
 - Logical operators are essential for complex conditional logic
 - They short-circuit: **and** stops at first False, **or** stops at first True
-

★ Example 12: Membership Operators

```
text = "python"
print("py" in text)      # True
print("java" not in text) # True
```

Output:

```
True
True
```

Explanation:

- **in** checks if a substring/value exists in a sequence

- `not in` checks if a substring/value does NOT exist
 - Works with strings, lists, tuples, sets, and dictionaries (checks keys)
 - Case-sensitive for strings
 - Very useful for validation and filtering operations
-

★ Example 13: Identity vs Equality

```
x = [1, 2, 3]
y = [1, 2, 3]

print(x == y)    # True (values same)
print(x is y)    # False (different memory)
```

Output:

```
True
False
```

Explanation:

- `==` checks if values are equal (content comparison)
 - `is` checks if both variables point to the same object in memory (identity comparison)
 - `x` and `y` have the same content but are different objects in memory
 - Use `==` for value comparison, `is` for identity comparison (especially with `None`)
 - `is` is faster but only appropriate for checking object identity
-

★ Example 14: Operator Precedence

```
result = 10 + 3 * 2 ** 2
print(result)
```

Output:

```
22
```

Explanation (Step-by-Step):

1. $2 ** 2 = 4$ (Exponentiation has highest precedence)
2. $3 * 4 = 12$ (Multiplication comes next)
3. $10 + 12 = 22$ (Addition is last)

Precedence Order (High to Low):

- `()` Parentheses
- `**` Exponentiation
- `*`, `/`, `//`, `%` Multiplication, Division
- `+`, `-` Addition, Subtraction

Tip: Use parentheses for clarity: `result = 10 + (3 * (2 ** 2))`

★ Example 15: Mutable vs Immutable Types

```
# Immutable example (string)
name = "Python"
print("Original string ID:", id(name))

name = name + "3"
print("Modified string ID:", id(name))

print()

# Mutable example (list)
nums = [1, 2, 3]
print("Original list ID:", id(nums))

nums.append(4)
print("Modified list ID:", id(nums))
```

Output:

```
Original string ID: 140234567891234
Modified string ID: 140234567892456

Original list ID: 140234567893678
Modified list ID: 140234567893678
```

Explanation:

-

Immutable types (string, int, float, tuple, bool, frozenset):

-

- Cannot be changed after creation
- Any "modification" creates a new object → different memory address
- Strings concatenation creates new objects

-

Mutable types (list, dict, set):

-

- Can be modified in place
- Same object is updated → same memory address
- Lists can have items added, removed, or changed without creating new object

-

`id()` function returns the memory address of an object

-
-

Understanding mutability is crucial for avoiding bugs, especially with function arguments

-

Memory Impact:

- Immutable: More memory-intensive for frequent changes
- Mutable: Memory-efficient for large data structures that need updates

Additional Examples

Example 16: Assignment Operators

```
x = 10
print("Initial:", x)

x += 5    # x = x + 5
print("After +=:", x)

x -= 3    # x = x - 3
print("After -=:", x)

x *= 2    # x = x * 2
print("After *:=:", x)

x //= 4   # x = x // 4
print("After //=:", x)
```

Output:

```
Initial: 10
After +=: 15
After -=: 12
After *:=: 24
After //=: 6
```

Example 17: Boolean Type Conversion

```
# Truthy and Falsy values
print(bool(0))          # False
print(bool(1))          # True
print(bool(-5))         # True
print(bool(""))         # False (empty string)
print(bool("Python"))   # True
print(bool([]))         # False (empty list)
```

```
print(bool([1, 2]))      # True
print(bool(None))       # False
```

Output:

```
False
True
True
False
True
False
True
False
```

Explanation:

- **Falsy values:** 0, 0.0, empty string "", empty list [], empty dict {}, None, False
 - **Truthy values:** Everything else (non-zero numbers, non-empty collections)
 - Very useful in conditional statements
-

Example 18: String Formatting Comparison

```
name = "Hemanth"
age = 21
course = "Python"

# Method 1: Concatenation
print("Name: " + name + ", Age: " + str(age))

# Method 2: Multiple arguments
print("Name:", name, "Age:", age)

# Method 3: format() method
print("Name: {}, Age: {}".format(name, age))

# Method 4: f-strings (recommended)
print(f"Name: {name}, Age: {age}, Course: {course}")
```

Output:

```
Name: Hemanth, Age: 21
Name: Hemanth Age: 21
Name: Hemanth, Age: 21
Name: Hemanth, Age: 21, Course: Python
```

Explanation:

- f-strings are the most modern and readable approach (Python 3.6+)
 - They allow expressions inside { }
 - Fastest and most Pythonic method
-

Practice Tips

1. **Experiment:** Try modifying values and observe the output
2. **Type checking:** Use `type()` frequently to understand data types
3. **Memory:** Use `id()` to understand object identity
4. **Errors:** Intentionally cause errors to learn from them
5. **Documentation:** Add comments to explain your understanding

Happy Learning!

MODULE 2 — LECTURES

Variables, Data Types & Operators

This module contains 6 detailed lectures.

★ Lecture 1 — Variables in Python

Topic

"Understanding Variables, Storage, Naming Rules, and Assignments"

Definition (Detailed)

A variable in Python is a symbolic name that refers to a value stored in memory.

Variables do not store data directly; instead, they store references to objects created in memory.

Python uses dynamic typing, meaning variable types are decided at runtime.

Variables are created automatically when a value is assigned—no declaration is needed.

Variable names act as identifiers and must follow Python naming rules (no starting with digits, no spaces, no keywords).

Python variables are case-sensitive (`Name` and `name` are different).

Syntax

```
variable_name = value
```

Examples:

```
x = 10
name = "Python"
pi = 3.14
```

Examples

Example 1 — Basic Assignment

```
x = 5
print(x)
```

Example 2 — Multiple Assignment

```
a, b, c = 1, 2, 3
print(a, b, c)
```

Example 3 — Same Value Assignment

```
x = y = z = 100
```

Example 4 — Variable Name Rules

Valid:

```
student_name = "Ravi"
roll12 = 52
_total = 500
```

Invalid:

```
2name = "John"    # starts with digit
my name = "Ravi"  # space not allowed
class = 10        # keyword
```

Key Takeaways

- Variables reference memory locations.
- Naming rules must be followed.
- Python is dynamically typed.
- Multiple assignments are allowed.
- Variables can be reassigned to new values any time.

Practice Section

ready_to_practice: You understand how to create and use variables.

description: Try assigning different types of values to variables.

mcqs: Attempt MCQs 1–3 from Module 2.

coding_challenges: Create three variables: name, age, marks → print all in one line.

★ Lecture 2 — Data Types in Python

Topic

"Built-In Data Types: Numeric, Boolean, String, List, Tuple, Set, Dict, NoneType"

Definition (Detailed)

A data type defines the type of value a variable stores and what operations can be performed on it.

Python has several built-in data types categorized as:

- **Numeric** (int, float, complex)
- **Boolean**
- **Sequence types** (str, list, tuple, range)
- **Mapping types** (dict)
- **Set types** (set, frozenset)
- **NoneType**

Python automatically infers data types based on assigned values.

Data types help the interpreter allocate memory and optimize operations.

Python's rich data types make it extremely flexible and expressive.

Syntax

```
x = 10          # int
pi = 3.14       # float
name = "John"   # string
marks = [10, 20, 30] # list
```

Examples

Example 1 — Numeric Types

```
x = 10
y = 3.14
z = 3 + 4j

print(type(x), type(y), type(z))
```

Example 2 — Boolean Type

```
is_logged = True
print(is_logged)
```

Example 3 — String Type

```
msg = "Python is fun!"  
print(msg)
```

Example 4 — List Type

```
numbers = [1, 2, 3, 4]  
print(numbers)
```

Example 5 — Dictionary

```
student = {"name": "Hemanth", "age": 21}  
print(student["name"])
```

Key Takeaways

- Python supports many built-in data types.
- Strings are immutable; lists are mutable.
- Dictionaries store key–value pairs.
- Booleans are subclasses of integers.
- Knowing data types helps in choosing correct operations.

Practice Section

ready_to_practice: You can now identify and use Python data types.

description: Try creating variables of each data type.

mcqs: Attempt MCQs 4–6.

coding_challenges: Create a dict storing your info and print it.

★ Lecture 3 — Type Conversion (Casting)

Topic

"Implicit & Explicit Conversion, int(), float(), str(), bool()"

Definition (Detailed)

Type conversion allows converting one data type into another.

Implicit conversion is done automatically by Python (e.g., $\text{int} + \text{float} \rightarrow \text{float}$).

Explicit conversion requires constructors like:

- `int()`

- `float()`
- `str()`
- `bool()`

Conversion ensures compatibility between data types during operations.

Python prevents invalid conversions (e.g., `int("abc")` raises an error).

Syntax

```
x = int("10")
y = float("5.5")
z = str(100)
flag = bool(1)
```

Examples

Example 1 — Implicit Conversion

```
x = 5      # int
y = 2.5    # float
print(x + y)
```

Example 2 — Explicit Conversion

```
num = int("50")
print(num + 10)
```

Example 3 — String to Float Conversion

```
x = float("3.14")
```

Example 4 — Boolean Conversion

```
bool(0)      # False
bool(10)     # True
```

Key Takeaways

- Use casting functions for safe conversions.
- Implicit conversion happens automatically.
- Strings must contain numeric characters for `int()` conversion.
- `bool()` returns `False` only for: `0`, `""`, `None`, `[]`, `{}`, `()`.

Practice Section

ready_to_practice: You can convert values between different types.

description: Practice converting user input into numbers.

mcqs: Attempt MCQs 7–9.

coding_challenges: Ask user for 2 numbers and print sum as integer.

★ Lecture 4 — Operators in Python

Topic

"Arithmetic, Comparison, Logical, Assignment, Bitwise, Identity, Membership Operators"

Definition (Detailed)

Operators are symbols used to perform specific operations on variables and values.

Python operators are categorized as:

- **Arithmetic** → +, -, *, /, //, %, **
- **Comparison** → ==, !=, >, <, >=, <=
- **Logical** → and, or, not
- **Assignment** → =, +=, -=, etc.
- **Bitwise** → &, |, ^, <<, >>
- **Identity** → is, is not
- **Membership** → in, not in

Operators return different types of results depending on the operation (boolean, numeric, etc.).

Operators follow precedence rules to evaluate expressions in correct order.

Logical and comparison operators form the basis of decision-making in programs.

Syntax Example

```
a = 10
b = 3
print(a + b)
print(a > b)
print(a and b)
```

Examples

Example 1 — Arithmetic Operators

```
x = 15
y = 4
print(x // y)
print(x % y)
print(x ** 2)
```

Example 2 — Comparison Operators

```
print(10 > 3)
print(5 == 5)
print(7 != 2)
```

Example 3 — Logical Operators

```
print(True and False)
print(True or False)
print(not False)
```

Example 4 — Membership Operators

```
print('py' in 'python')
print(5 not in [1, 2, 3])
```

Example 5 — Identity Operators

```
x = [1, 2, 3]
y = [1, 2, 3]
print(x is y)      # False
print(x == y)     # True
```

Key Takeaways

- Operators help in computations and logic building.
- Logical operators combine boolean expressions.
- Membership checks for existence.
- Identity checks if two variables refer to same object.

Practice Section

ready_to_practice: You understand all major operator types.

description: Practice expression evaluations.

mcqs: Attempt MCQs 10–12.

coding_challenges: Create a mini calculator using arithmetic operators.

★ Lecture 5 — Expressions & Operator Precedence

Topic

"Expressions, Order of Evaluation, Associativity, Parentheses Usage"

Definition (Detailed)

An expression is a combination of values, variables, and operators that evaluates to a single value.

Python follows strict precedence rules that define which operator executes first.

When operators have the same precedence, Python uses left-to-right associativity (except exponent).

Parentheses `()` override all precedence rules.

Understanding precedence is crucial for avoiding bugs in calculations.

Syntax Example

```
result = 10 + 3 * 2
```

Examples

Example 1 — Precedence Demonstration

```
x = 10 + 3 * 2
print(x)    # 16
```

Example 2 — Parentheses Override

```
x = (10 + 3) * 2
print(x)    # 26
```

Example 3 — Complex Expression

```
result = 10 + 3 * 2 ** 2 - 5
print(result)
```

Step-by-step evaluation:

1. $2 ** 2 = 4$ (Exponentiation first)
2. $3 * 4 = 12$ (Multiplication)
3. $10 + 12 = 22$ (Addition)
4. $22 - 5 = 17$ (Subtraction)

Key Takeaways

- `()` has highest priority.
- `**` is evaluated before `*` / `%` / `/`.
- Comparison happens after arithmetic.
- `and` evaluates before `or`.

Precedence Order (Highest to Lowest):

1. `()` — Parentheses
2. `**` — Exponentiation
3. `*`, `/`, `//`, `%` — Multiplication, Division
4. `+`, `-` — Addition, Subtraction
5. `==`, `!=`, `>`, `<`, `>=`, `<=` — Comparison
6. `not` — Logical NOT
7. `and` — Logical AND
8. `or` — Logical OR

Practice Section

ready_to_practice: You can now evaluate expressions confidently.

description: Try rewriting expressions using parentheses.

mcqs: Attempt MCQs 13–15.

coding_challenges: Evaluate $(a+b)^2$ using Python precedence.

★ Lecture 6 — Mutable vs Immutable Types

Topic

"Difference in Memory Behavior, `id()`, mutability, object identity"

Definition (Detailed)

A **mutable object** can be modified after creation (list, dict, set).

An **immutable object** cannot be modified after creation (int, float, bool, string, tuple).

Modifying an immutable object creates a new object with a new memory reference.

Modifying a mutable object updates the existing memory location.

`id()` function returns the memory address (identity) of an object.

Syntax Example

```
id(variable)
```

Examples

Example 1 — Immutable Behavior

```
x = "Python"
```

```
print(id(x))

x = x + "3"
print(id(x))
```

Output:

```
140234567891234
140234567892456 # Different ID
```

Example 2 — Mutable Behavior

```
nums = [1, 2, 3]
print(id(nums))

nums.append(4)
print(id(nums))
```

Output:

```
140234567893678
140234567893678 # Same ID
```

Key Takeaways

- Strings, ints → immutable
- Lists, dicts → mutable
- Immutable modification creates new object
- Mutable modification changes same object

Summary Table:

Immutable Types	Mutable Types
-----------------	---------------

int	list
float	dict
bool	set
string	bytearray
tuple	
frozenset	

Practice Section

ready_to_practice: You understand memory behavior in Python.

description: Try modifying strings and lists to observe memory id changes.

mcqs: Covered in higher-level MCQs.

coding_challenges: Demonstrate mutability/immutability using `id()`.

Module 2 Lecture Summary

What You've Learned:

- ✓ **Lecture 1:** Variables - naming, assignment, and references
- ✓ **Lecture 2:** Data types - numeric, boolean, string, collections
- ✓ **Lecture 3:** Type conversion - implicit and explicit casting
- ✓ **Lecture 4:** Operators - all 7 categories with examples
- ✓ **Lecture 5:** Expressions and operator precedence rules
- ✓ **Lecture 6:** Mutable vs immutable types and memory behavior

Skills Acquired:

- Creating and naming variables properly
- Understanding all Python data types
- Converting between data types safely
- Using all operator categories effectively
- Evaluating complex expressions correctly
- Understanding memory management and object identity

MODULE 2 - MCQs (30 Questions With Answers + Explanations)

★ MCQ 1. Which of the following is a valid variable name in Python?

- a) 2value
- b) value_2
- c) value-2
- d) value 2

Answer: b) value_2

Explanation:

Python variable naming rules:

- Must start with a letter (a-z, A-Z) or underscore (`_`)
- Cannot start with a digit
- Can contain letters, digits, and underscores
- Cannot contain spaces or special characters like hyphens

Therefore: `2value` starts with digit (invalid), `value-2` has hyphen (invalid), `value 2` has space (invalid), but `value_2` follows all rules (valid).

★ MCQ 2. Python variables are:

- a) Statically typed
- b) Dynamically typed
- c) Not typed
- d) Strongly typed

Answer: b) Dynamically typed

Explanation:

Python uses dynamic typing, meaning you don't need to declare variable types explicitly. The type is determined at runtime based on the assigned value. You can reassign a variable to a different type:

```
x = 10          # x is int
x = "hello"     # now x is str (allowed in Python)
```

★ MCQ 3. What will be the type of variable x?

```
x = 3.5
```

- a) int
- b) float
- c) decimal
- d) real

Answer: b) float

Explanation:

Any number with a decimal point in Python is automatically of type `float`. The `decimal` type requires importing the decimal module, and `real` is not a Python data type.

★ MCQ 4. Which function returns the type of a variable?

- a) `datatype()`
- b) `typeof()`
- c) `type()`
- d) `check()`

Answer: c) `type()`

Explanation:

The built-in `type()` function returns the data type of any object in Python.

```
x = 10
print(type(x)) # <class 'int'>
```

★ MCQ 5. What is the output?

```
a = 10
b = 3
print(a / b)
```

- a) 3
- b) 3.333...
- c) 3.0
- d) Error

Answer: b) 3.333...

Explanation:

In Python 3, the `/` operator always performs true division and returns a float, even when dividing two integers. The result is $10/3 = 3.333333...$ (repeating decimal).

★ MCQ 6. What does floor division (`//`) return?

- a) Decimal value
- b) Fraction
- c) Rounded down integer
- d) Absolute value

Answer: c) Rounded down integer

Explanation:

The `//` operator performs floor division, which divides and rounds down to the nearest integer.

```
10 // 3 # Returns 3 (not 3.33...)
7 // 2  # Returns 3 (not 3.5)
```

★ MCQ 7. Which of the following is immutable?

- a) list
- b) dictionary

- c) set
- d) tuple

Answer: d) tuple

Explanation:

Tuples are immutable, meaning once created, their elements cannot be changed, added, or removed. Lists, dictionaries, and sets are mutable (can be modified after creation).

```
t = (1, 2, 3)
t[0] = 10 # Error: tuples don't support item assignment
```

★ MCQ 8. What will this print?

```
print(type(True))
```

- a) <class 'bool'>
- b) <class 'int'>
- c) <class 'boolean'>
- d) <class 'str'>

Answer: a) <class 'bool'>

Explanation:

`True` and `False` are boolean values in Python, and their type is `bool`. Note: `bool` is a subclass of `int`, so `True == 1` and `False == 0`, but the type is still `bool`.

★ MCQ 9. Identify the data type:

```
x = {"a": 1, "b": 2}
```

- a) set
- b) dict
- c) tuple
- d) list

Answer: b) dict

Explanation:

Curly braces `{ }` with key-value pairs (using colons) define a dictionary. Sets use curly braces without colons, tuples use parentheses, and lists use square brackets.

★ MCQ 10. Which of the following is NOT a numeric type?

- a) int
- b) float
- c) complex
- d) char

Answer: d) char

Explanation:

Python has three numeric types: `int`, `float`, and `complex`. Python does not have a `char` type; single characters are simply strings of length 1.

★ MCQ 11. What is the output?

```
print(10 == "10")
```

- a) True
- b) False
- c) Error
- d) None

Answer: b) False

Explanation:

Python's `==` operator checks for value equality but also considers type. Since `10` is an integer and `"10"` is a string, they are not equal. Python does not perform automatic type conversion for equality checks.

★ MCQ 12. Which operator has highest precedence?

- a) +
- b) *
- c) **
- d) //

****Answer: c) ****

Explanation:

In Python's operator precedence, exponentiation `**` has the highest precedence among arithmetic operators, followed by `*`, `/`, `//`, `%`, then `+` and `-`.

2 + 3 ** 2 # Result: 11 (not 25), because ** is evaluated first

★ MCQ 13. What is the output?

```
print(5 and 0)
```

- a) 5
- b) 0
- c) True
- d) False

Answer: b) 0

Explanation:

The `and` operator returns the first falsy value or the last value if all are truthy. Here, 5 is truthy, so it checks 0, which is falsy, and returns 0. It doesn't convert to boolean True/False.

★ MCQ 14. What is the output?

```
print(5 or 0)
```

- a) 5
- b) 0
- c) True
- d) False

Answer: a) 5

Explanation:

The `or` operator returns the first truthy value or the last value if all are falsy. Since 5 is truthy, it returns 5 immediately without evaluating 0.

★ MCQ 15. The expression `not 0` returns:

- a) 0
- b) 1
- c) True
- d) False

Answer: c) True

Explanation:

The `not` operator returns a boolean. Since `0` is falsy in Python, `not 0` returns `True` (not the integer `1`, but the boolean `True`).

★ MCQ 16. What does this return?

```
bool("")
```

- a) True
- b) False
- c) Error

Answer: b) False

Explanation:

An empty string `""` is considered falsy in Python. When converted to boolean using `bool()`, it returns `False`. Other falsy values include `0`, `None`, empty lists `[]`, empty tuples `()`, and empty dicts `{}`.

★ MCQ 17. Which operator checks identity?

- a) `==`
- b) `!=`
- c) `is`
- d) `===`

Answer: c) is

Explanation:

The `is` operator checks if two variables refer to the same object in memory (identity), while `==` checks if values are equal. Python doesn't have a `===` operator like JavaScript.

```
a = [1, 2]
b = [1, 2]
a == b # True (same values)
a is b # False (different objects)
```

★ MCQ 18. What is the output?

```
x = [1,2]
y = [1,2]
print(x is y)
```

- a) True
- b) False

Answer: b) False

Explanation:

Although `x` and `y` have the same values, they are two separate list objects in memory. The `is` operator checks object identity (memory address), not value equality. `x == y` would return `True`, but `x is y` returns `False`.

★ MCQ 19. Which is a membership operator?

- a) `is`
- b) `in`
- c) `not`
- d) `==`

Answer: b) `in`

Explanation:

The `in` operator checks if a value exists within a sequence (string, list, tuple, set, etc.). Its counterpart is `not in`.

```
"a" in "cat"      # True
5 in [1, 2, 3]    # False
```

★ MCQ 20. Predict the output:

```
print(10 // 3)
```

- a) 3.33
- b) 3
- c) 4
- d) Error

Answer: b) 3

Explanation:

Floor division `//` divides and rounds down to the nearest integer. 10 divided by 3 is 3.333..., which rounds down to 3.

★ MCQ 21. What is the result of:

```
int(3.99)
```

- a) 3
- b) 4
- c) 3.99
- d) Error

Answer: a) 3

Explanation:

The `int()` function truncates (cuts off) the decimal part; it does not round. So `int(3.99)` returns 3, not 4. To round, use `round(3.99)` which would give 4.

★ MCQ 22. What is the output?

```
print("5" + "10")
```

- a) 15
- b) 510
- c) 5 10
- d) Error

Answer: b) 510

Explanation:

The `+` operator concatenates (joins) strings. Since both `"5"` and `"10"` are strings, they are joined together to form `"510"`, not added mathematically.

★ MCQ 23. What is the output?

```
print(5 + int("10"))
```

- a) 15
- b) 510
- c) Error

Answer: a) 15

Explanation:

`int("10")` converts the string `"10"` to the integer 10. Then `5 + 10` performs mathematical addition, resulting in 15.

★ MCQ 24. Which conversion is invalid?

- a) `int("20")`
- b) `float("2.5")`
- c) `int("ABC")`
- d) `str(50)`

Answer: c) `int("ABC")`

Explanation:

You cannot convert a non-numeric string to an integer. `int("ABC")` will raise a `ValueError` because "ABC" doesn't represent a valid number. The other conversions are all valid.

★ MCQ 25. What does `5 % 2` give?

- a) 2
- b) 1
- c) 5
- d) 0

Answer: b) 1

Explanation:

The modulo operator `%` returns the remainder of division. 5 divided by 2 equals 2 with a remainder of 1, so `5 % 2 = 1`.

★ MCQ 26. What is the output?

```
print(2 ** 3 ** 2)
```

- a) 64
- b) 512
- c) 256

Answer: b) 512

Explanation:

Exponentiation `**` is right-associative, meaning it evaluates from right to left. So this evaluates as `2 ** (3 ** 2)`:

- First: `3 ** 2 = 9`
- Then: `2 ** 9 = 512`

If it were left-associative, it would be `(2 ** 3) ** 2 = 8 ** 2 = 64`.

★ MCQ 27. Which returns True?

- a) 5 in 12345
- b) "a" in "java"
- c) 2 in [3,4,5]

Answer: b) "a" in "java"

Explanation:

- Option a: `5 in 12345` is invalid because `in` checks membership in sequences, and `12345` is an integer, not a sequence.
 - Option b: `"a" in "java"` returns `True` because the character "a" exists in the string "java".
 - Option c: `2 in [3, 4, 5]` returns `False` because 2 is not in the list.
-

★ MCQ 28. Which shows mutable behavior?

```
x = [1, 2, 3]
x.append(4)
```

- a) Mutable
- b) Immutable

Answer: a) Mutable

Explanation:

Lists are mutable, meaning they can be modified after creation. The `append()` method adds an element to the existing list without creating a new object. After this operation, `x` becomes `[1, 2, 3, 4]`.

★ MCQ 29. Which shows immutable behavior?

```
s = "Python"
s = s + "3"
```

- a) Mutable
- b) Immutable

Answer: b) Immutable

Explanation:

Strings are immutable in Python. When you do `s = s + "3"`, Python doesn't modify the original string. Instead, it creates a new string object `"Python3"` and assigns it to `s`. The original string `"Python"` remains unchanged in memory.

★ MCQ 30. What is the output?

```
print(type(None))
```

- a) <class 'none'>
- b) <class 'None'>
- c) <class 'NoneType'>
- d) <class 'Null'>

Answer: c) <class 'NoneType'>

Explanation:

None is a special constant in Python representing the absence of a value. Its type is NoneType. Note that Python is case-sensitive: None (capitalized) is the keyword, not null or none.

MODULE 2 — CODING QUESTIONS (20 Tasks)

★ Task 1 — Type Identification

Problem: Take input from user and print its type.

Solution:

```
user_input = input("Enter something: ")
print(f"Type: {type(user_input)}")
print(f"Value: {user_input}")
```

Note: input() always returns a string, so the type will always be <class 'str'>.

★ Task 2 — Swap Two Variables

Problem: Swap two numbers using Pythonic assignment.

Solution:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

print(f"Before swap: a = {a}, b = {b}")
```

```
a, b = b, a
print(f"After swap: a = {a}, b = {b}")
```

★ Task 3 — Simple Calculator

Problem: Take two numbers and print sum, difference, product, quotient, modulo, and power.

Solution:

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print(f"Sum: {num1 + num2}")
print(f"Difference: {num1 - num2}")
print(f"Product: {num1 * num2}")
print(f"Quotient: {num1 / num2}")
print(f"Modulo: {num1 % num2}")
print(f"Power: {num1 ** num2}")
```

★ Task 4 — Find Area of Circle

Problem: Calculate area of circle using formula: $\text{area} = \pi * r * r$

Solution:

```
import math

radius = float(input("Enter radius: "))
area = math.pi * radius * radius
print(f"Area of circle: {area:.2f}")
```

★ Task 5 — Check if Number is Positive/Negative/Zero

Problem: Using comparison operators to check number status.

Solution:

```
num = float(input("Enter a number: "))

if num > 0:
    print("Positive")
elif num < 0:
    print("Negative")
else:
    print("Zero")
```

★ Task 6 — Convert Fahrenheit to Celsius

Problem: Use formula: $C = (F - 32) * 5/9$

Solution:

```
fahrenheit = float(input("Enter temperature in Fahrenheit: "))
celsius = (fahrenheit - 32) * 5/9
print(f"{fahrenheit}° F = {celsius:.2f}° C")
```

★ Task 7 — Using Type Casting

Problem: Take input as string, convert to int, print double of it.

Solution:

```
user_input = input("Enter a number: ")
number = int(user_input)
double = number * 2
print(f"Double of {number} is {double}")
```

★ Task 8 — Evaluate Expression

Problem: Compute $result = (a + b) ** 2$

Solution:

```
a = float(input("Enter value of a: "))
b = float(input("Enter value of b: "))
result = (a + b) ** 2
print(f"(a + b)2 = {result}")
```

★ Task 9 — Check Membership

Problem: Take a string and check whether "a" exists in it.

Solution:

```
text = input("Enter a string: ")
if "a" in text:
    print("'a' is present in the string")
else:
    print("'a' is not present in the string")
```

★ Task 10 — Logical Operator Test

Problem: Print "Adult" if age > 18 AND user answered "yes" to "Are you a student?".

Solution:

```
age = int(input("Enter your age: "))
is_student = input("Are you a student? (yes/no): ").lower()

if age > 18 and is_student == "yes":
    print("Adult Student")
else:
    print("Not an Adult Student")
```

★ Task 11 — Compare Two Numbers

Problem: Take two numbers and print the larger one.

Solution:

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

if num1 > num2:
    print(f"Larger number: {num1}")
elif num2 > num1:
    print(f"Larger number: {num2}")
else:
    print("Both numbers are equal")
```

★ Task 12 — Remainder Calculator

Problem: Take two numbers and print the modulo result.

Solution:

```
num1 = int(input("Enter dividend: "))
num2 = int(input("Enter divisor: "))
remainder = num1 % num2
print(f"Remainder: {remainder}")
```

★ Task 13 — Square & Cube Calculator

Problem: Take a number and print its square and cube.

Solution:

```
num = float(input("Enter a number: "))
square = num ** 2
cube = num ** 3
print(f"Square: {square}")
print(f"Cube: {cube}")
```

★ Task 14 — Identity Check

Problem: Create two lists with same values and check equality and identity.

Solution:

```
list1 = [1, 2, 3]
list2 = [1, 2, 3]

print(f"Are they equal? {list1 == list2}")
print(f"Are they identical? {list1 is list2}")
print(f"ID of list1: {id(list1)}")
print(f"ID of list2: {id(list2)}")
```

★ Task 15 — Data Type Converter App

Problem: Ask user for input and print it as int, float, bool, and string.

Solution:

```
value = input("Enter a value: ")

print(f"As int: {int(float(value))}") # float() first handles
decimal inputs
print(f"As float: {float(value)}")
print(f"As bool: {bool(value)}")
print(f"As string: {str(value)}")
```

★ Task 16 — Floor Division Practice

Problem: Take two numbers and print the floor division result.

Solution:

```
num1 = float(input("Enter dividend: "))
num2 = float(input("Enter divisor: "))
result = num1 // num2
print(f"Floor division result: {result}")
```

★ Task 17 — Bitwise Operator Practice

Problem: Print results of bitwise AND, OR, and XOR.

Solution:

```
a = int(input("Enter first number: "))
```

```
b = int(input("Enter second number: "))

print(f"a & b (AND): {a & b}")
print(f"a | b (OR): {a | b}")
print(f"a ^ b (XOR): {a ^ b}")
```

★ Task 18 — Temperature Conversion

Problem: Convert Celsius to Fahrenheit using: $F = C * 9/5 + 32$

Solution:

```
celsius = float(input("Enter temperature in Celsius: "))
fahrenheit = celsius * 9/5 + 32
print(f"{celsius}° C = {fahrenheit:.2f}° F")
```

★ Task 19 — Expression Precedence

Problem: Compute $(10 + 3 * 2 ** 2) / 5$

Solution:

```
result = (10 + 3 * 2 ** 2) / 5
print(f"Result: {result}")

# Step-by-step breakdown:
# 2 ** 2 = 4
# 3 * 4 = 12
# 10 + 12 = 22
# 22 / 5 = 4.4
```

★ Task 20 — Mutable/Immutable Test

Problem: Demonstrate that strings are immutable and lists are mutable using `id()`.

Solution:

```
# Immutable: String
s = "Python"
print(f"Original string ID: {id(s)}")
s = s + "3"
print(f"Modified string ID: {id(s)}")
print("String is immutable (ID changed)\n")

# Mutable: List
lst = [1, 2, 3]
print(f"Original list ID: {id(lst)}")
lst.append(4)
print(f"Modified list ID: {id(lst)}")
print("List is mutable (ID unchanged)")
```

MODULE 3 THEORY — Control Flow Statements in Python

(3500+ words, complete and deeply structured)

★ 1. Introduction to Control Flow in Python

In every real-world program, execution does not simply run from top to bottom. Programs need to make decisions, repeat tasks, choose between alternative paths, and control the direction in which instructions execute.

This ability to guide the execution of instructions is known as **control flow**.

Python provides powerful constructs to control program flow:

Conditional Statements

- `if`
- `if-else`
- `if-elif-else`
- Nested decision blocks

Looping Statements

- `for`
- `while`

Loop Control Statements

- `break`
- `continue`
- `pass`

Range function for iteration control

Boolean expressions — conditions inside decisions and loops

Control flow statements allow the program to become dynamic, interactive, and logical, enabling solutions to complex problems.

★ 2. Boolean Logic — The Foundation of Control Flow

Before understanding control flow statements, we must understand **Boolean logic**.

A Boolean expression evaluates to:

- `True`
- `False`

These values form the backbone of every decision-making mechanism in programming.

★ 2.1 Boolean Expressions

Examples of boolean expressions:

Comparisons:

```
5 > 3
10 == 10
"Python" != "Java"
```

Logical operations:

```
True and False
10 > 5 or 3 > 4
```

Boolean expressions are evaluated inside:

- `if` conditions
- `while` loops
- loop controls
- decision-making algorithms
- data validation
- filtering logic

Python treats the following as False:

- `0`
- `0.0`
- `0j`
- `""` (empty string)
- `[]`
- `{}`
- `set()`
- `None`
- `False`

Everything else evaluates to `True`.

★ 3. Conditional Statements (Decision Making)

Conditional statements allow the programmer to execute certain blocks of code depending on conditions.

Python supports:

- `if`
- `if-else`
- `if-elif-else`
- nested `if`
- multiple conditional expressions

★ 3.1 The `if` Statement

The simplest decision-making statement.

Syntax:

```
if condition:
    statement-block
```

- The condition must evaluate to `True` or `False`.
- If the condition is `True` → statement block executes.
- If `False` → block is skipped.

Indentation is critical since Python uses it to define code blocks.

★ 3.2 The `if-else` Statement

Adds an alternate block that executes when the condition is `False`.

Syntax:

```
if condition:
    block1
else:
    block2
```

Real-world use cases:

- Login validation
- Odd-even checks
- Minimum/maximum comparisons
- Error messages
- Status messages

★ 3.3 The `if-elif-else` Ladder

Used when there are multiple conditions.

Syntax:

```
if condition1:
    block1
elif condition2:
    block2
elif condition3:
    block3
else:
    blockN
```

Key points:

- Only one block executes.
- Python checks conditions sequentially.
- Once a `True` condition is found, the rest are ignored.

Use cases:

- Grading systems
- Menu-driven programs
- Age or salary classifications
- Tier-based logic

★ 3.4 Nested if Statements

Indicated when decisions depend on previous decisions.

Syntax:

```
if condition1:
    if condition2:
        block1
    else:
        block2
else:
    block3
```

Use cases:

- Bank authentication (username → password → OTP)
- Multi-level decision trees
- Classification systems

Nested conditions should be used carefully to avoid unnecessary complexity.

★ 3.5 Conditional Expressions (Ternary Operator)

Python supports a short form of if-else.

Syntax:

```
value_if_true if condition else value_if_false
```

Used for compact decisions:

- Finding max/min
 - Quick checks
 - Assigning values based on conditions
-

★ 4. Looping Statements

Loops allow a program to repeat a block of code multiple times.

Python supports:

- `for` loop
- `while` loop

Loops reduce redundancy and improve automation.

★ 4.1 The `for` Loop

The `for` loop is the most widely used looping construct in Python.

Python's `for` loop is **iterator-based**, meaning it iterates over:

- sequences (list, tuple, string)
- ranges
- dictionaries
- sets
- files
- custom iterators

General Syntax:

```
for variable in iterable:  
    block of code
```

The loop variable takes values from the iterable sequentially.

★ For Loop with Different Iterables

1. Looping through a list

```
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit)
```

2. Looping through a string

```
for char in "Python":  
    print(char)
```

3. Looping through a dictionary

```
person = {"name": "Alice", "age": 25}
for key in person:
    print(key, person[key])
```

4. Looping through a tuple

```
numbers = (1, 2, 3, 4, 5)
for num in numbers:
    print(num)
```

5. Looping through range()

```
for i in range(5):
    print(i)
```

★ 4.1.1 Iteration Order & Behavior

- `for` loop automatically stops at the end of the iterable.
- No index is needed (but available if required).
- Python loops are clean and highly readable.

★ 4.2 The `while` Loop

A `while` loop executes as long as a condition remains `True`.

Syntax:

```
while condition:
    block
```

Use cases:

- Repetitive input until valid
- Game loops
- Timers
- Infinite loops (with exit conditions)

Key precautions:

- Ensure condition eventually becomes `False`
- Avoid infinite loops unless intended

★ 4.3 Loop Control Statements

Python provides statements to control loop flow:

- ✓ **`break`** — Exits the loop immediately.
- ✓ **`continue`** — Skips the current iteration and moves to next.
- ✓ **`pass`** — Does nothing; used as a placeholder.

★ 4.3.1 break Statement

Used to stop a loop prematurely.

Examples:

- exit when a match is found
- stop when a user enters a special value
- stop after 3 attempts

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

★ 4.3.2 continue Statement

Used to skip certain iterations.

Common uses:

- skip invalid input
- skip negative values
- ignore blank lines

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

★ 4.3.3 pass Statement

Used as a placeholder when syntax expects a statement but logic is yet to be implemented.

Example use cases:

- empty functions
- empty classes
- empty loops
- stubs for future code

```
for i in range(5):  
    pass # TODO: implement later
```

★ 5. The range() Function in Detail

The `range()` function generates a sequence of numbers.

Syntax 1: range(stop)

`range(5) → 0 1 2 3 4`

Syntax 2: `range(start, stop)`

`range(2, 6) → 2 3 4 5`

Syntax 3: `range(start, stop, step)`

`range(1, 10, 2) → 1 3 5 7 9`

★ Characteristics of `range()`

- Generates numbers efficiently
 - Used heavily in `for` loops
 - Step can be negative
 - Does not support float steps
 - Produces an immutable sequence
-

★ 6. Nested Loops

Nested loops are loops inside loops.

Used for:

- patterns
- matrices
- tables
- nested data structures
- multi-dimensional logic

Example nested loop structure:

```
for i in range():  
    for j in range():  
        ...
```

Performance note: nested loops increase time complexity.

★ 7. Patterns & Loop Logic

Pattern printing is the most common use of nested loops in beginner Python.

Patterns help learners understand:

- nested control flow
- loop indexing

- indentation
- structure-based iteration

Examples include:

- stars pattern
 - pyramids
 - squares
 - number patterns
-

★ 8. Infinite Loops

An infinite loop runs forever unless manually stopped.

Example condition:

```
while True:
    ...
```

Used for:

- game loops
- event listeners
- live applications

To avoid logical bugs, infinite loops must:

- include a `break` condition
 - or include user-controlled exit input
-

★ 9. The `else` Block in Loops (Unique Python Feature)

Python uniquely supports `else` with loops.

For `for` loops — `else` executes when the loop completes naturally (without `break`).

For `while` loops — `else` executes when condition becomes `False`.

Syntax:

```
for item in items:
    ...
else:
    # execute if no break occurred
```

Use cases:

- searching in a list
- detecting absence of an item
- finalizing after a loop

★ 10. Comparison Between if, for, while

Feature	if	for	while
Purpose	decision making	iteration over iterable	repetition based on condition
Number of executions	0 or 1	depends on iterable	depends on condition
Can become infinite	no	no	yes
Controls flow	branching	looping	looping

Choosing between loops:

- Use `for` for known iteration counts.
- Use `while` for conditional repetition.

★ 11. Indentation & Block Structure

Since Python uses indentation instead of braces:

- Indentation signals a block
- Nested logic requires deeper indentation
- Mistakes cause `IndentationError`

Programming style guidelines:

- Use 4 spaces (PEP8 standard)
- Forbidden to mix tabs & spaces
- Maintain consistent indentation

★ 12. Error Handling in Control Flow

Some common issues:

✗ 12.1 IndentationError

Occurs when indentation is incorrect.

✗ 12.2 SyntaxError

Occurs when `if/for/while` is written incorrectly.

✗ 12.3 Infinite Loops

Occurs when `while` condition never becomes `False`.

✗ 12.4 Logic Errors

Even if code runs, the logic may be wrong (e.g., using `=` instead of `==`).

★ 13. Real-World Applications of Control Flow

✓ Decision Making

- user authentication
- eligibility checks
- form validation
- grading systems

✓ Looping

- reading files line-by-line
- processing database rows
- printing patterns
- checking prime numbers

✓ Combined

- ATM logic
- menu-driven systems
- games
- simulations
- automation scripts

Every Python application uses control flow heavily.

★ 14. Best Practices for Control Flow

- Write readable conditions
 - Use parentheses for complex conditions
 - Avoid deep nested loops (prefer breaking logic into functions)
 - Use descriptive variable names
 - Avoid infinite loops unless required
 - Use comments when logic is complex
 - Keep consistent indentation
 - Prefer `for` loops over `while` loops unless condition-based
-

★ 15. Summary of Module 3

- Control flow defines how program statements execute.
- Boolean expressions govern decision-making.
- Conditional statements: `if`, `elif`, `else`.
- Loops allow repetition: `for`, `while`.
- Loop control: `break`, `continue`, `pass`.
- `range()` generates numeric sequences for iteration.
- Nested loops allow complex logic.
- Python supports loop `else` – a unique feature.
- Avoid logical and indentation errors.
- Control flow is essential in every Python program.

This module gives learners the foundation needed to write dynamic, interactive, and logical Python programs.

MODULE 3 – Examples & Code Snippets (20 Examples)

★ 1. Simple if Statement

```
age = 18
if age >= 18:
    print("You are eligible to vote.")
```

Output:

```
You are eligible to vote.
```

★ 2. if-else Statement

```
num = 5
```

```
if num % 2 == 0:
    print("Even Number")
else:
    print("Odd Number")
```

Output:

Odd Number

★ 3. if-elif-else Ladder

```
marks = 85

if marks >= 90:
    print("Grade A")
elif marks >= 80:
    print("Grade B")
elif marks >= 70:
    print("Grade C")
else:
    print("Grade D")
```

Output:

Grade B

★ 4. Nested if Example

```
username = "admin"
password = "1234"

if username == "admin":
    if password == "1234":
        print("Login successful.")
    else:
        print("Wrong password")
else:
    print("User not found")
```

Output:

Login successful.

★ 5. Ternary Operator (Short if-else)

```
a = 10
b = 20

result = "a is greater" if a > b else "b is greater"
print(result)
```

Output:

```
b is greater
```

★ 6. Boolean Expression Demo

```
x = 10
y = 5

print(x > y and x != 0)
print(y < 3 or y == 5)
print(not (x < y))
```

Output:

```
True
True
True
```

★ 7. for Loop with List

```
fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)
```

Output:

```
apple
banana
mango
```

★ 8. Looping Through a String

```
for ch in "PYTHON":
    print(ch)
```

Output:

```
P
Y
T
H
O
N
```

★ 9. Using range() in a Loop

```
for i in range(1, 6):
```

```
print("Count:", i)
```

Output:

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

★ 10. Custom Step in range()

```
for i in range(0, 10, 2):
    print(i)
```

Output:

```
0
2
4
6
8
```

★ 11. Negative Step Range

```
for i in range(10, 0, -2):
    print(i)
```

Output:

```
10
8
6
4
2
```

★ 12. while Loop Basic Example

```
count = 1
while count <= 5:
    print("Hello")
    count += 1
```

Output:

```
Hello
Hello
Hello
Hello
Hello
```

★ 13. Infinite Loop (with break)

```
while True:
    cmd = input("Type 'exit' to stop: ")
    if cmd == "exit":
        break
```

Output:

```
Type 'exit' to stop: hello
Type 'exit' to stop: world
Type 'exit' to stop: exit
```

★ 14. break Statement

```
for i in range(1, 10):
    if i == 5:
        break
    print(i)
```

Output:

```
1
2
3
4
```

★ 15. continue Statement

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Output:

```
1
2
4
5
```

★ 16. pass Statement in Loops

```
for i in range(5):
    pass # placeholder for future logic

print("Loop executed successfully")
```

Output:

Loop executed successfully

★ 17. Loop-else Example

```
numbers = [2, 4, 6, 8]

for num in numbers:
    if num == 5:
        print("Found 5")
        break
else:
    print("5 not found in the list")
```

Output:

5 not found in the list

★ 18. Nested Loops (Multiplication Table)

```
for i in range(1, 6):
    for j in range(1, 6):
        print(i * j, end=" ")
    print()
```

Output:

```
1 2 3 4 5
2 4 6 8 10
3 6 9 12 15
4 8 12 16 20
5 10 15 20 25
```

★ 19. Pattern Printing: Right-Angled Triangle

```
for i in range(1, 6):
    print("*" * i)
```

Output:

```
*
**
***
****
*****
```

★ 20. Pattern Printing: Square Pattern

```
n = 4
for i in range(n):
    for j in range(n):
        print("*", end=" ")
    print()
```

Output:

```
* * * *
* * * *
* * * *
* * * *
```

MODULE 3 — LECTURES

Control Flow Statements in Python

This module includes 6 full lectures covering all types of decision-making and looping constructs.

★ Lecture 1 — Boolean Expressions & Introduction to Control Flow

Topic

"Boolean Logic, Truth Values, Conditions & Role of Control Flow"

Definition (Detailed)

Control flow refers to the order in which individual statements are executed in a program.

Unlike linear execution, real programs require decision-making, branching, and repetition.

Control flow in Python is built upon Boolean expressions, which evaluate to `True` or `False`.

Boolean expressions are constructed using comparison operators (`==`, `!=`, `>`, `<`, etc.) and logical operators (`and`, `or`, `not`).

Values like 0, "", [], {}, None, and False evaluate as False; everything else is True.

All conditional statements and loops in Python depend on Boolean expressions for deciding flow.

Syntax Overview

Boolean expression:

```
condition = (a > b and b != 0)
```

Using Boolean in a condition:

```
if a > b:  
    print("a is greater")
```

Examples

Example 1: Simple Boolean Check

```
x = 10  
print(x > 5)    # True
```

Example 2: Logical Evaluation

```
a = 5  
b = 0  
print(a > 3 and b == 0)    # True
```

Example 3: Truthy/Falsy Values

```
print(bool(""))    # False  
print(bool("Hi"))  # True
```

Key Takeaways

- Boolean expressions are the backbone of all control statements.
- True and False determine execution flow.
- Logical operators combine multiple conditions.
- Empty values are considered False.

Practice Section

ready_to_practice: You understand basics of Boolean logic.

description: Try building conditions using comparison and logical operators.

mcqs: Attempt MCQs 1–4.

coding_challenges: Write a condition to check if a string is empty or not.

★ Lecture 2 — if, if-else & if-elif-else Statements

Topic

"Decision Making with Conditional Statements"

Definition (Detailed)

Conditional statements allow selective execution of code based on conditions.

`if` executes its block only when the condition is `True`.

`if-else` provides an alternate block when the condition is `False`.

`if-elif-else` handles multiple conditions sequentially.

Only one block from the `if-elif-else` ladder executes.

Proper indentation is mandatory because Python uses it to define code blocks.

Syntax

if:

```
if condition:
    block
```

if-else:

```
if condition:
    block1
else:
    block2
```

if-elif-else:

```
if condition1:
    block1
elif condition2:
    block2
else:
    blockN
```

Examples

Example 1: Simple if

```
num = 10
if num > 0:
    print("Positive")
```

Example 2: if-else

```
x = 9
if x % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Example 3: if-elif-else

```
score = 75

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
else:
    print("D")
```

Key Takeaways

- Use `if` for simple checks.
- Use `if-else` when two alternatives exist.
- Use `if-elif-else` when handling multiple conditions.
- Indentation is crucial.

Practice Section

ready_to_practice: You can now write decision-making statements.

description: Try writing a grading program using if-elif ladder.

mcqs: Attempt MCQs 5–10.

coding_challenges: Create a program that checks if number is divisible by both 2 and 3.

★ Lecture 3 — Nested if Statements & Ternary Operator

Topic

"Multi-Level Decisions & Single-Line Conditional Expressions"

Definition (Detailed)

A nested `if` means placing an `if` inside another `if` block.

Nested conditions help solve multi-step validation or decision sequences.

Deep nesting can make code difficult to read, so it must be used carefully.

Python offers a compact form of `if-else` through the conditional expression, also called the ternary operator.

The ternary operator evaluates a condition and returns one of two values in a single line.

Syntax

Nested if:

```
if condition1:
    if condition2:
        block
```

Ternary Operator:

```
value = x if condition else y
```

Examples

Example 1: Nested if

```
user = "admin"
password = "1234"

if user == "admin":
    if password == "1234":
        print("Access granted")
```

Example 2: Ternary Expression

```
a = 10
b = 20
result = a if a > b else b
print(result)
```

Key Takeaways

- Nested `if` structures allow hierarchical decisions.
- Use ternary operator for short, single-line decisions.
- Avoid deep nesting for cleaner code.

Practice Section

ready_to_practice: You can implement multi-step logic.

description: Build a login validator using nested if.

mcqs: Attempt MCQs 11–14.

coding_challenges: Write a ternary expression to find the minimum of two numbers.

★ Lecture 4 — for Loop & range() Function

Topic

"Iterating Over Sequences, range() Variations & Loop Mechanics"

Definition (Detailed)

The `for` loop in Python iterates over sequences like lists, strings, tuples, sets, dictionaries, and generators.

Unlike C-style loops, Python's `for` loop does not use index counters by default.

The `range()` function generates a sequence of integers for controlled iteration.

`range()` supports start, stop, and step parameters.

The loop variable takes one value at a time from the iterable.

Syntax

Basic for loop:

```
for variable in iterable:
    block
```

range():

```
range(stop)
range(start, stop)
range(start, stop, step)
```

Examples

Example 1: Looping through a list

```
for fruit in ["apple", "banana", "mango"]:
    print(fruit)
```

Example 2: Using range()

```
for i in range(1, 6):  
    print("Count:", i)
```

Example 3: range with step

```
for i in range(0, 10, 2):  
    print(i)
```

Key Takeaways

- `for` loop is iterator-based and clean.
- `range()` gives flexibility in iteration.
- Start defaults to 0, step defaults to 1.
- `range` cannot generate float steps.

Practice Section

ready_to_practice: You can use `for` loop efficiently.

description: Try printing numbers from 1 to 100 using `range()`.

mcqs: Attempt MCQs 15–20.

coding_challenges: Write a program to print even numbers from 1 to 50.

★ Lecture 5 — while Loop & Loop Control Statements

Topic

"while Loop, break, continue, pass & Infinite Loops"

Definition (Detailed)

`while` loop continues as long as a Boolean condition remains `True`.

It is used for condition-driven repetition.

The loop body must eventually make the condition `False`, to prevent infinite looping.

`break` immediately exits the loop.

`continue` skips the current iteration and moves to next.

`pass` does nothing — used when a statement is required syntactically.

Infinite loops are intentionally created using `while True:` for interactive programs.

Syntax

while:

```
while condition:
    block
```

break:

```
if condition:
    break
```

continue:

```
if condition:
    continue
```

pass:

```
if condition:
    pass
```

Examples

Example 1: Simple while Loop

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

Example 2: break

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Example 3: continue

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Example 4: pass

```
for i in range(3):
    pass
```

Key Takeaways

- `while` is condition-based; `for` is sequence-based.
- `break` exits loop; `continue` skips iteration.
- `pass` is a no-op placeholder.
- Must avoid infinite loops unintentionally.

Practice Section

ready_to_practice: You understand loop control flow.

description: Try building loops with `break` and `continue`.

mcqs: Attempt MCQs 21–25.

coding_challenges: Create a loop that prints numbers until user enters 0.

★ Lecture 6 — Nested Loops & Loop-else

Topic

"Nested Loops, Pattern Logic & Python's Unique Loop-else Feature"

Definition (Detailed)

A nested loop is a loop placed inside another loop.

It is used for working with 2D structures, patterns, matrices, and tables.

Execution of inner loop happens completely for each iteration of outer loop.

Python supports `else` on loops — a special feature not found in many languages.

The `else` block executes when the loop completes normally without hitting a `break` statement.

Syntax

Nested loop:

```
for i in range():  
    for j in range():  
        block
```

Loop-else:

```
for element in sequence:
```



```
        if condition:
            break
    else:
        block_no_break
```

Examples

Example 1: Nested Loop for Multiplication Table

```
for i in range(1, 6):
    for j in range(1, 6):
        print(i * j, end=" ")
    print()
```

Example 2: Loop-else

```
nums = [1, 2, 3]
for n in nums:
    if n == 10:
        print("Found")
        break
else:
    print("10 not found")
```

Key Takeaways

- Nested loops handle multi-dimensional logic.
- `loop-else` executes only if loop completes without `break`.
- Good for search operations.
- Use nested loops carefully due to higher time complexity.

Practice Section

ready_to_practice: You now know advanced loop structures.

description: Try writing pattern programs using nested loops.

mcqs: Attempt MCQs 26–30.

coding_challenges: Write a program to search for a value in list using `loop-else`.

MODULE 3 - MCQs (15 Questions with Answers + Explanations)

★ MCQ 1. What does the following expression return?

`5 > 3 and 10 > 2`

- a) True
- b) False
- c) Error

Answer: a) True

Explanation: Both conditions are True $\rightarrow$ True and True = True.

★ MCQ 2. Which of the following values is considered False in Python?

- a) 1
- b) "Hello"
- c) []
- d) [0]

Answer: c) []

Explanation: Empty list evaluates to False.

★ MCQ 3. What will be the output?

```
if 0:
    print("True")
else:
    print("False")
```

- a) True
- b) False

Answer: b) False

Explanation: 0 is treated as False.

★ MCQ 4. How many times will the loop run?

```
for i in range(5):
    print(i)
```

- a) 4
- b) 5

- c) Infinite
- d) 1

Answer: b) 5

Explanation: `range(5)` → 0,1,2,3,4 (5 iterations)

★ MCQ 5. What is the output?

```
if "python":  
    print("Yes")  
else:  
    print("No")
```

- a) Yes
- b) No

Answer: a) Yes

Explanation: Non-empty string evaluates to True.

★ MCQ 6. What will break do?

- a) Skip iteration
- b) Stop entire loop
- c) Restart loop
- d) Execute else block

Answer: b) Stop entire loop

Explanation: `break` immediately exits the loop completely.

★ MCQ 7. What is the output?

```
for i in range(3):  
    if i == 1:  
        continue  
    print(i)
```

- a) 0 1 2
- b) 0 2
- c) 1 2
- d) 0

Answer: b) 0 2

Explanation: When `i=1`, `continue` skips the print statement. So only 0 and 2 are printed.

★ MCQ 8. Which loop is condition-based?

- a) for
- b) while
- c) both
- d) none

Answer: b) while

Explanation: `for` loop is sequence-based; `while` is condition-based.

★ MCQ 9. Output of the code:

```
count = 0
while count < 3:
    count += 1
print(count)
```

- a) 0
- b) 3
- c) 4
- d) Error

Answer: b) 3

Explanation: Loop runs 3 times (`count=1,2,3`), then stops when `count` becomes 3 and condition becomes False.

★ MCQ 10. What does this print?

```
for i in range(5):
    pass
print("Done")
```

- a) Done
- b) Error
- c) 0 1 2 3 4

Answer: a) Done

Explanation: `pass` does nothing, so loop completes without any output, then "Done" is printed.

★ MCQ 11. What is the output?

```
for x in [1, 2, 3]:
    if x == 2:
        break
else:
    print("Completed")
```

- a) Completed
- b) Nothing
- c) Error

Answer: b) Nothing

Explanation: `break` prevents else from executing.

★ MCQ 12. Output of nested loop:

```
for i in range(2):
    for j in range(2):
        print("A")
```

- a) A
- b) AA
- c) AAAA
- d) 4 A's in separate lines

Answer: d) 4 A's in separate lines

Explanation: Outer loop runs 2 times, inner loop runs 2 times for each outer iteration = $2 \times 2 = 4$ times total.

★ MCQ 13. Output?

```
i = 1
while i <= 3:
    print(i)
    i += 1
```

- a) 1 2
- b) 1 2 3
- c) 1 2 3 4
- d) infinite

Answer: b) 1 2 3

Explanation: Loop runs while $i \leq 3$, printing 1, 2, and 3.

★ MCQ 14. Which statement does nothing and is used as a placeholder?

- a) break
- b) return
- c) continue
- d) pass

Answer: d) pass

Explanation: `pass` is a null operation used as a syntactic placeholder.

★ MCQ 15. Output?

```
num = 10
if num > 5:
    print("A")
elif num > 8:
    print("B")
else:
    print("C")
```

- a) A
- b) B
- c) C

Answer: a) A

Explanation: First condition is True, so other conditions are ignored.

MCQs Completed (15)

MODULE 3 - Coding Tasks (15 Exercises)

(Beginner-friendly + Intermediate-level)

★ Task 1 — Check if Number is Positive, Negative or Zero

Problem: Take user input and print classification using if-elif-else.

Solution:

```
num = float(input("Enter a number: "))

if num > 0:
    print("Positive")
elif num < 0:
    print("Negative")
else:
    print("Zero")
```

★ Task 2 — Largest of Three Numbers

Problem: Input three integers and print the largest using nested if.

Solution:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

if a >= b:
    if a >= c:
        print(f"Largest: {a}")
    else:
        print(f"Largest: {c}")
else:
    if b >= c:
        print(f"Largest: {b}")
    else:
        print(f"Largest: {c}")
```

★ Task 3 — Login System

Problem:

- Ask for username and password

- If username = "admin" and password = "1234", print "Access Granted"
- Otherwise print "Access Denied"

Solution:

```
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "1234":
    print("Access Granted")
else:
    print("Access Denied")
```

★ Task 4 — Check Vowel or Consonant

Problem: Input a letter and determine if it is vowel using if-else.

Solution:

```
letter = input("Enter a letter: ").lower()

if letter in ['a', 'e', 'i', 'o', 'u']:
    print("Vowel")
else:
    print("Consonant")
```

★ Task 5 — Print Numbers from 1 to N using for Loop

Problem: User enters N.

Solution:

```
n = int(input("Enter N: "))

for i in range(1, n + 1):
    print(i)
```

★ Task 6 — Sum of First N Natural Numbers

Problem: Use a loop to calculate sum.

Solution:

```
n = int(input("Enter N: "))
total = 0

for i in range(1, n + 1):
    total += i
```



```
print(f"Sum: {total}")
```

★ Task 7 — Multiplication Table

Problem: Input a number and print its table (1 to 10).

Solution:

```
num = int(input("Enter a number: "))  
  
for i in range(1, 11):  
    print(f"{num} × {i} = {num * i}")
```

★ Task 8 — Factorial using while Loop

Problem: Compute factorial of a number.

Solution:

```
num = int(input("Enter a number: "))  
factorial = 1  
i = 1  
  
while i <= num:  
    factorial *= i  
    i += 1  
  
print(f"Factorial: {factorial}")
```

★ Task 9 — Count Digits in a Number

Problem: Use while loop to count digits.

Solution:

```
num = int(input("Enter a number: "))  
count = 0  
temp = abs(num) # Handle negative numbers  
  
while temp > 0:  
    temp //= 10  
    count += 1  
  
print(f"Number of digits: {count}")
```

★ Task 10 — Reverse a Number

Problem: Use while loop to reverse digits.

Solution:

```
num = int(input("Enter a number: "))
reversed_num = 0
temp = abs(num)

while temp > 0:
    digit = temp % 10
    reversed_num = reversed_num * 10 + digit
    temp //= 10

print(f"Reversed number: {reversed_num}")
```

★ Task 11 — Print Even Numbers Between 1 to 100

Problem: Use for loop with range.

Solution:

```
for i in range(2, 101, 2):
    print(i, end=" ")
```

★ Task 12 — Skip Multiples of 3 (continue)

Problem: Print numbers 1–20 except multiples of 3.

Solution:

```
for i in range(1, 21):
    if i % 3 == 0:
        continue
    print(i, end=" ")
```

★ Task 13 — Stop Loop at First Negative Number (break)

Problem: Given a list of numbers, stop when a negative is found.

Solution:

```
numbers = [5, 10, 15, -3, 20, 25]

for num in numbers:
    if num < 0:
        print(f"Found negative number: {num}")
        break
```

```
        break
    print(num)
```

★ Task 14 — Search an Element using loop-else

Problem: Input a value, check if it exists in a list.
If found → print "Found"
Else → "Not Found"

Solution:

```
numbers = [10, 20, 30, 40, 50]
search = int(input("Enter number to search: "))

for num in numbers:
    if num == search:
        print("Found")
        break
else:
    print("Not Found")
```

★ Task 15 — Pattern Printing (Right Triangle)

Problem: Print the following pattern:

```
*
**
***
****
*****
```

Solution:

```
n = 5

for i in range(1, n + 1):
    print("*" * i)
```

MODULE 4 THEORY — Functions, Modules & Packages in Python

(3500+ words, fully detailed, structured for LMS)

★ 1. Introduction to Functions in Python

Functions are the fundamental building blocks of any programming language. They transform raw code into well-structured, reusable, organized units. In real-world software systems, thousands of operations are executed repeatedly — without functions, code becomes extremely long, repetitive, and complex.

A **function** is a named block of code designed to perform a specific task. Instead of writing the same logic multiple times, programmers define a function once and use it whenever required. This improves modularity, maintainability, reusability, clarity, and performance.

Python treats functions as **first-class objects**, meaning:

- Functions can be stored in variables.
- Passed as arguments.
- Returned from other functions.
- Nested inside other functions.
- Created dynamically at runtime.

This makes Python exceptionally powerful for writing flexible and expressive programs.

★ 2. Why Functions Are Important

✓ Code Reusability

Once a function is defined, it can be reused multiple times.

✓ Modularity

Code is divided into smaller logical components.

✓ Improved Maintainability

Errors become easier to detect and fix.

✓ Avoid Repetition (DRY Principle)

"Don't Repeat Yourself" — functions prevent redundant code.

✓ Enhances Readability

Clear structure → easier understanding.

✓ Reduces Complexity

Breaks large problems into smaller tasks.

✓ Supports Abstraction

Implementation details are hidden inside the function.

★ 3. Anatomy of a Function in Python

Python function has the following components:

- **Function Name** → Identifier that refers to the function.
- **Parameters** → Data passed to function for processing.
- **Function Body** → Actual logic or computations.
- **Return Statement** → Specifies output of the function.
- **Function Call** → Where function gets executed.

General syntax:

```
def function_name(parameters):  
    statements  
    return expression
```

★ 4. Types of Functions in Python

Python supports multiple kinds of functions:

1 Built-in Functions

Provided by Python: `print()`, `len()`, `type()`, `max()`, `input()` etc.

2 User-Defined Functions

Created by programmer using `def`.

3 Lambda Functions

Anonymous functions created using `lambda`.

4 Recursive Functions

Functions calling themselves.

5 Higher-Order Functions

Functions that accept other functions as parameters (e.g., `map`, `filter`).

★ 5. Function Parameters in Python

Parameters control the inputs a function can receive.

Python supports several kinds of parameters:

★ 5.1 Positional Parameters

Values passed in order.

```
def add(a, b):  
    return a + b
```

Call must follow the order: `add(5, 10)`

★ 5.2 Default Parameters

Provide a default value.

```
def greet(name="User"):  
    return "Hello " + name
```

★ 5.3 Keyword Parameters

Arguments passed using names.

```
greet(name="Hemanth")
```

Order does not matter.

★ 5.4 Variable-Length Parameters

**a args (Non-Keyword Arguments)*

Collects multiple positional arguments as tuple.

****b) kwargs (Keyword Arguments)**

Collects multiple keyword arguments as dictionary.

★ 5.5 Required vs Optional Parameters

- **Required parameters** → must be supplied.
 - **Optional (default) parameters** → optional.
-

★ 6. Return Statement & Function Output

The `return` statement sends a value back to the caller.

Important rules:

- A function can have multiple return statements.
 - If no return exists → returns `None`.
 - Can return multiple values at once (packed as tuple).
-

★ 7. Variable Scope in Python (LEGB Rule)

Python resolves variable names using the **LEGB hierarchy**:

L – Local Scope

Variables inside function.

E – Enclosed Scope

Variables in nested functions.

G – Global Scope

Variables at the top-level file.

B – Built-in Scope

Predefined names like `len`, `sum`, etc.

★ 7.1 local Variables

Defined inside a function.

★ 7.2 global Variables

Defined outside functions. Accessible inside using `global` keyword.

★ 7.3 nonlocal Keyword

Used inside nested functions to modify outer function variables.

★ 8. Anonymous Functions (Lambda Expressions)

A **lambda function** is a small, unnamed function defined using a single-line expression:

```
lambda parameters: expression
```

Characteristics:

- One expression only
- Returns value implicitly
- Often used with `map()`, `filter()`, `sorted()`, `reduce()`

Lambda functions promote concise functional-style programming.

★ 9. Function as First-Class Citizen

Functions in Python behave like objects:

- stored in variables
- passed as arguments
- returned from functions
- used in data structures

Example scenarios:

- callback functions
 - event handlers
 - decorators
 - functional programming
-

★ 10. Higher Order Functions

Any function that accepts another function as argument or returns a function is called **higher-order**.

Examples:

- `map(function, iterable)`
- `filter(function, iterable)`
- `reduce(function, iterable)` (from `functools`)

Higher-order functions improve code conciseness and reduce loops.

★ 11. Recursion in Python

Recursion is a technique where a function calls itself.

Used in:

- factorial
- fibonacci
- tree traversal
- divide-and-conquer algorithms

Recursion must include a **base case** to avoid infinite calls.

Python has recursion depth limit (~1000).

★ 12. Modules in Python

A **module** is a file containing Python code.

It may include:

- functions
- classes
- variables
- executable statements

Modules promote:

- code organization
- reusability
- maintainability
- namespace separation

★ 12.1 Importing Modules

Python supports several ways to import:

```
import module_name
import module_name as alias
from module_name import function
from module_name import *
```

★ 12.2 Module Types

Built-in Modules

Examples: `math`, `os`, `sys`, `json`, `datetime`

User-Defined Modules

Python files created by user (e.g., `my_utils.py`)

Third-party Modules

Installed via `pip` (e.g., `numpy`, `pandas`)

★ 12.3 Creating a Module

Every `.py` file is a module automatically.

★ 13. The Python Package System

A **package** is a collection of related modules stored in a directory.

A package folder contains a special file:

```
__init__.py
```

Packages allow hierarchical organization.

Example package structure:

```
mypackage/  
    __init__.py  
    calculator.py  
    geometry.py
```

★ 14. Importing from Packages

You can import modules from packages using:

```
import package.module  
from package import module  
from package.module import function
```

Packages help build large-scale applications.

★ 15. Virtual Environments

Virtual environments isolate dependencies between different projects.

Key reasons:

- Avoid version conflicts
- Maintain clean environments
- Project-specific dependency management

Tools:

- `venv` (built-in)
 - `virtualenv`
 - `conda env`
-

★ 16. Installing External Packages (pip)

pip is Python's package manager.

Common commands:

```
pip install package_name
pip uninstall package_name
pip list
pip freeze > requirements.txt
```

Packages allow advanced functionality built by the community.

★ 17. Built-in Modules in Python

This module covers details of essential built-in modules:

★ 17.1 `math` Module

Provides advanced mathematical functions:

- `sqrt`
- `factorial`
- `gcd`
- `pow`
- `sin/cos`

★ 17.2 `random` Module

Used for random number generation.

Functions:

- `randint`

- random
- choice
- shuffle

Used in:

- games
- simulations
- data sampling
- testing

★ 17.3 datetime Module

Manages:

- dates
- times
- timestamps
- formatting

★ 17.4 os Module

Handles:

- files
- folders
- environment variables
- system commands

★ 17.5 sys Module

Handles:

- command-line arguments
- system functions
- interpreter settings

★ 18. Python Standard Libraries: A Foundation

Python comes with a huge standard library that supports:

- math operations
- data serialization
- file system handling
- network programming
- datetime manipulation
- regular expressions

Understanding modules is essential for building real-world applications.

★ 19. Packages vs Modules — Key Differences

Feature	Module	Package
Type	Single file	Directory of modules
Structure	module.py	folder with multiple .py files
Scale	Small	Large applications
<code>__init__.py</code>	Not required	Required
Use case	Small utilities	Organization at large scale

★ 20. Namespaces & Scope with Modules

Modules create their own namespaces.
This prevents naming conflicts.

```
module.variable  
module.function()
```

★ 21. The name and main Concept

Python sets `__name__` as:

- `"__main__"` → when script is executed directly
- module name → when imported

Used to write code that executes only when file is run, not imported.

```
if __name__ == "__main__":  
    # Code here runs only when script is executed directly
```

★ 22. Packages for Real-World Applications

Python packages drive modern development:

- **numpy** → scientific computing
- **pandas** → data analysis
- **matplotlib** → visualization
- **django** → web development
- **tensorflow** → deep learning
- **flask** → microservices
- **requests** → API calls

MODULE 4 — Python Functions & Modules

20 Essential Examples & Snippets

Basic Functions

★ 1. Basic Function Definition

```
def greet():  
    print("Hello, welcome to Python!")
```

★ 2. Function with Parameters

```
def add(a, b):  
    return a + b  
  
print(add(5, 10)) # Output: 15
```

★ 3. Function with Default Argument

```
def greet(name="User"):  
    print("Hello", name)  
  
greet() # Output: Hello User  
greet("Hemanth") # Output: Hello Hemanth
```

★ 4. Keyword Arguments

```
def person(name, age):  
    print(name, age)  
  
person(age=20, name="Hemanth") # Output: Hemanth 20
```

Advanced Arguments

★ 5. Variable-Length Arguments (*args)

```
def total(*numbers):  
    return sum(numbers)  
  
print(total(1, 2, 3, 4, 5)) # Output: 15
```

★ 6. Keyword Variable-Length Arguments (**kwargs)

```
def info(**details):
    for key, value in details.items():
        print(key, ":", value)

info(name="Hemanth", age=21, course="Python")
# Output:
# name : Hemanth
# age : 21
# course : Python
```

★ 7. Returning Multiple Values

```
def operations(a, b):
    return a + b, a - b, a * b

s, d, m = operations(10, 5)
print(s, d, m) # Output: 15 5 50
```

Scope & Nested Functions

★ 8. Function Inside Another Function

```
def outer():
    msg = "Hello"
    def inner():
        print(msg)
    inner()

outer() # Output: Hello
```

★ 9. Using global Keyword

```
x = 10

def change():
    global x
    x = 20

change()
print(x) # Output: 20
```

Lambda & Higher-Order Functions

★ 10. Lambda Function — Square of a Number

```
square = lambda x: x * x
print(square(5)) # Output: 25
```

★ 11. map() Function with Lambda

```
numbers = [1, 2, 3, 4]
```

```
squares = list(map(lambda x: x*x, numbers))
print(squares) # Output: [1, 4, 9, 16]
```

★ 12. filter() Function — Even Numbers

```
nums = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens) # Output: [2, 4, 6]
```

★ 13. reduce() Function — Sum of List

```
from functools import reduce

nums = [1, 2, 3, 4]
result = reduce(lambda a, b: a + b, nums)
print(result) # Output: 10
```

Recursion

★ 14. Recursive Function — Factorial

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5)) # Output: 120
```

Creating & Using Modules

★ 15. Creating a Module (my_module.py)

```
# my_module.py
def greet(name):
    return f"Hello {name}"

def add(a, b):
    return a + b
```

Using the module:

```
import my_module
print(my_module.greet("Hemanth")) # Output: Hello Hemanth
```

★ 16. Importing Everything from Module

```
from math import *

print(sqrt(25)) # Output: 5.0
print(factorial(6)) # Output: 720
```

Built-in Modules

★ 17. Using math Module

```
import math

print(math.sqrt(64))    # Output: 8.0
print(math.pow(2, 5))   # Output: 32.0
```

★ 18. Using random Module

```
import random

print(random.randint(1, 10))           # Random number 1-10
print(random.choice(["apple", "banana", "mango"])) # Random choice
```

★ 19. Using datetime Module

```
from datetime import datetime

now = datetime.now()
print("Current Time:", now)
```

★ 20. Using os Module to List Files

```
import os

files = os.listdir()
print(files) # Lists all files in current directory
```

Module 4 Summary

✓ Covered Topics:

- ✓ Basic function definitions
- ✓ Parameters & return values
- ✓ Default & keyword arguments
- ✓ *args and **kwargs
- ✓ Lambda functions
- ✓ map(), filter(), reduce()
- ✓ Recursion
- ✓ Creating custom modules
- ✓ Built-in modules (math, random, datetime, os)
- ✓ Global vs local scope

Key Takeaways:

1. Functions make code reusable and organized
2. Lambda functions are great for simple operations
3. Modules help structure larger programs

4. Python's built-in modules provide powerful functionality
5. Understanding scope is crucial for debugging

MODULE 4 — LECTURES

Functions, Modules & Packages

This module includes 6 full, detailed lectures.

★ Lecture 1 — Introduction to Functions

Topic

"What Are Functions? Need for Functions, Benefits, Structure, and Fundamentals."

Definition (Detailed)

A function is a reusable block of code designed to perform a specific operation.

Functions increase reusability and follow the DRY (Don't Repeat Yourself) principle.

They help break large programs into smaller, logical, manageable parts.

A function may accept parameters, process them, and optionally return a result.

Python functions are first-class objects — they can be passed to other functions, stored in variables, and returned as values.

Functions promote modular programming, making debugging and scaling easy.

Syntax

```
def function_name(parameters):  
    # function body  
    statements  
    return value
```

Examples

Example 1 — Basic Function

```
def greet():
```

```
print("Welcome to Python!")
```

Example 2 — Calling a Function

```
greet()
```

Example 3 — Function with Parameter

```
def square(n):  
    return n * n
```

```
print(square(5))
```

Key Takeaways

- Functions make code organized and reusable.
- Use `def` keyword to define.
- Can accept parameters and return values.
- Function names should be meaningful.
- Functions must be called to execute their logic.

Practice Section

ready_to_practice: You understand what functions are and why they are needed.

description: Create simple functions that print messages or perform basic calculations.

mcqs: Attempt MCQs 1–4.

coding_challenges: Write a function that prints your name 5 times.

★ Lecture 2 — Function Parameters & Return Types

Topic

"Different Types of Function Parameters, Return Values, and Argument Behavior."

Definition (Detailed)

Functions can receive inputs called parameters, and the actual values provided are arguments.

Python supports multiple types of parameters:

- **positional**
- **default**
- **keyword**
- **variable-length (args)*
- ****keyword variable-length (kwargs)**

The return statement sends result back to the caller.

A function can:

- return nothing
- return one value
- return multiple values

Python parameters are evaluated dynamically during the call.

Syntax

Positional:

```
def add(a, b):
    return a + b
```

Default:

```
def greet(name="User"):
    print("Hello", name)
```

Keyword:

```
greet(name="Hemanth")
```

*****args and kwargs:**

```
def test(*numbers, **details):
    pass
```

Examples

Example 1 — Positional Args

```
def subtract(a, b):
    return a - b
```

Example 2 — Default Parameter

```
def greet(name="Guest"):
    print("Hi", name)
```

**Example 3 — args*

```
def total(*nums):
```

```
print(sum(nums))
```

****Example 4 — kwargs**

```
def info(**data):  
    print(data)
```

Example 5 — Multiple Return Values

```
def calculate(a, b):  
    return a+b, a-b  
  
x, y = calculate(10, 5)
```

Key Takeaways

- Functions can accept multiple kinds of parameters.
- Use default args when you want optional parameters.
- Keyword args improve clarity.
- *args collects extra positional args → tuple.
- **kwargs collects extra keyword args → dict.
- Return statement ends function execution.

Practice Section

ready_to_practice: You can now use all parameter types.

description: Create functions using each parameter style.

mcqs: Attempt MCQs 5–10.

coding_challenges: Write a function that returns sum, average, and product of numbers using *args.

★ Lecture 3 — Variable Scope & LEGB Rule

Topic

"Understanding Local, Global, Enclosed & Built-in Scopes in Python."

Definition (Detailed)

Variable scope determines where a variable can be accessed.

Python follows the **LEGB Rule**:

- Local → Variables inside function
- Enclosed → Variables in nested functions

- **Global** → Variables declared at top level
- **Built-in** → Reserved Python keywords & functions

Local variables override global ones with same name.

The `global` keyword allows modifying global variables inside functions.

The `nonlocal` keyword allows nested functions to modify outer function variables.

Syntax

Global:

```
global variable_name
```

Nonlocal:

```
nonlocal variable_name
```

Examples

Example 1 — Global Variable

```
x = 10
def show():
    print(x)
```

Example 2 — Using global

```
x = 5
def change():
    global x
    x = 20
```

Example 3 — Enclosed Scope

```
def outer():
    msg = "Hello"
    def inner():
        print(msg)
    inner()
```

Example 4 — Using nonlocal

```
def outer():
    x = "Python"
    def inner():
        nonlocal x
        x = "Java"
    inner()
    print(x)
```

Key Takeaways

- LEGB defines variable lookup order.
- Use `global` to modify outer variable.
- Use `nonlocal` inside nested functions.
- Always avoid unnecessary global variables for best practice.

Practice Section

ready_to_practice: You understand variable scopes.

description: Experiment with local/global variables.

mcqs: Attempt MCQs 11–14.

coding_challenges: Write a nested function where inner modifies outer variable using `nonlocal`.

★ Lecture 4 — Lambda Functions & Higher Order Functions

Topic

"Anonymous Functions, map(), filter(), reduce(), Functional Programming Basics."

Definition (Detailed)

Lambda functions are anonymous, one-line functions created using `lambda` keyword.

Useful for short operations, callbacks, temporary functions.

Higher-order functions accept other functions as arguments.

`map()` applies a function to each item in iterable.

`filter()` selects items that satisfy a condition.

`reduce()` applies cumulative operation and reduces iterable to single value.

Syntax

`lambda arguments: expression`

Higher-order functions:

```
map(function, iterable)
filter(function, iterable)
```

```
reduce(function, iterable)
```

Examples

Example 1 — Basic Lambda

```
square = lambda x: x*x
```

Example 2 — map()

```
nums = [1, 2, 3]
squares = list(map(lambda x: x*x, nums))
```

Example 3 — filter()

```
even = list(filter(lambda x: x % 2 == 0, nums))
```

Example 4 — reduce()

```
from functools import reduce
total = reduce(lambda a, b: a + b, nums)
```

Key Takeaways

- Lambdas are single-expression anonymous functions.
- map() transforms iterable values.
- filter() extracts items meeting condition.
- reduce() combines all elements into one.
- Useful in data processing, AI/ML pipelines.

Practice Section

ready_to_practice: You can now use lambdas & HOFs.

description: Use map/filter on a list of numbers.

mcqs: Attempt MCQs 15–20.

coding_challenges: Write filter logic to extract odd numbers.

★ Lecture 5 — Modules in Python

Topic

"Understanding Modules, import Statements, Built-in Modules, and Creating Custom Modules."

Definition (Detailed)

A module is simply a `.py` file containing Python code (functions, classes, variables).

Modules allow code reuse and organization.

Python provides many built-in modules such as `math`, `random`, `json`.

You can create your own module by defining functions in a separate Python file.

Modules prevent naming conflicts using namespaces.

Several import styles exist depending on requirements.

Syntax

Basic import:

```
import module_name
```

Import specific function:

```
from module_name import function
```

Alias:

```
import math as m
```

Examples

Example 1 — Using math module

```
import math
print(math.sqrt(25))
```

Example 2 — Using alias

```
import math as m
print(m.pow(2, 3))
```

Example 3 — User-created module (`my_module.py`)

```
def greet(name):
    return "Hello " + name
```

Using it:

```
import my_module
print(my_module.greet("Hemanth"))
```

Key Takeaways

- Modules help organize and reuse code.

- Built-in modules save development time.
- Use alias for shorter names.
- Functions from modules accessed as: `module.function()`

Practice Section

ready_to_practice: You can import and use modules.

description: Try creating a custom module.

mcqs: Attempt MCQs 21–25.

coding_challenges: Create a module with two math functions & import them.

★ Lecture 6 — Python Packages & Project Structure

Topic

"Packages, Directory Structure, `init.py`, Importing from Packages."

Definition (Detailed)

A package is a directory containing multiple modules.

It must contain an `__init__.py` file to be recognized as a package.

Packages allow hierarchical grouping of related modules.

Used extensively in real-world large applications, frameworks, and libraries.

Importing from packages supports modular design and prevents code clutter.

Syntax

Package Structure:

```
mypackage/  
  __init__.py  
  maths.py  
  strings.py
```

Import:

```
import mypackage.maths  
from mypackage.strings import reverse
```

Examples

Example 1 — Package Import

```
import mypackage.maths
print(mypackage.maths.add(5, 10))
```

Example 2 — Selective Import

```
from mypackage.strings import reverse
```

Key Takeaways

- Packages organize larger codebases.
- `__init__.py` indicates a package directory.
- Nested packages allow scalable structures.
- Importing from packages improves clarity & modularity.

Practice Section

ready_to_practice: You understand package architecture.

description: Build a package with 2 modules.

mcqs: Attempt MCQs 26–30.

coding_challenges: Create a package named `tools` with string and math modules.

MODULE 4 — MCQs (20 Questions with Answers + Explanations)

★ MCQ 1. Which keyword is used to define a function in Python?

- a) func
- b) function
- c) define
- d) def

Answer: d

Explanation: In Python, the `def` keyword is used to define a function. The syntax is `def function_name():`. Options a, b, and c are not valid Python keywords for function definition.

★ MCQ 2. What is the output of the following?

```
def add(a, b):  
    return a + b  
  
print(add(2, 3))
```

- a) 23
- b) 5
- c) Error
- d) None

Answer: b

Explanation: The function `add(2, 3)` performs addition of 2 and 3, which returns 5. Option a would be the result if they were concatenated as strings. The function works correctly, so no error occurs.

★ MCQ 3. What is the default return value of a Python function without return?

- a) 0
- b) Null
- c) None
- d) False

Answer: c

Explanation: If a function doesn't explicitly return a value, Python automatically returns `None`. This is Python's special null value. Java uses `null`, but Python uses `None`.

★ MCQ 4. Which of the following is a correct lambda function?

- a) `lambda x: xx`
- b) `lambda(x) return xx`

- c) `func lambda x: xx`
- d) `lambda: xx`

Answer: a

Explanation: Lambda syntax is `lambda arguments: expression`. Option a correctly follows this syntax. Option b uses `return` which is not allowed in lambda. Option c uses `func` which is invalid. Option d references `x` without defining it as a parameter.

★ MCQ 5. Which function applies a function to all items in an iterable?

- a) `map()`
- b) `filter()`
- c) `reduce()`
- d) `apply()`

Answer: a

Explanation: `map()` applies a given function to each item in an iterable and returns a map object (iterator). `filter()` filters items based on condition. `reduce()` combines items into a single value. `apply()` is not a built-in Python function.

★ MCQ 6. What does `filter()` return?

- a) A list
- b) An iterator
- c) A tuple
- d) None

Answer: b

Explanation: `filter()` returns a filter object, which is an iterator. To get a list, you need to convert it using `list(filter(...))`. This is memory-efficient as it doesn't create the entire list immediately.

★ MCQ 7. To use `reduce()`, which module must be imported?

- a) math
- b) functools
- c) itertools
- d) operator

Answer: b

Explanation: `reduce()` is located in the `functools` module. You must import it using `from functools import reduce`. In Python 2, it was a built-in, but in Python 3, it was moved to `functools`.

★ MCQ 8. What will be the output?

```
def fun(a, b=2):  
    return a * b  
  
print(fun(5))
```

- a) 10
- b) 5
- c) Error
- d) None

Answer: a

Explanation: The function is called with only one argument (5). Since `b` has a default value of 2, the function calculates $5 * 2 = 10$. Default parameters are used when the argument is not provided.

★ MCQ 9. The `*args` parameter collects arguments into a:

- a) List
- b) Tuple
- c) Set
- d) Dictionary

Answer: b

Explanation: `*args` collects all extra positional arguments into a tuple. Tuples are immutable sequences. This allows functions to accept any number of positional arguments.

★ MCQ 10. The ****kwargs** parameter collects arguments into a:

- a) List
- b) Tuple
- c) Dictionary
- d) Set

Answer: c

Explanation: ****kwargs** collects all extra keyword arguments into a dictionary where keys are parameter names and values are the passed values. This allows functions to accept any number of keyword arguments.

★ MCQ 11. What is the output?

```
x = 10
def show():
    x = 5
    print(x)
```

```
show()
print(x)
```

- a) 5 5
- b) 10 10
- c) 5 10
- d) Error

Answer: c

Explanation: Inside the function, `x = 5` creates a local variable that shadows the global `x`. So `show()` prints 5. The second `print(x)` accesses the global `x`, which is still 10. Local scope doesn't affect global scope without the `global` keyword.

★ MCQ 12. Which keyword allows modifying global variables inside functions?

- a) global
- b) nonlocal
- c) shared
- d) scope

Answer: a

Explanation: The `global` keyword tells Python that you want to modify the global variable, not create a new local one. Without `global`, assignment creates a new local variable.

★ MCQ 13. Which keyword is used inside nested functions to modify outer variables?

- a) outer
- b) global
- c) nonlocal
- d) inherit

Answer: c

Explanation: `nonlocal` is used in nested functions to modify variables from the enclosing (outer) function scope. `global` is for module-level variables, while `nonlocal` is for enclosing function variables.

★ MCQ 14. What is a module in Python?

- a) A variable
- b) A function
- c) A `.py` file
- d) A program in C

Answer: c

Explanation: A module is simply a Python file (with `.py` extension) containing Python code such as functions, classes, and variables. Modules allow code organization and reusability.

★ MCQ 15. Which file marks a directory as a Python package?

- a) `start.py`
- b) `main.py`
- c) `package.py`
- d) `init.py`

Answer: d

Explanation: The `__init__.py` file (note the double underscores) tells Python that the directory should be treated as a package. It can be empty or contain initialization code for the package.

★ MCQ 16. Which import statement is correct?

- a) `import math()`
- b) `import(math)`
- c) `import math`
- d) `include math`

Answer: c

Explanation: The correct syntax is `import module_name` without parentheses or special syntax. Option a and b incorrectly use parentheses. Option d uses `include` which is not a Python keyword (that's C/C++).

★ MCQ 17. Which module is used for random numbers?

- a) `random`
- b) `chaos`
- c) `rand`
- d) `randint`

Answer: a

Explanation: The `random` module provides functions for generating random numbers. `randint` is a function within the `random` module, not a module itself. Options b and c are not standard Python modules.

★ MCQ 18. What is the output?

```
square = lambda x: x*x
print(square(4))
```

- a) 8
- b) 4
- c) 16
- d) Error

Answer: c

Explanation: The lambda function squares its input. `square(4)` returns $4*4 = 16$. Lambda functions are anonymous functions that can be assigned to variables and called like regular functions.

★ MCQ 19. What will happen if two modules contain same function name?

- a) Python crashes
- b) The latest imported module overrides
- c) Error occurs
- d) Both run automatically

Answer: b

Explanation: When you import functions with the same name from different modules, the last import overwrites the previous one. This is because Python's namespace simply reassigns the name to the new function. To avoid this, use `import module` syntax or import with aliases.

★ MCQ 20. What is printed?

```
from math import sqrt
print(sqrt(16))
```

- a) `sqrt(16)`
- b) 4
- c) 8
- d) Error

Answer: b

Explanation: `sqrt(16)` calculates the square root of 16, which is 4.0. The function is properly imported from the math module and executes correctly. Option a would be the result if it were a string, not a function call.

MCQs Completed (20)

MODULE 4 — Coding Tasks (20 Exercises)

(Beginner to Intermediate)

★ Task 1 — Create a Basic Function

Problem: Write a function `greet()` that prints "Hello Python".

Solution:

```
def greet():  
    print("Hello Python")  
  
# Test  
greet()
```

Output:

```
Hello Python
```

Explanation: This is the simplest function with no parameters and no return value. It just executes the print statement when called.

★ Task 2 — Add Two Numbers

Problem: Function `add(a, b)` returns sum.

Solution:

```
def add(a, b):  
    return a + b  
  
# Test  
result = add(10, 20)  
print(result)  
print(add(5, 15))
```

Output:

```
30  
20
```

Explanation: This function takes two parameters, adds them, and returns the result. The return value can be stored in a variable or printed directly.

★ Task 3 — Find Maximum Using Function

Problem: Write a function that returns the larger of two numbers.

Solution:

```
def find_max(a, b):
    if a > b:
        return a
    else:
        return b

# Alternative solution using max()
def find_max_v2(a, b):
    return max(a, b)

# Test
print(find_max(15, 20))
print(find_max(50, 30))
print(find_max_v2(25, 25))
```

Output:

```
20
50
25
```

Explanation: The function compares two numbers and returns the larger one. We can use if-else or Python's built-in `max()` function.

★ Task 4 — Factorial Using Function

Problem: Implement factorial using both loop and recursion.

Solution:

```
# Method 1: Using Loop
def factorial_loop(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

# Method 2: Using Recursion
def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
```

```
        return n * factorial_recursive(n - 1)

# Test
print("Using Loop:", factorial_loop(5))
print("Using Recursion:", factorial_recursive(5))
print("Factorial of 0:", factorial_recursive(0))
```

Output:

```
Using Loop: 120
Using Recursion: 120
Factorial of 0: 1
```

Explanation: Factorial of n ($n!$) = $n \times (n-1) \times (n-2) \times \dots \times 1$. The loop version iterates through numbers, while the recursive version calls itself with $n-1$ until reaching the base case.

★ Task 5 — Function with Default Parameter

Problem: Function `welcome(name="User")`.

Solution:

```
def welcome(name="User"):
    print(f"Welcome, {name}!")

# Test
welcome()
welcome("Hemanth")
welcome("Alice")
```

Output:

```
Welcome, User!
Welcome, Hemanth!
Welcome, Alice!
```

Explanation: When no argument is provided, the default value "User" is used. When an argument is passed, it overrides the default value.

★ Task 6 — Keyword Argument Practice

Problem: Create function `info(name, age)`; call with keyword args.

Solution:

```
def info(name, age):
    print(f"Name: {name}")
    print(f"Age: {age}")
```

```

    print("-" * 20)

# Test with keyword arguments
info(name="Hemanth", age=21)
info(age=25, name="Alice") # Order doesn't matter
info("Bob", 30) # Positional also works

```

Output:

```

Name: Hemanth
Age: 21
-----
Name: Alice
Age: 25
-----
Name: Bob
Age: 30
-----

```

Explanation: Keyword arguments allow you to pass arguments by parameter name, making the code more readable and allowing arguments in any order.

★ Task 7 — *args Usage

Problem: Write a function `total(*numbers)` that returns sum of all arguments.

Solution:

```

def total(*numbers):
    return sum(numbers)

# Alternative with manual calculation
def total_manual(*numbers):
    result = 0
    for num in numbers:
        result += num
    return result

# Test
print(total(1, 2, 3, 4, 5))
print(total(10, 20))
print(total(5))
print(total_manual(1, 2, 3, 4, 5, 6, 7, 8, 9, 10))

```

Output:

```

15
30
5
55

```

Explanation: `*args` collects all positional arguments into a tuple. The function can accept any number of arguments and process them.

★ Task 8 — **kwargs Usage

Problem: Write a function `profile(**details)` that prints all passed info.

Solution:

```
def profile(**details):
    print("Profile Information:")
    print("-" * 30)
    for key, value in details.items():
        print(f"{key.capitalize()}: {value}")
    print("-" * 30)

# Test
profile(name="Hemanth", age=21, course="Python", college="ABC University")
profile(name="Alice", city="New York", profession="Developer")
```

Output:

```
Profile Information:
-----
Name: Hemanth
Age: 21
Course: Python
College: ABC University
-----
Profile Information:
-----
Name: Alice
City: New York
Profession: Developer
-----
```

Explanation: `**kwargs` collects all keyword arguments into a dictionary. We iterate through the dictionary to print all key-value pairs.

★ Task 9 — Multiple Return Values

Problem: Function returns sum, diff, product of two numbers.

Solution:

```
def calculate(a, b):
    sum_result = a + b
    diff_result = a - b
    product_result = a * b
    return sum_result, diff_result, product_result

# Test
s, d, p = calculate(10, 5)
```

```
print(f"Sum: {s}")
print(f"Difference: {d}")
print(f"Product: {p}")

# Alternative: Return as tuple
result = calculate(20, 4)
print(f"\nAll results: {result}")
```

Output:

```
Sum: 15
Difference: 5
Product: 50
```

```
All results: (24, 16, 80)
```

Explanation: Python allows returning multiple values as a tuple. You can unpack them into separate variables or keep them as a tuple.

★ Task 10 — Lambda Function Practice

Problem: Create lambda for square, cube, even-check.

Solution:

```
# Lambda functions
square = lambda x: x ** 2
cube = lambda x: x ** 3
is_even = lambda x: x % 2 == 0

# Test
print("Square of 5:", square(5))
print("Cube of 3:", cube(3))
print("Is 10 even?", is_even(10))
print("Is 7 even?", is_even(7))

# Using lambdas in list
numbers = [1, 2, 3, 4, 5]
squares = list(map(square, numbers))
print("Squares:", squares)
```

Output:

```
Square of 5: 25
Cube of 3: 27
Is 10 even? True
Is 7 even? False
Squares: [1, 4, 9, 16, 25]
```

Explanation: Lambda functions are concise one-line functions. They're useful for simple operations and can be assigned to variables or used directly with map/filter.

★ Task 11 — Using map()

Problem: Use map() to get squares of a list of numbers.

Solution:

```
# Using map with lambda
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
squares = list(map(lambda x: x**2, numbers))
print("Original:", numbers)
print("Squares:", squares)

# Using map with named function
def square(x):
    return x * x

squares_v2 = list(map(square, numbers))
print("Squares v2:", squares_v2)
```

Output:

```
Original: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Squares v2: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Explanation: map() applies a function to every item in an iterable. It returns a map object, which we convert to a list for display.

★ Task 12 — Using filter()

Problem: Filter out even numbers from a given list.

Solution:

```
# Using filter with lambda
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
odd_numbers = list(filter(lambda x: x % 2 != 0, numbers))

print("Original:", numbers)
print("Even numbers:", even_numbers)
print("Odd numbers:", odd_numbers)

# Filter numbers greater than 5
greater_than_5 = list(filter(lambda x: x > 5, numbers))
print("Greater than 5:", greater_than_5)
```

Output:

```
Original: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
Even numbers: [2, 4, 6, 8, 10, 12]
Odd numbers: [1, 3, 5, 7, 9, 11]
```

Greater than 5: [6, 7, 8, 9, 10, 11, 12]

Explanation: `filter()` returns only items for which the function returns True. It's useful for extracting items that meet certain conditions.

★ Task 13 — Using `reduce()`

Problem: Find product of list elements using `reduce()`.

Solution:

```
from functools import reduce

# Product of all numbers
numbers = [1, 2, 3, 4, 5]
product = reduce(lambda a, b: a * b, numbers)
print("Numbers:", numbers)
print("Product:", product)

# Sum using reduce
sum_result = reduce(lambda a, b: a + b, numbers)
print("Sum:", sum_result)

# Find maximum using reduce
maximum = reduce(lambda a, b: a if a > b else b, numbers)
print("Maximum:", maximum)
```

Output:

```
Numbers: [1, 2, 3, 4, 5]
Product: 120
Sum: 15
Maximum: 5
```

Explanation: `reduce()` applies a function cumulatively to items, reducing the iterable to a single value. For product: $((((1*2)*3)*4)*5) = 120$.

★ Task 14 — Build Custom Module

Problem: Create `mycalc.py` with functions `add`, `sub`, `mul`, `div` and import it.

Solution:

File: mycalc.py

```
# mycalc.py

def add(a, b):
    """Returns sum of two numbers"""
```

```

    return a + b

def sub(a, b):
    """Returns difference of two numbers"""
    return a - b

def mul(a, b):
    """Returns product of two numbers"""
    return a * b

def div(a, b):
    """Returns division of two numbers"""
    if b == 0:
        return "Error: Division by zero"
    return a / b

# Optional: Test code
if __name__ == "__main__":
    print("Testing mycalc module")
    print("10 + 5 =", add(10, 5))
    print("10 - 5 =", sub(10, 5))
    print("10 * 5 =", mul(10, 5))
    print("10 / 5 =", div(10, 5))

```

File: main.py (using the module)

```

# main.py
import mycalc

print("Addition:", mycalc.add(15, 25))
print("Subtraction:", mycalc.sub(50, 20))
print("Multiplication:", mycalc.mul(7, 8))
print("Division:", mycalc.div(100, 4))
print("Division by zero:", mycalc.div(10, 0))

```

Output:

```

Addition: 40
Subtraction: 30
Multiplication: 56
Division: 25.0
Division by zero: Error: Division by zero

```

Explanation: A module is a .py file containing Python code. We import it using `import module_name` and access functions using `module_name.function_name()`.

★ Task 15 — Import from Module

Problem: From math import sqrt, factorial and use.

Solution:

```

from math import sqrt, factorial

```

```

# Using sqrt
print("Square root of 25:", sqrt(25))
print("Square root of 64:", sqrt(64))
print("Square root of 2:", sqrt(2))

# Using factorial
print("\nFactorial of 5:", factorial(5))
print("Factorial of 0:", factorial(0))
print("Factorial of 10:", factorial(10))

# Calculate hypotenuse
a, b = 3, 4
hypotenuse = sqrt(a**2 + b**2)
print(f"\nHypotenuse of triangle ({a}, {b}):", hypotenuse)

```

Output:

```

Square root of 25: 5.0
Square root of 64: 8.0
Square root of 2: 1.4142135623730951

Factorial of 5: 120
Factorial of 0: 1
Factorial of 10: 3628800

Hypotenuse of triangle (3, 4): 5.0

```

Explanation: `from module import function` allows us to import specific functions and use them directly without the module prefix.

★ Task 16 — Create a Package

Problem: Create package `tools` containing modules: `string_tools.py` and `math_tools.py`

Solution:

Directory Structure:

```

tools/
  __init__.py
  string_tools.py
  math_tools.py

```

File: `tools/init.py`

```

# __init__.py
# This file makes 'tools' a package
print("Tools package initialized")

```

File: `tools/string_tools.py`

```

# string_tools.py

```

```

def reverse_string(s):
    """Reverses a string"""
    return s[::-1]

def count_vowels(s):
    """Counts vowels in a string"""
    vowels = "aeiouAEIOU"
    return sum(1 for char in s if char in vowels)

def to_title_case(s):
    """Converts string to title case"""
    return s.title()

```

File: tools/math_tools.py

```

# math_tools.py

def is_prime(n):
    """Check if number is prime"""
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

def fibonacci(n):
    """Returns nth Fibonacci number"""
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b

def gcd(a, b):
    """Returns greatest common divisor"""
    while b:
        a, b = b, a % b
    return a

```

File: main.py (using the package)

```

# main.py
from tools import string_tools, math_tools

# Using string_tools
print("=== String Tools ===")
text = "Hello Python"
print("Original:", text)
print("Reversed:", string_tools.reverse_string(text))
print("Vowels:", string_tools.count_vowels(text))
print("Title Case:", string_tools.to_title_case("hello world"))

# Using math_tools
print("\n=== Math Tools ===")
print("Is 17 prime?", math_tools.is_prime(17))
print("Is 20 prime?", math_tools.is_prime(20))
print("10th Fibonacci:", math_tools.fibonacci(10))

```

```
print("GCD of 48 and 18:", math_tools.gcd(48, 18))
```

Output:

```
Tools package initialized
=== String Tools ===
Original: Hello Python
Reversed: nohtyP olleH
Vowels: 4
Title Case: Hello World

=== Math Tools ===
Is 17 prime? True
Is 20 prime? False
10th Fibonacci: 55
GCD of 48 and 18: 6
```

Explanation: A package is a directory containing `__init__.py` and multiple modules. It allows organizing related modules together.

★ Task 17 — Using random Module

Problem: Write a program that generates: random number (1–100), random choice from list, random float.

Solution:

```
import random

print("=== Random Number Generation ===\n")

# Random integer between 1 and 100
random_int = random.randint(1, 100)
print(f"Random integer (1-100): {random_int}")

# Random choice from list
fruits = ["apple", "banana", "mango", "orange", "grape"]
random_fruit = random.choice(fruits)
print(f"Random fruit: {random_fruit}")

# Random float between 0 and 1
random_float = random.random()
print(f"Random float (0-1): {random_float}")

# Random float in range
random_float_range = random.uniform(10.5, 50.5)
print(f"Random float (10.5-50.5): {random_float_range}")

# Shuffle a list
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
random.shuffle(numbers)
print(f"Shuffled list: {numbers}")

# Random sample (multiple random choices)
sample = random.sample(fruits, 3)
```

```
print(f"Random sample of 3 fruits: {sample}")
```

Output: (Output will vary each time)

```
=== Random Number Generation ===
```

```
Random integer (1-100): 42
Random fruit: mango
Random float (0-1): 0.7384920134
Random float (10.5-50.5): 28.3456789
Shuffled list: [7, 2, 9, 1, 5, 3, 10, 4, 8, 6]
Random sample of 3 fruits: ['orange', 'apple', 'mango']
```

Explanation: The `random` module provides various functions for generating random values, useful for games, simulations, and testing.

★ Task 18 — Using datetime Module

Problem: Print current date and time in formatted output.

Solution:

```
from datetime import datetime, date, timedelta

print("=== DateTime Examples ===\n")

# Current date and time
now = datetime.now()
print("Current date and time:", now)

# Formatted output
print("\nFormatted outputs:")
print("Date:", now.strftime("%Y-%m-%d"))
print("Time:", now.strftime("%H:%M:%S"))
print("Full format:", now.strftime("%A, %B %d, %Y %I:%M:%S %p"))
print("Short format:", now.strftime("%d/%m/%Y %H:%M"))

# Individual components
print("\nIndividual components:")
print("Year:", now.year)
print("Month:", now.month)
print("Day:", now.day)
print("Hour:", now.hour)
print("Minute:", now.minute)

# Today's date
today = date.today()
print("\nToday's date:", today)
print("Day of week:", today.strftime("%A"))

# Date arithmetic
tomorrow = today + timedelta(days=1)
week_ago = today - timedelta(weeks=1)
print("\nTomorrow:", tomorrow)
print("One week ago:", week_ago)
```

Output: (Example output)

```
=== DateTime Examples ===

Current date and time: 2024-12-07 14:30:45.123456

Formatted outputs:
Date: 2024-12-07
Time: 14:30:45
Full format: Saturday, December 07, 2024 02:30:45 PM
Short format: 07/12/2024 14:30

Individual components:
Year: 2024
Month: 12
Day: 7
Hour: 14
Minute: 30

Today's date: 2024-12-07
Day of week: Saturday

Tomorrow: 2024-12-08
One week ago: 2024-11-30
```

Explanation: The `datetime` module provides classes for working with dates and times. `strftime()` formats datetime objects into strings using format codes.

★ Task 19 — Simple Calculator Using Functions

Problem: Operations: +, -, *, / using separate functions.

Solution:

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Error: Cannot divide by zero"
    return a / b

def calculator():
    print("=== Simple Calculator ===")
    print("Operations:")
    print("1. Addition (+)")
    print("2. Subtraction (-)")
```



```

print("3. Multiplication (*)")
print("4. Division (/)")
print("5. Exit")

while True:
    choice = input("\nEnter choice (1-5): ")

    if choice == '5':
        print("Thank you for using calculator!")
        break

    if choice in ['1', '2', '3', '4']:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))

        if choice == '1':
            result = add(num1, num2)
            print(f"{num1} + {num2} = {result}")
        elif choice == '2':
            result = subtract(num1, num2)
            print(f"{num1} - {num2} = {result}")
        elif choice == '3':
            result = multiply(num1, num2)
            print(f"{num1} * {num2} = {result}")
        elif choice == '4':
            result = divide(num1, num2)
            print(f"{num1} / {num2} = {result}")
        else:
            print("Invalid choice! Please select 1-5.")

# Run calculator
calculator()

```

Output: (Interactive session example)

```

=== Simple Calculator ===
Operations:
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Exit

Enter choice (1-5): 1
Enter first number: 25
Enter second number: 15
25.0 + 15.0 = 40.0

Enter choice (1-5): 3
Enter first number: 7
Enter second number: 8
7.0 * 8.0 = 56.0

Enter choice (1-5): 5
Thank you for using calculator!

```

Explanation: This calculator uses separate functions for each operation and a main loop that takes user input. It demonstrates function organization and user interaction.

★ Task 20 — Scope Practice

Problem: Create nested functions and modify outer variable using nonlocal.

Solution:

```
# Example 1: Basic nested function with nonlocal
def outer_function():
    message = "Original message"

    def inner_function():
        nonlocal message
        message = "Modified message"
        print("Inside inner:", message)

    print("Before inner call:", message)
    inner_function()
    print("After inner call:", message)

print("=== Example 1 ===")
outer_function()

# Example 2: Counter using nonlocal
def create_counter():
    count = 0

    def increment():
        nonlocal count
        count += 1
        return count

    def decrement():
        nonlocal count
        count -= 1
        return count

    def get_count():
        return count

    return increment, decrement, get_count

print("\n=== Example 2: Counter ===")
inc, dec, get = create_counter()
print("Count:", get())
print("After increment:", inc())
print("After increment:", inc())
print("After increment:", inc())
print("After decrement:", dec())
print("Final count:", get())

# Example 3: Multiple levels of nesting
def level1():
    x = "Level 1"

    def level2():
        nonlocal x
        x = "Modified at Level 2"
```

```

        def level3():
            nonlocal x
            x = "Modified at Level 3"
            print("Level 3:", x)

        print("Level 2 before level3:", x)
        level3()
        print("Level 2 after level3:", x)

    print("Level 1 before level2:", x)
    level2()
    print("Level 1 after level2:", x)

print("\n=== Example 3: Multiple Levels ===")
level1()

# Example 4: Comparing local vs nonlocal
def comparison():
    value = 100

    def without_nonlocal():
        value = 200 # Creates new local variable
        print("Without nonlocal:", value)

    def with_nonlocal():
        nonlocal value
        value = 300 # Modifies outer variable
        print("With nonlocal:", value)

    print("Original:", value)
    without_nonlocal()
    print("After without_nonlocal:", value)
    with_nonlocal()
    print("After with_nonlocal:", value)

print("\n=== Example 4: Comparison ===")
comparison()

```

Output:

```

=== Example 1 ===
Before inner call: Original message
Inside inner: Modified message
After inner call: Modified message

=== Example 2: Counter ===
Count: 0
After increment: 1
After increment: 2
After increment: 3
After decrement: 2
Final count: 2

=== Example 3: Multiple Levels ===
Level 1 before level2: Level 1
Level 2 before level3: Modified at Level 2
Level 3: Modified at Level 3
Level 2 after level3: Modified at Level 3
Level 1 after level2: Modified at Level 3

```

```
=== Example 4: Comparison ===  
Original: 100  
Without nonlocal: 200  
After without_nonlocal: 100  
With nonlocal: 300  
After with_nonlocal: 300
```

Explanation: The `nonlocal` keyword allows inner functions to modify variables from the enclosing scope. Without it, assignment creates a new local variable instead of modifying the outer one. This is useful for closures and maintaining state.

MODULE 5 THEORY — Python Data Structures

(3500+ words, complete, structured, highly descriptive)

★ 1. Introduction to Python Data Structures

Data structures form the backbone of every programming language and are essential for storing, organizing, and efficiently manipulating data. In Python, data structures are extremely powerful because:

- They are built-in
- Dynamically sized
- Highly optimized
- Easy to use
- Provide rich inbuilt methods
- Are implemented using efficient memory models

Python's data structure ecosystem includes:

- **Strings (str)**
- **Lists (list)**
- **Tuples (tuple)**
- **Sets (set)**
- **Dictionaries (dict)**

These are known as core data structures because they are extensively used in:

- Data analysis
- Web development
- Machine learning
- Automation
- Competitive programming
- System development
- Game development

Understanding these structures is essential for writing efficient, clean, and optimized programs.

★ 2. Strings in Python

Strings are immutable sequences of Unicode characters. They are one of the most important data types for handling:

- Text data
- User input
- File content
- Messages
- API responses
- Logging information

★ 2.1 Characteristics of Strings

- **Immutable** – once created, cannot be modified.
- **Indexed** – characters accessed by index.
- **Ordered** – maintain sequence.
- **Iterable** – can be looped over.
- **Unicode** – supports global languages.
- **Dynamic Length** – grows as needed.

★ 2.2 String Memory Model

Each modification creates a new string object.

Example:

```
s = "Python"
s = s + "3"
```

Two memory objects are created because strings cannot be edited in-place.

This immutability:

- Makes strings thread-safe
- Improves performance in some operations
- Prevents unexpected modifications

★ 2.3 String Operations

1. Indexing

Access individual characters (0-based):

```
s[0]
s[-1]
```

2. Slicing

Extract substring:

```
s[1:5]
s[:3]
s[2:]
s[::-1] # reverse
```

3. Concatenation

`s1 + s2`

4. Repetition

`s * 3`

5. Membership

`"a" in "apple"`

★ 2.4 Common String Methods

- `upper()`, `lower()`
- `strip()`, `split()`, `join()`
- `startswith()`, `endswith()`
- `find()`, `count()`
- `replace()`
- `isalpha()`, `isdigit()`, etc.

★ 2.5 Mutable Alternatives to Strings

For heavy string modifications, use:

- `"".join(list)`
- `StringIO` from `io` module

★ 3. Lists in Python

Lists are the most versatile and frequently used data structure in Python.

★ 3.1 Characteristics of Lists

- **Mutable** – can change in place.
- **Dynamic arrays** – underlying implementation uses dynamic resizing.
- **Ordered** – maintain insertion order.
- **Indexed** – random access allowed.
- **Holds heterogeneous data** – strings, ints, objects, lists, etc.
- **Grow and shrink dynamically.**

★ 3.2 List Memory Model

Python lists are implemented as dynamic arrays:

- Store pointers to objects
- Resize by allocating larger blocks
- Amortized $O(1)$ append

★ 3.3 List Operations

1. Indexing

`L[0], L[-1]`

2. Slicing

`L[1:4]`

3. Add Elements

- `append()`
- `insert()`
- `extend()`

4. Remove Elements

- `remove()`
- `pop()`
- `del`

5. Searching

- `index()`
- `in` operator

6. Sorting

- `sort()` (in-place)
- `sorted()` (new list)

★ 3.4 List Methods

- `append(), extend(), insert()`
- `remove(), pop(), clear()`
- `sort(), reverse()`
- `count(), index()`

★ 3.5 List Comprehensions

A compact way to create lists:

```
[x*x for x in range(1, 6)]
```

Used extensively for:

- Data transformation
- Filtering
- Mapping

★ 3.6 Nested Lists (2D Lists)

Python allows lists within lists:

```
matrix = [[1,2], [3,4], [5,6]]
```

Used for:

- Tables
 - Matrices
 - Game boards
 - Multi-dimensional data
-

★ 4. Tuples in Python

Tuples are immutable sequences.

★ 4.1 Characteristics of Tuples

- **Immutable** → cannot modify after creation
- **Ordered**
- **Indexed**
- **Faster than lists**
- **Used as dictionary keys** (lists cannot)
- **Used for fixed collections**

★ 4.2 Use Cases of Tuples

- Returning multiple values
- Storing constant data
- Creating read-only collections
- Optimizing performance
- Safe data (prevents modification)

★ 4.3 Tuple Packing and Unpacking

```
a, b, c = (10, 20, 30)
```

Python supports flexible unpacking:

```
a, *b = (1, 2, 3, 4)
```

★ 5. Sets in Python

A set is an unordered collection of unique elements.

★ 5.1 Characteristics of Sets

- **Unique elements only**
- **Unordered** (no indexing)
- **Mutable**, but elements must be immutable
- **Fast membership checking** ($O(1)$)
- **Internally implemented using hash tables**

★ 5.2 Set Operations

- **Union** $\rightarrow A \mid B$
- **Intersection** $\rightarrow A \ \& \ B$
- **Difference** $\rightarrow A - B$
- **Symmetric difference** $\rightarrow A \wedge B$
- **Membership test** $\rightarrow in$

★ 5.3 Set Methods

- `add()`
- `update()`
- `remove()`
- `discard()`
- `pop()`
- `clear()`

★ 5.4 frozenset

An immutable version of set.

Used when:

- Set needs to be used as dictionary key
- Fixed, unmodifiable collections
- Hashable structures are needed

★ 6. Dictionaries in Python

A dictionary is a key-value pair data structure, implemented using hash tables.

★ 6.1 Characteristics of Dictionaries

- **Store key-value data**
- **Keys must be immutable**
- **Values can be any type**
- **Fast lookup** – average $O(1)$
- **Ordered in Python 3.7+**
- **Dynamic growth**

★ 6.2 Dictionary Operations

Access

```
d["name"]
```

Insert

```
d["age"] = 21
```

Update

```
d.update({"course": "Python"})
```

Delete

```
del d["name"]
```

★ 6.3 Dictionary Methods

- `get()`
- `items()`
- `keys()`
- `values()`
- `pop()`
- `popitem()`
- `clear()`

★ 6.4 Dictionary Comprehensions

```
{i: i*i for i in range(1, 6)}
```

Useful for:

- mapping
- transformations
- filtering

★ 7. Mutability in Data Structures

Data Type Mutable?

String ✗ No

List ✓ Yes

Tuple ✗ No

Set ✓ Yes

Dict ✓ Yes

Mutability affects:

- performance
- memory behavior
- ability to be used as keys
- modification safety

★ 8. Time Complexity of Data Structures

Understanding complexity is essential.

★ 8.1 List

Operation	Time
append	$O(1)$ amortized
pop end	$O(1)$
pop middle	$O(n)$
index search	$O(n)$

★ 8.2 Set

Operation	Time
membership	$O(1)$
insert	$O(1)$
delete	$O(1)$

★ 8.3 Dictionary

Operation	Time
lookup	$O(1)$
insert	$O(1)$
delete	$O(1)$

★ 9. Real-World Use Cases of Data Structures

Strings

- File names
- Logs
- User input

Lists

- Storing records
- Data processing
- Result collections

Tuples

- Coordinates
- DB records
- Immutable configs

Sets

- Removing duplicates
- Fast lookup
- Membership testing

Dictionaries

- JSON data
- Configurations
- Database-like structures
- Caching & indexing

★ 10. Best Practices for Data Structures

- Use **lists** when order & mutability are needed.
- Use **tuples** for fixed unchangeable data.
- Use **sets** to remove duplicates quickly.
- Use **dicts** to store key-value mappings.
- Use **comprehensions** for elegant data transformations.
- Choose mutable vs immutable based on use case.
- Prefer dictionary lookups over searching lists.
- Avoid deep nested lists unless required.

★ 11. Summary of Module 5

By now, the learner understands:

- Python's five core data structures
- Their characteristics
- Memory behavior
- Mutability
- Operations & methods
- Time complexities
- Real-world usage patterns
- Performance considerations

This module forms the foundation for future topics such as:

- **OOP (Module 6)**
- **File handling (Module 7)**
- **Data processing & algorithms (later modules)**

MODULE 5 — Examples & Snippets (25 Examples)

★ STRING EXAMPLES (1 – 6)

★ 1. Basic String Operations

```
s = "Python Programming"

print(s[0])      # P
print(s[-1])     # g
print(s[0:6])    # Python
print(s[::-1])   # Reverse string
```

Output:

```
P
g
Python
gnimmargorP nohtyP
```

★ 2. String Methods

```
text = "  hello python  "

print(text.strip())
print(text.upper())
print(text.capitalize())
print(text.replace("python", "world"))
```

Output:

```
hello python
HELLO PYTHON
hello python
hello world
```

★ 3. Splitting and Joining

```
sentence = "Python is awesome"
parts = sentence.split()

result = "-".join(parts)
print(result)
```

Output:

```
Python-is-awesome
```

★ 4. Checking Conditions in Strings

```
s = "Python123"

print(s.isalpha())    # False
print(s.isdigit())    # False
print(s.isalnum())    # True
```

Output:

```
False
False
True
```

★ 5. Counting Words in a Sentence

```
sentence = "python is easy to learn"
print(sentence.count("easy"))
```

Output:

```
1
```

★ 6. Mutable Alternative - List of Characters

```
s = "python"
char_list = list(s)
char_list[0] = "P"
s = "".join(char_list)
print(s)
```

Output:

```
Python
```

★ LIST EXAMPLES (7 - 14)

★ 7. Creating List and Accessing Elements

```
fruits = ["apple", "banana", "mango"]  
  
print(fruits[0])  
print(fruits[-1])
```

Output:

```
apple  
mango
```

★ 8. Adding and Removing Elements

```
nums = [1, 2, 3]  
  
nums.append(4)  
nums.insert(1, 10)  
nums.remove(2)  
print(nums)
```

Output:

```
[1, 10, 3, 4]
```

★ 9. Slicing Lists

```
L = [10, 20, 30, 40, 50]  
  
print(L[1:4])  
print(L[:3])  
print(L[::2])
```

Output:

```
[20, 30, 40]  
[10, 20, 30]  
[10, 30, 50]
```

★ 10. Sorting and Reversing

```
nums = [5, 2, 8, 1]  
  
nums.sort()  
print(nums)  
  
nums.reverse()  
print(nums)
```

Output:


```
[1, 2, 5, 8]
[8, 5, 2, 1]
```

★ 11. List Comprehension

```
squares = [x*x for x in range(1, 6)]
print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

★ 12. Filtering Even Numbers

```
nums = [1, 2, 3, 4, 5, 6]
evens = [x for x in nums if x % 2 == 0]
print(evens)
```

Output:

```
[2, 4, 6]
```

★ 13. Nested Lists (Matrix)

```
matrix = [
    [1, 2],
    [3, 4],
    [5, 6]
]

print(matrix[1][1]) # 4
```

Output:

```
4
```

★ 14. Cloning a List

```
original = [1, 2, 3]
copy_list = original[:] # or list(original)
print(copy_list)
```

Output:

```
[1, 2, 3]
```

★ TUPLE EXAMPLES (15 – 17)

★ 15. Tuple Creation and Indexing

```
t = (10, 20, 30)

print(t[0])
print(t[-1])
```

Output:

```
10
30
```

★ 16. Tuple Packing and Unpacking

```
a, b, c = (1, 2, 3)
print(a, b, c)
```

Output:

```
1 2 3
```

★ 17. Using Tuples as Dictionary Keys

```
locations = {
    (17.23, 78.12): "Hyderabad",
    (12.97, 77.59): "Bangalore"
}

print(locations[(17.23, 78.12)])
```

Output:

```
Hyderabad
```

★ SET EXAMPLES (18 – 20)

★ 18. Basic Set Operations

```
A = {1, 2, 3}
B = {3, 4, 5}

print(A | B)    # Union
print(A & B)    # Intersection
print(A - B)    # Difference
print(A ^ B)    # Symmetric difference
```

Output:

```
{1, 2, 3, 4, 5}
{3}
```

```
{1, 2}
{1, 2, 4, 5}
```

★ 19. Removing Duplicates from a List

```
nums = [1, 1, 2, 2, 3, 4, 4]

unique = list(set(nums))
print(unique)
```

Output:

```
[1, 2, 3, 4]
```

★ 20. Set Methods

```
items = {10, 20, 30}

items.add(40)
items.discard(20)
print(items)
```

Output:

```
{10, 30, 40}
```

★ DICTIONARY EXAMPLES (21 – 25)

★ 21. Creating Dictionary & Accessing Values

```
student = {
    "name": "Hemanth",
    "age": 21,
    "course": "Python"
}

print(student["name"])
print(student.get("age"))
```

Output:

```
Hemanth
21
```

★ 22. Adding & Updating Dictionary Values

```
student["email"] = "hemanth@mail.com"
student.update({"age": 22})
print(student)
```

Output:

```
{'name': 'Hemanth', 'age': 22, 'course': 'Python', 'email':  
'hemanth@mail.com'}
```

★ 23. Looping Through Dictionary

```
for key, value in student.items():  
    print(key, "->", value)
```

Output:

```
name -> Hemanth  
age -> 21  
course -> Python
```

★ 24. Dictionary Comprehension

```
squares = {x: x*x for x in range(5)}  
print(squares)
```

Output:

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

★ 25. Nested Dictionary (Real-World Example)

```
database = {  
    "user1": {"name": "Hemanth", "age": 21},  
    "user2": {"name": "Rahul", "age": 22}  
}  
  
print(database["user1"]["name"])
```

Output:

```
Hemanth
```

Module 5 Examples Completed (25 Snippets)

You now have:

- ✓ Strings examples
- ✓ Lists examples
- ✓ Tuples examples
- ✓ Sets examples
- ✓ Dictionaries examples

- ✓ Comprehensions
- ✓ Nested & advanced structures

MODULE 5 — LECTURES

Python Data Structures: Strings, Lists, Tuples, Sets, Dictionaries

★ Lecture 1 — Strings in Python

Topic

"Understanding Python Strings: Properties, Operations, Slicing, Methods, and Immutability."

Definition (Detailed & Multi-Line)

A string in Python is an immutable sequence of Unicode characters. It is represented using quotes (" or ''") and is used to store textual information.

Python strings support indexing, slicing, iteration, concatenation, membership tests, and a wide variety of built-in methods to manipulate text.

Strings are immutable, meaning once created, they cannot be changed. Each modification creates a new string object in memory.

Python supports Unicode, allowing storage and manipulation of multi-lingual content (e.g., English, Hindi, Telugu, emojis).

Strings are widely used in applications such as logs, user input, file processing, network communication, text transformation, and data parsing.

Syntax

```
s = "Hello Python"
```

Examples

Example 1 — Accessing Characters

```
s = "Python"
print(s[0])    # P
print(s[-1])   # n
```

Example 2 — Slicing

```
s = "Programming"
print(s[0:5])  # Progr
print(s[::-1]) # Reverse
```

Example 3 — String Methods

```
text = " hello python "
print(text.strip())
print(text.upper())
print(text.replace("python", "world"))
```

Key Takeaways

- Strings are immutable
- Indexed and ordered
- Support slicing and advanced operations
- Provide many built-in methods (split, join, strip, find, etc.)
- Use list() or StringIO for heavy modifications

Practice Section

ready_to_practice: You understand string properties and operations.

description: Try slicing, joining, formatting, and using built-in methods.

mcqs: Attempt Module 5 MCQs 1–5.

coding_challenges: Write a program to count vowels in a string.

★ Lecture 2 — Lists in Python

Topic

"List Characteristics, Indexing, Slicing, Mutability, Methods, and Comprehensions."

Definition (Detailed & Multi-Line)

A list is a mutable, ordered, dynamic data structure in Python capable of storing heterogeneous values.

Lists are implemented using dynamic arrays, meaning they resize automatically as elements are added or removed.

They support operations such as appending, inserting, updating, deleting, slicing, searching, and sorting.

Lists are used in almost every Python application — data processing pipelines, file I/O, machine learning datasets, database records, and more.

Because they are mutable, list operations modify the original list in memory.

Syntax

```
L = [10, 20, 30]
```

Examples

Example 1 — Adding Elements

```
nums = [1, 2, 3]
nums.append(4)
nums.insert(1, 100)
```

Example 2 — Removing Elements

```
nums.remove(1)
nums.pop()
```

Example 3 — Sorting and Reversing

```
values = [4, 1, 3, 2]
values.sort()
values.reverse()
```

Example 4 — List Comprehension

```
squares = [x*x for x in range(1, 6)]
```

Key Takeaways

- Lists are mutable and support in-place updates
- Provide rich built-in methods (append, pop, sort, reverse)
- Support slicing, iteration, nesting, and comprehension
- Used for collections, dynamic data, tables, and algorithmic operations

Practice Section

ready_to_practice: You can modify, sort, and filter lists.

description: Try adding/removing values and using comprehensions.

mcqs: Attempt Module 5 MCQs 6–10.

coding_challenges: Create a program to remove duplicates using lists.

★ Lecture 3 — Tuples in Python

Topic

"Tuple Properties, Immutability, Packing/Unpacking, Use Cases, and Efficiency."

Definition (Detailed & Multi-Line)

A tuple is an immutable, ordered collection of values.

Unlike lists, tuple elements cannot be modified after creation, making them safe and reliable for fixed datasets.

Tuples are memory-efficient and faster than lists for iteration and access.

They can store heterogeneous data types and are often used in database rows, coordinates, configuration values, and data that must remain constant.

Tuples support packing and unpacking, which is useful for returning multiple values from functions.

Syntax

```
t = (10, 20, 30)
```

Examples

Example 1 — Tuple Access

```
t = (10, 20, 30)
print(t[0])
print(t[-1])
```

Example 2 — Packing & Unpacking

```
a, b, c = (1, 2, 3)
```

Example 3 — Tuple as Dictionary Key

```
points = {(1,2): "A", (3,4): "B"}
```

Key Takeaways

- Tuples are immutable and safe
- Faster than lists
- Support unpacking and nested tuples
- Ideal for constant, read-only data

Practice Section

ready_to_practice: You understand tuple operations and immutability.

description: Experiment with tuple packing, slicing, and nesting.

mcqs: Attempt Module 5 MCQs 11–14.

coding_challenges: Convert a list into a tuple and access elements.

★ Lecture 4 — Sets in Python

Topic

"Set Characteristics, Hashing, Unique Elements, Operations, and Methods."

Definition (Detailed & Multi-Line)

A set is an unordered, mutable collection of unique elements.

Sets are implemented using hash tables, allowing extremely fast membership checks ($O(1)$).

Sets automatically remove duplicates and are ideal for mathematical operations like union, intersection, and difference.

Elements must be hashable (immutable) — meaning lists cannot be added, but tuples can.

Used extensively for filtering unique data, fast lookups, and mathematical set logic.

Syntax

```
s = {1, 2, 3}
```

Examples

Example 1 — Basic Operations

```
A = {1, 2, 3}
B = {3, 4, 5}
```

```
print(A | B) # Union
print(A & B) # Intersection
print(A - B) # Difference
```

Example 2 — Removing Duplicates

```
nums = [1, 1, 2, 2]
unique = list(set(nums))
```

Example 3 — Set Methods

```
items = {10, 20, 30}
items.add(40)
items.discard(20)
```

Key Takeaways

- Set values are unique
- No indexing or ordering
- Hashing makes lookups very fast
- Supports powerful mathematical operations

Practice Section

ready_to_practice: You can perform operations & filtering using sets.

description: Try converting lists to sets and performing unions/intersections.

mcqs: Attempt Module 5 MCQs 15–18.

coding_challenges: Given a list, extract only unique elements.

★ Lecture 5 — Dictionaries in Python

Topic

"Dictionary Structure, Key-Value Mappings, Hashing, Methods, and Nested Dictionaries."

Definition (Detailed & Multi-Line)

A dictionary is an unordered, mutable mapping of key-value pairs.

Keys must be immutable (string, number, tuple) and unique.

Dictionaries are implemented using hash tables, enabling $O(1)$ average lookup time.

Dictionaries form the backbone of many real-world applications including JSON data, database storage, API responses, configurations, and caching.

Dictionaries support nested structures, making them powerful for hierarchical data storage.

Syntax

```
d = {"name": "Hemanth", "age": 21}
```

Examples

Example 1 — Accessing & Updating

```
student = {"name": "Hemanth", "age": 21}
student["age"] = 22
student["email"] = "hemanth@mail.com"
```

Example 2 — Looping

```
for key, value in student.items():
    print(key, value)
```

Example 3 — Dictionary Comprehension

```
squares = {x: x*x for x in range(5)}
```

Example 4 — Nested Dictionary

```
db = {
    "user1": {"name": "Hemanth"},
    "user2": {"name": "Rahul"}
}
```

Key Takeaways

- Key-value structure
- Keys must be immutable
- Values can be anything
- Very fast lookups (hashing)
- Supports nesting and comprehension

Practice Section

ready_to_practice: You understand dictionary access, update, and methods.

description: Practice CRUD operations on dictionaries.

mcqs: Attempt Module 5 MCQs 19–22.

coding_challenges: Create a student database using nested dictionaries.

★ Lecture 6 — Data Structure Operations, Mutability & Comprehensions

Topic

"Mutability, Time Complexity, Comprehensions, Nested Data Structures, and DS Best Practices."

Definition (Detailed & Multi-Line)

Mutability refers to whether an object can be modified in place. Lists, sets, and dictionaries are mutable, while strings and tuples are immutable.

Comprehensions (list, set, dict comprehensions) provide a concise, readable way to construct and transform collections.

Understanding time complexity is essential for writing efficient code, especially in loops and large-scale data processing.

Nested data structures (e.g., lists inside dictionaries) allow storing complex hierarchical information.

Choosing the correct data structure ensures better performance and cleaner code.

Syntax (Comprehensions)

List:

```
[x*x for x in range(5)]
```

Set:

```
{x for x in [1,1,2,2,3]}
```

Dict:

```
{x: x*x for x in range(5)}
```

Examples

Example 1 — Mutability Demonstration

```
L = [1,2,3]
L.append(4)      # Mutates list

s = "python"
# s[0] = "P" ✗ Error (immutable)
```

Example 2 — Time Complexity

```
L = [10, 20, 30]
L.append(40)      # O(1)
L.pop(1)          # O(n)
```

Example 3 — Nested DS (Dictionary with List)

```
data = {
    "marks": [90, 80, 70],
    "name": "Hemanth"
}
```

Example 4 — Set Comprehension

```
unique_squares = {x*x for x in [1, 2, 2, 3, 3]}
```

Key Takeaways

- Strings & tuples → Immutable
- Lists, sets, dicts → Mutable
- Choose right DS based on operations needed
- Comprehensions simplify transformations
- Time complexity affects performance

Practice Section

ready_to_practice: You can now choose correct data structures for real tasks.

description: Build examples with nested structure & comprehensions.

mcqs: Attempt Module 5 MCQs 23–30.

coding_challenges: Write a dictionary comprehension to map words to lengths.

MODULE 5 LECTURES COMPLETE

You now have:

- ✓ Complete Theory
- ✓ 25 Examples
- ✓ 6 Full Lectures
- ✓ Ready for MCQs & Coding Tasks

PART 4: CODING TASKS (20 Exercises with Solutions)

Task 1 — Count Vowels in a String

Problem: Input a string → count and print the number of vowels.

Solution:

```
text = input("Enter a string: ")
vowels = "aeiouAEIOU"
count = 0

for char in text:
    if char in vowels:
        count += 1

print(f"Number of vowels: {count}")
```

Explanation: Loop through each character and check if it exists in the vowels string.

Task 2 — Check Palindrome

Problem: Check whether a string reads the same backward.

Solution:

```
text = input("Enter a string: ")

if text == text[::-1]:
    print("Palindrome")
else:
    print("Not a palindrome")
```

Explanation: Reverse the string using slicing `[::-1]` and compare with original.

Task 3 — Frequency of Each Character

Problem: Use dictionary to count character occurrences.

Solution:

```
text = input("Enter a string: ")
freq = {}

for char in text:
    freq[char] = freq.get(char, 0) + 1

for char, count in freq.items():
    print(f"{char}: {count}")
```

Explanation: Use dictionary's `get()` method to count each character's frequency.

Task 4 — Remove Duplicates From a List

Problem: Convert list to set and back to list.

Solution:

```
numbers = [1, 2, 2, 3, 4, 4, 5]
unique = list(set(numbers))
print("Unique elements:", unique)
```

Explanation: Sets automatically remove duplicates, then convert back to list.

Task 5 — Find Largest Element in List

Problem: Without using max().

Solution:

```
numbers = [12, 45, 23, 67, 34]
largest = numbers[0]

for num in numbers:
    if num > largest:
        largest = num

print("Largest:", largest)
```

Explanation: Initialize with first element, then compare with each element.

Task 6 — Reverse a List Without reverse()

Problem: Use slicing.

Solution:

```
numbers = [1, 2, 3, 4, 5]
reversed_list = numbers[::-1]
print("Reversed:", reversed_list)
```

Explanation: Use slice notation `[::-1]` to reverse the list.

Task 7 — Merge Two Lists

Problem: Keep duplicates.

Solution:

```
list1 = [1, 2, 3]
list2 = [3, 4, 5]
merged = list1 + list2
print("Merged list:", merged)
```

Explanation: Use + operator to concatenate lists, preserving all elements.

Task 8 — List Comprehension for Squares

Problem: Create list of squares from 1–20.

Solution:

```
squares = [x**2 for x in range(1, 21)]
print("Squares:", squares)
```

Explanation: List comprehension creates new list by applying operation to each element.

Task 9 — Tuple Packing & Unpacking

Problem: Pack values into tuple → unpack into variables.

Solution:

```
# Packing
data = (25, "Hemanth", "Python")
print("Packed tuple:", data)

# Unpacking
age, name, course = data
print(f"Age: {age}, Name: {name}, Course: {course}")
```

Explanation: Tuple packing combines values; unpacking assigns them to individual variables.

Task 10 — Convert List of Tuples to Dict

Problem: Example: [(1,"a"),(2,"b")] → {1:"a",2:"b"}

Solution:

```
tuple_list = [(1, "a"), (2, "b"), (3, "c")]
result_dict = dict(tuple_list)
```



```
print("Dictionary:", result_dict)
```

Explanation: `dict()` constructor converts list of key-value tuples to dictionary.

Task 11 — Unique Words in Sentence

Problem: Input a string → print unique words using set.

Solution:

```
sentence = input("Enter a sentence: ")
words = sentence.split()
unique_words = set(words)
print("Unique words:", unique_words)
```

Explanation: Split sentence into words and use set to get unique values.

Task 12 — Set Operations

Problem: Perform union, intersection, difference on user sets.

Solution:

```
set1 = {1, 2, 3, 4, 5}
set2 = {4, 5, 6, 7, 8}

print("Union:", set1 | set2)
print("Intersection:", set1 & set2)
print("Difference:", set1 - set2)
print("Symmetric Difference:", set1 ^ set2)
```

Explanation: Use operators `|` (union), `&` (intersection), `-` (difference), `^` (symmetric difference).

Task 13 — Dictionary CRUD Program

Problem: Add, update, delete and print dictionary entries.

Solution:

```
student = {}

# Create/Add
student['name'] = 'Hemanth'
student['roll'] = 101
print("After adding:", student)
```

```

# Update
student['roll'] = 102
print("After updating:", student)

# Delete
del student['roll']
print("After deleting:", student)

# Display
print("Final dictionary:", student)

```

Explanation: Use assignment to add/update, `del` to remove, and `print` to display.

Task 14 — Sort Dictionary by Values

Problem: Use `sorted()`.

Solution:

```

scores = {'Alice': 85, 'Bob': 92, 'Charlie': 78, 'David': 95}

sorted_dict = dict(sorted(scores.items(), key=lambda x: x[1]))
print("Sorted by values:", sorted_dict)

# For descending order
sorted_desc = dict(sorted(scores.items(), key=lambda x: x[1],
reverse=True))
print("Sorted descending:", sorted_desc)

```

Explanation: `sorted()` with `key=lambda x: x[1]` sorts by dictionary values.

Task 15 — Nested Dictionary Student Database

Problem: Store roll, name, marks for multiple students.

Solution:

```

students = {
    101: {'name': 'Hemanth', 'marks': 85},
    102: {'name': 'Rahul', 'marks': 90},
    103: {'name': 'Priya', 'marks': 88}
}

# Display all students
for roll, info in students.items():
    print(f"Roll: {roll}, Name: {info['name']}, Marks: {info['marks']}")

# Add new student
students[104] = {'name': 'Amit', 'marks': 92}

# Update marks

```

```

students[101]['marks'] = 87

print("\nUpdated database:")
for roll, info in students.items():
    print(f"Roll: {roll}, Name: {info['name']], Marks: {info['marks']}")

```

Explanation: Nested dictionary stores complex data with roll numbers as outer keys.

Task 16 — Word Length Dictionary

Problem: Input: list of words, Output: {word: len(word)} using comprehension.

Solution:

```

words = ['Python', 'Java', 'JavaScript', 'C++', 'Ruby']

word_lengths = {word: len(word) for word in words}
print("Word lengths:", word_lengths)

```

Explanation: Dictionary comprehension creates key-value pairs with word and its length.

Task 17 — Intersection of Two Lists

Problem: Return common elements.

Solution:

```

list1 = [1, 2, 3, 4, 5]
list2 = [4, 5, 6, 7, 8]

# Method 1: Using set intersection
common = list(set(list1) & set(list2))
print("Common elements:", common)

# Method 2: Using list comprehension
common2 = [x for x in list1 if x in list2]
print("Common elements (method 2):", common2)

```

Explanation: Convert to sets and use intersection, or use list comprehension to filter.

Task 18 — Convert List → Tuple and Tuple → List

Solution:

```

# List to Tuple

```

```
my_list = [1, 2, 3, 4, 5]
my_tuple = tuple(my_list)
print("List to Tuple:", my_tuple)

# Tuple to List
my_tuple2 = (10, 20, 30, 40)
my_list2 = list(my_tuple2)
print("Tuple to List:", my_list2)
```

Explanation: Use `tuple()` and `list()` constructors for conversion.

Task 19 — Find Minimum & Maximum Using Loop

Solution:

```
numbers = [45, 23, 67, 12, 89, 34]

minimum = numbers[0]
maximum = numbers[0]

for num in numbers:
    if num < minimum:
        minimum = num
    if num > maximum:
        maximum = num

print(f"Minimum: {minimum}")
print(f"Maximum: {maximum}")
```

Explanation: Initialize with first element, then compare each element to find min and max.

Task 20 — Menu-Driven Program Using All Data Structures

Solution:

```
def string_operations():
    text = input("Enter a string: ")
    print(f"Uppercase: {text.upper()}")
    print(f"Lowercase: {text.lower()}")
    print(f"Length: {len(text)}")
    print(f"Reversed: {text[::-1]}")

def list_operations():
    my_list = [1, 2, 3, 4, 5]
    print("Original list:", my_list)
    my_list.append(6)
    print("After append:", my_list)
    my_list.reverse()
    print("After reverse:", my_list)
    print(f"Sum: {sum(my_list)}")
```

```

def set_operations():
    set1 = {1, 2, 3, 4}
    set2 = {3, 4, 5, 6}
    print(f"Set 1: {set1}")
    print(f"Set 2: {set2}")
    print(f"Union: {set1 | set2}")
    print(f"Intersection: {set1 & set2}")
    print(f"Difference: {set1 - set2}")

def dict_operations():
    student = {'name': 'Hemanth', 'roll': 101, 'marks': 85}
    print("Original dict:", student)
    student['age'] = 20
    print("After adding age:", student)
    print(f"Keys: {list(student.keys())}")
    print(f"Values: {list(student.values())}")

while True:
    print("\n===== MENU =====")
    print("1. String Operations")
    print("2. List Operations")
    print("3. Set Operations")
    print("4. Dictionary Operations")
    print("5. Exit")

    choice = input("Enter your choice (1-5): ")

    if choice == '1':
        string_operations()
    elif choice == '2':
        list_operations()
    elif choice == '3':
        set_operations()
    elif choice == '4':
        dict_operations()
    elif choice == '5':
        print("Exiting program. Goodbye!")
        break
    else:
        print("Invalid choice! Please try again.")

```

Explanation: Menu-driven program demonstrates all major data structure operations with user interaction.

MODULE 6 — Object-Oriented Programming (OOP) in Python

PART 1: THEORY (4000+ Words)

1. Introduction to OOP

Object-Oriented Programming (OOP) is a programming paradigm that organizes programs around objects—entities that contain both data (attributes) and behavior (methods). Python fully supports OOP and encourages writing modular, reusable, scalable applications.

OOP helps in modeling real-world systems such as:

- Student management systems
- Banking systems
- E-commerce applications
- Games
- Chatbots
- APIs
- Large-scale enterprise software

OOP makes code cleaner, more organized, easier to maintain, and reduces redundancy.

2. Importance of OOP

OOP solves several problems found in procedural programming:

- Data & methods are grouped together → objects
 - Code duplication is reduced
 - Inheritance decreases repetition further
 - Encapsulation protects data
 - Polymorphism allows flexible function usage
 - Modularity & reusability increase
 - Large projects become manageable
-

3. Key OOP Concepts

Python supports all major OOP concepts:

1. **Class**
2. **Object**
3. **Constructor**
4. **Methods**
5. **Encapsulation**
6. **Abstraction**
7. **Inheritance**

- 8. Polymorphism
 - 9. Method Overriding
 - 10. Operator Overloading
-

4. Understanding Classes & Objects

4.1 What Is a Class?

A class is a blueprint or template for creating objects. It defines:

- Attributes (variables)
- Behaviors (methods)

A class does not occupy memory until an object (instance) is created.

4.2 What Is an Object?

An object is an instance of a class. It contains:

- Individual data
- Unique identity
- Methods bound to its class

Objects interact with each other to perform operations.

4.3 Example: Class & Object

```
class Student:
    pass

s1 = Student()
```

5. Constructors (init)

A constructor is a special method that initializes objects.

5.1 Constructor Syntax

```
class ClassName:
    def __init__(self, parameters):
        # initialization statements
```

5.2 Types of Constructors

1. **Default constructor** (no parameters)
2. **Parameterized constructor**
3. **Dynamic constructor** (logic inside)

5.3 Constructor Example

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

6. Types of Methods

Python has 3 types of methods:

1. **Instance Methods**
2. **Class Methods**
3. **Static Methods**

6.1 Instance Methods

- First parameter is always `self`
- Can access attributes of that object

```
def show(self):
    print(self.name)
```

6.2 Class Methods

- Defined using `@classmethod`
- First parameter is `cls`
- Access class-level data

```
@classmethod
def info(cls):
    print("This is Student class")
```

6.3 Static Methods

- Defined using `@staticmethod`
- No default parameters (`self / cls`)
- Used for utility operations

```
@staticmethod
def add(a, b):
    return a + b
```

7. Class Variables vs Instance Variables

7.1 Instance Variables

- Created inside constructor
- Unique for each object


```
self.name = name
```

7.2 Class Variables

- Common for all objects
- Defined inside class but outside methods

```
class Student:  
    school = "MLRIT"
```

8. Encapsulation

Encapsulation means wrapping data and methods into a single unit and restricting direct access to variables.

Python uses naming conventions:

Access Level	Syntax	Meaning
Public	variable	Accessible everywhere
Protected	<code>_variable</code>	Accessible within class + subclass
Private	<code>__variable</code>	Name mangled, not directly accessible

8.1 Example of Private Variables

```
class Bank:  
    def __init__(self):  
        self.__balance = 1000  
  
    def get_balance(self):  
        return self.__balance
```

9. Abstraction

Abstraction hides internal implementation and exposes only essential features.

Python provides abstraction using:

- Abstract Base Classes (ABC)
- `abstractmethod` decorator

```
from abc import ABC, abstractmethod  
  
class Shape(ABC):  
    @abstractmethod  
    def area(self):  
        pass
```

10. Inheritance

Inheritance allows a class to acquire properties & behaviors of another class.

10.1 Types of Inheritance in Python

1. **Single Inheritance**
2. **Multilevel Inheritance**
3. **Multiple Inheritance**
4. **Hierarchical Inheritance**
5. **Hybrid Inheritance**

Python supports all forms.

10.2 Single Inheritance Example

```
class Animal:
    def speak(self):
        return "Sound"

class Dog(Animal):
    pass
```

10.3 Multilevel Inheritance Example

```
class A:
    pass

class B(A):
    pass

class C(B):
    pass
```

10.4 Multiple Inheritance Example

```
class A: pass
class B: pass

class C(A, B): pass
```

10.5 super() Function

super() is used to call methods/constructors of parent class:

```
class Dog(Animal):
    def __init__(self):
        super().__init__()
```

11. Polymorphism

Polymorphism means many forms, allowing the same function name to behave differently based on object/class.

11.1 Method Overriding

Child class redefines parent method.

```
class A:
    def show(self):
        print("Parent")

class B(A):
    def show(self):
        print("Child")
```

11.2 Method Overloading

Python does not support true overloading. Instead, default arguments are used.

```
def add(a, b=0, c=0):
    return a + b + c
```

11.3 Operator Overloading

Python allows operators to be overloaded using magic methods.

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other.value
```

12. Magic Methods / Dunder Methods

Python provides special methods like:

- `__init__`
- `__str__`
- `__len__`
- `__add__`
- `__eq__`
- `__repr__`

Magic methods begin and end with double underscores.

Example:

```
def __str__(self):
    return "Object info"
```

13. Composition & Aggregation

These are alternatives to inheritance.

13.1 Composition ("has-a" strong relation)

```
class Engine: pass

class Car:
    def __init__(self):
        self.engine = Engine()
```

Car owns Engine completely.

13.2 Aggregation ("has-a" weak relation)

```
class Team:
    def __init__(self, players):
        self.players = players
```

Team uses players but does not own them.

14. MRO (Method Resolution Order)

Python uses C3 Linearization to solve method resolution order in multiple inheritance.

Use:

```
ClassName.mro()
```

15. OOP Real-World Uses

OOP is used in:

- ✓ Banking System (Account, Customer, Loan classes)
 - ✓ University System (Student, Course, Faculty)
 - ✓ Hospital System (Patient, Doctor, Appointment)
 - ✓ Games (Player, Enemy, Weapons)
 - ✓ E-commerce (Product, Order, Cart)
 - ✓ GUI Applications (Button, Window, Frame)
-

16. OOP Best Practices

- Use PascalCase for class names
- Keep methods short and meaningful
- Only expose necessary attributes
- Use inheritance appropriately
- Prefer composition over inheritance
- Use encapsulation to protect data
- Implement `__str__` for readable object output
- Use static and class methods wisely

17. Summary of Module 6

This module taught:

- OOP basics and principles
- Classes, objects, constructors
- Instance, class, static methods
- Variable types
- Inheritance types
- Polymorphism, overriding, operator overloading
- Encapsulation & abstraction
- Composition & aggregation
- MRO & magic methods
- Real-world applications

This builds a strong foundation for writing scalable Python applications.

PART 2: EXAMPLES & SNIPPETS (25 Examples)

Example 1: Basic Class and Object Creation

```
class Student:
    pass

s1 = Student()
print(type(s1))
```

Output: <class '__main__.Student'>

Explanation: Creates an empty class and an instance. `type()` shows the object's class.

Example 2: Class with Constructor

```
class Student:
    def __init__(self, name, roll):
        self.name = name
        self.roll = roll

s = Student("Hemanth", 101)
print(s.name, s.roll)
```

Output: Hemanth 101

Explanation: Constructor `__init__` initializes object attributes when creating an instance.

Example 3: Instance Methods

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print("Hello", self.name)

p = Person("Rahul")
p.greet()
```

Output: Hello Rahul

Explanation: Instance methods take `self` as first parameter and can access object attributes.

Example 4: Class Variables vs Instance Variables

```
class Employee:
    company = "TCS"    # class variable

    def __init__(self, name):
        self.name = name # instance variable

e1 = Employee("Hemanth")
print(e1.company, e1.name)
```

Output: TCS Hemanth

Explanation: Class variables are shared by all instances; instance variables are unique to each object.

Example 5: Class Method

```
class Student:
    school = "MLRIT"

    @classmethod
    def info(cls):
        print("School:", cls.school)

Student.info()
```

Output: School: MLRIT

Explanation: Class methods use `@classmethod` decorator and access class-level data through `cls`.

Example 6: Static Method

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b

print(Math.add(5, 6))
```

Output: 11

Explanation: Static methods don't need `self` or `cls`. Used for utility functions.

Example 7: Encapsulation (Private Variables)

```
class Bank:
    def __init__(self):
        self.__balance = 5000

    def get_balance(self):
        return self.__balance

b = Bank()
print(b.get_balance())
```

Output: 5000

Explanation: Private variables (double underscore) can't be accessed directly from outside the class.

Example 8: Encapsulation with Protected Variable

```
class Student:
    def __init__(self):
        self._marks = 90 # protected

class Test(Student):
    def show(self):
        print(self._marks)

t = Test()
t.show()
```

Output: 90

Explanation: Protected variables (single underscore) are accessible in subclasses.

Example 9: Single Inheritance

```
class Animal:
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    pass

d = Dog()
d.sound()
```

Output: Animal sound

Explanation: Dog inherits from Animal and can use its methods without redefining them.

Example 10: Multilevel Inheritance

```
class A:
    pass

class B(A):
    pass

class C(B):
    pass

c = C()
print(C.mro())
```

Output: [<class '__main__.C'>, <class '__main__.B'>, <class '__main__.A'>, <class 'object'>]

Explanation: C inherits from B, which inherits from A. MRO shows the inheritance chain.

Example 11: Multiple Inheritance

```
class A:
    def show(self):
        print("A")

class B:
    def show(self):
        print("B")

class C(A, B):
    pass

c = C()
```



```
c.show()
```

Output: A

Explanation: C inherits from both A and B. MRO determines which `show()` method is called (A comes first).

Example 12: Method Overriding

```
class Parent:
    def display(self):
        print("Parent method")

class Child(Parent):
    def display(self):
        print("Child method")

c = Child()
c.display()
```

Output: Child method

Explanation: Child class overrides the parent's method with its own implementation.

Example 13: Using `super()`

```
class A:
    def show(self):
        print("A method")

class B(A):
    def show(self):
        super().show()
        print("B method")

b = B()
b.show()
```

Output:

```
A method
B method
```

Explanation: `super()` calls the parent class method before executing child's own code.

Example 14: Polymorphism Using Common Method Names

```
class Dog:
    def sound(self):
        print("Bark")

class Cat:
    def sound(self):
        print("Meow")

for animal in (Dog(), Cat()):
    animal.sound()
```

Output:

```
Bark
Meow
```

Explanation: Same method name `sound()` behaves differently based on the object type.

Example 15: Operator Overloading (add)

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other.value

n1 = Number(10)
n2 = Number(20)
print(n1 + n2)
```

Output: 30

Explanation: `__add__` magic method allows custom behavior for the `+` operator.

Example 16: `str()` Magic Method

```
class Student:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"Student: {self.name}"

s = Student("Hemanth")
print(s)
```

Output: Student: Hemanth

Explanation: `__str__` defines how the object is represented as a string when printed.

Example 17: Abstract Class & Abstract Method

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def area(self):
        return 3.14 * 5 * 5

c = Circle()
print(c.area())
```

Output: 78.5

Explanation: Abstract classes force subclasses to implement specific methods.

Example 18: Composition (HAS-A Relationship)

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine() # composition

c = Car()
c.engine.start()
```

Output: Engine started

Explanation: Car contains an Engine object. If Car is deleted, Engine is also deleted.

Example 19: Aggregation (Weak HAS-A)

```
class Student:
    def __init__(self, name):
        self.name = name

class Classroom:
    def __init__(self, students):
        self.students = students # aggregation

s1 = Student("A")
s2 = Student("B")
c = Classroom([s1, s2])
```

Explanation: Classroom uses students but doesn't own them. Students can exist independently.

Example 20: Method Resolution Order (MRO)

```
class A: pass
class B(A): pass
class C(B): pass
```

```
print(C.mro())
```

Output: [<class '__main__.C'>, <class '__main__.B'>, <class '__main__.A'>, <class 'object'>]

Explanation: Shows the order in which Python searches for methods in inheritance hierarchy.

Example 21: Property Decorator (Getter & Setter)

```
class Student:
    def __init__(self, marks):
        self._marks = marks

    @property
    def marks(self):
        return self._marks

    @marks.setter
    def marks(self, value):
        self._marks = value

s = Student(90)
s.marks = 95
print(s.marks)
```

Output: 95

Explanation: @property allows controlled access to private attributes like public variables.

Example 22: Delete Attribute Using delattr()

```
class Person:
    def __init__(self):
        self.age = 21

p = Person()
delattr(p, 'age')
# print(p.age) # This would raise AttributeError
```

Explanation: `delattr()` removes an attribute from an object at runtime.

Example 23: Inheritance with Constructor Overriding

```
class A:
    def __init__(self):
        print("A constructor")

class B(A):
    def __init__(self):
        super().__init__()
        print("B constructor")

b = B()
```

Output:

```
A constructor
B constructor
```

Explanation: Child constructor calls parent constructor using `super().__init__()`.

Example 24: len Magic Method

```
class Message:
    def __init__(self, text):
        self.text = text

    def __len__(self):
        return len(self.text)

m = Message("Hello")
print(len(m))
```

Output: 5

Explanation: `__len__` allows using built-in `len()` function on custom objects.

Example 25: Storing Objects in a List

```
class Student:
    def __init__(self, name):
        self.name = name

students = [Student("Hemanth"), Student("Rahul")]

for s in students:
    print(s.name)
```

Output:

Hemanth
Rahul

Explanation: Objects can be stored in lists and accessed like any other data type.

MODULE 6 — LECTURES

Object-Oriented Programming in Python

★ Lecture 1 — Classes & Objects

Topic

"Understanding Python Classes, Objects, Instance Variables, Class Variables, and Object Lifecycle."

Definition (Detailed & Multi-Line)

A class in Python is a blueprint or template that defines the structure and behavior (data + functions) of an object.

An object is an instance of a class created in memory, containing its own copy of instance variables and sharing class-level variables.

Classes help group related data and functions, enabling modular and reusable code.

A class defines attributes (variables) and methods (functions) that describe an object's state and behavior.

Python supports dynamic attribute creation, meaning attributes can be added after object creation.

Objects interact with one another to perform tasks, forming the foundation of object-oriented programming.

Syntax

```
class ClassName:  
    # class body  
    pass
```

```
obj = ClassName()
```

Examples

Example 1 — Simple Class

```
class Student:
    pass

s = Student()
print(type(s))
```

Example 2 — Adding Attributes

```
class Person:
    def __init__(self, name):
        self.name = name

p = Person("Hemanth")
print(p.name)
```

Key Takeaways

- A class defines structure; an object is the instance.
- Instance variables store object-specific data.
- Class variables store common values.
- Objects can be created dynamically.

Practice Section

ready_to_practice: You understand classes & object creation.

description: Create a Student class with name & roll attributes.

mcqs: Attempt MCQs 1–5.

coding_challenges: Write a class with three attributes & print object data.

★ Lecture 2 — Constructors & Types of Methods

Topic

"Constructors, Instance Methods, Class Methods, Static Methods, and Method Bindings."

Definition (Detailed & Multi-Line)

A constructor (`__init__`) is a special method that is called automatically when an object is created.

It initializes instance variables and prepares the object for use.

Python supports instance methods, class methods, and static methods.

Instance methods access object-level data via `self`.

Class methods operate on class-level data using `cls`.

Static methods perform general tasks without requiring class or object context.

Syntax

```
# Constructor:
def __init__(self, parameters):
    ...

# Class Method:
@classmethod
def method(cls):
    ...

# Static Method:
@staticmethod
def method():
    ...
```

Examples

Example 1 — Constructor

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Example 2 — Class Method

```
class School:
    school_name = "MLRIT"

    @classmethod
    def get_name(cls):
        return cls.school_name
```

Example 3 — Static Method

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b
```

Key Takeaways

- Constructors initialize objects.
- Instance methods operate on `self`.
- Class methods use `cls` and access class variables.

- Static methods do not depend on the class/object.

Practice Section

ready_to_practice: You understand constructors & method types.

description: Implement class, instance, and static methods.

mcqs: Attempt MCQs 6–10.

coding_challenges: Write a class Calculator with class & static methods.

★ Lecture 3 — Encapsulation & Data Hiding

Topic

"Public, Private, Protected Variables, Getters, Setters, and Controlled Access in Python."

Definition (Detailed & Multi-Line)

Encapsulation refers to the bundling of attributes and methods inside a class and restricting direct access to them.

Python uses naming conventions for access control:

- **Public** → accessible everywhere
- **Protected** (`_variable`) → accessible in class & subclass
- **Private** (`__variable`) → name-mangled, accessible only through methods

To safely access or modify private data, Python uses getter and setter methods or the `@property` decorator.

Encapsulation protects sensitive data from accidental modification.

Syntax

```
class Demo:
    def __init__(self):
        self.__private = 100    # private
```

Examples

Example 1 — Private Variable

```
class Bank:
    def __init__(self):
```

```
self.__balance = 5000

def get_balance(self):
    return self.__balance
```

Example 2 — Property Decorator

```
class Student:
    def __init__(self, marks):
        self._marks = marks

    @property
    def marks(self):
        return self._marks

    @marks.setter
    def marks(self, m):
        self._marks = m
```

Key Takeaways

- Encapsulation hides sensitive data.
- Use `_` for protected & `__` for private.
- Getters and setters provide controlled access.
- `@property` simplifies getter/setter logic.

Practice Section

ready_to_practice: You can implement encapsulation.

description: Create a class with private attributes & provide access using getters/setters.

mcqs: Attempt MCQs 11–15.

coding_challenges: Create a Bank class with deposit & withdraw methods.

★ Lecture 4 — Inheritance & super()

Topic

"Types of Inheritance, Parent–Child Relation, Constructor Inheritance, Method Overriding, super() Function."

Definition (Detailed & Multi-Line)

Inheritance enables a class to acquire attributes and methods of another class, increasing code reusability and reducing duplication.

Types of inheritance in Python include:

- Single
- Multilevel
- Multiple
- Hierarchical
- Hybrid

Child classes override parent methods to customize behavior — known as method overriding.

The `super()` function allows calling methods/constructors of the parent class.

Inheritance creates strong code hierarchies and helps build scalable applications.

Syntax

```
class Parent:
    ...

class Child(Parent):
    ...
```

Examples

Example 1 — Single Inheritance

```
class Animal:
    def speak(self):
        print("Sound")

class Dog(Animal):
    pass
```

Example 2 — Method Overriding

```
class A:
    def show(self):
        print("Parent")

class B(A):
    def show(self):
        print("Child")
```

Example 3 — Using `super()`

```
class Parent:
    def __init__(self):
        print("Parent constructor")

class Child(Parent):
    def __init__(self):
        super().__init__()
        print("Child constructor")
```

Key Takeaways

- Inheritance promotes code reusability.
- Child classes override parent behavior.
- `super()` helps access parent class features.
- Python supports multiple inheritance & MRO.

Practice Section

ready_to_practice: You can apply inheritance concepts.

description: Extend a parent class and override methods.

mcqs: Attempt MCQs 16–20.

coding_challenges: Create a multilevel inheritance example.

★ Lecture 5 — Polymorphism & Operator Overloading

Topic

"Method Overloading, Method Overriding, Duck Typing, Runtime Polymorphism, Magic Methods, Operator Overloading."

Definition (Detailed & Multi-Line)

Polymorphism means "many forms" — the same function name behaves differently depending on object/class.

Python supports:

- Method overriding (runtime polymorphism)
- Duck typing ("if it walks like a duck...")
- Operator overloading via magic methods (`__add__`, `__sub__`)

Python does not support true method overloading, but you can simulate it using default parameters.

Magic methods enable customizing object behavior with built-in operators.

Syntax

```
def __add__(self, other):  
    ...
```

Examples

Example 1 — Polymorphism using common method

```
class Dog:
    def sound(self):
        print("Bark")

class Cat:
    def sound(self):
        print("Meow")

for obj in (Dog(), Cat()):
    obj.sound()
```

Example 2 — Operator Overloading

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other.value
```

Example 3 — Simulated Overloading

```
def add(a, b=0, c=0):
    return a + b + c
```

Key Takeaways

- Overriding → runtime polymorphism
- Duck typing → Pythonic polymorphism
- Operator overloading customizes +, -, *, ==
- Magic methods control object behavior

Practice Section

ready_to_practice: You can create polymorphic behavior.

description: Implement overridden methods & overloaded operators.

mcqs: Attempt MCQs 21–25.

coding_challenges: Implement class with overloaded + operator.

★ Lecture 6 — Abstraction, Composition, Aggregation, and MRO

Topic

"Abstract Classes, Interfaces, HAS-A Relationships, Object Composition, Aggregation & Python's Method Resolution Order (MRO)."

Definition (Detailed & Multi-Line)

Abstraction hides internal details and exposes only essential functionalities using abstract classes.

Composition represents strong HAS-A relationship where a class owns another object.

Aggregation represents weak HAS-A relationship where one class uses another class object but does not control its lifecycle.

MRO (Method Resolution Order) determines how Python resolves methods when using multiple inheritance. It follows C3 Linearization.

Using abstraction, composition, and MRO helps structure large applications.

Syntax

Abstract class:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

Composition:

```
class Car:
    def __init__(self):
        self.engine = Engine()
```

Aggregation:

```
class Team:
    def __init__(self, players):
        self.players = players
```

Examples

Example 1 — Abstract Class

```
class Circle(Shape):
    def area(self):
        return 3.14 * 5 * 5
```

Example 2 — Composition

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()
```

Example 3 — MRO

```
class A: pass
class B(A): pass
class C(B): pass

print(C.mro())
```

Key Takeaways

- Abstraction hides implementation details
- Composition is a strong HAS-A
- Aggregation is a weak HAS-A
- MRO defines method lookup order in multiple inheritance

Practice Section

ready_to_practice: You can apply abstraction & composition.

description: Implement a Shape abstract class with multiple subclasses.

mcqs: Attempt MCQs 26–30.

coding_challenges: Implement composition in a Car–Engine model.

MODULE 6 LECTURES COMPLETED

You now have:

- ✓ Full Theory
- ✓ 25 Examples
- ✓ 6 Lectures
- ✓ Ready for MCQs & Coding Tasks

What's Next?

- Module 6 MCQs (30)

- Module 6 Coding Tasks (20)
- Start Module 7 Theory (File Handling)

MODULE 6 — MCQs (30 Questions with Answers + Explanations)

★ MCQ 1. What is a class in Python?

- a) A function
- b) A blueprint for creating objects
- c) A variable
- d) A module

Answer: b

Explanation: A class is a template that defines the structure and behavior (attributes and methods) for creating objects.

★ MCQ 2. Which method is automatically called when an object is created?

- a) `__start__()`
- b) `__create__()`
- c) `__init__()`
- d) `__new__()`

Answer: c

Explanation: `__init__()` is the constructor method that initializes object attributes when an instance is created.

★ MCQ 3. What is the first parameter of an instance method?

- a) `cls`
- b) `this`

- c) self
- d) object

Answer: c

Explanation: `self` refers to the instance of the class and must be the first parameter in instance methods.

★ MCQ 4. Which decorator is used for class methods?

- a) `@staticmethod`
- b) `@classmethod`
- c) `@property`
- d) `@method`

Answer: b

Explanation: `@classmethod` decorator is used to define methods that work with class-level data using `cls`.

★ MCQ 5. What does the `super()` function do?

- a) Creates a new object
- b) Calls parent class methods
- c) Deletes an object
- d) Overrides a method

Answer: b

Explanation: `super()` is used to call methods or constructors from the parent class in inheritance.

★ MCQ 6. Which access modifier makes a variable private in Python?

- a) private keyword
- b) Single underscore `_`
- c) Double underscore `__`
- d) Triple underscore `___`

Answer: c

Explanation: Double underscore prefix (`__variable`) makes variables private through name mangling.

★ MCQ 7. What is method overriding?

- a) Creating multiple methods with same name
- b) Child class redefining parent class method
- c) Calling parent method from child
- d) Deleting a method

Answer: b

Explanation: Method overriding occurs when a child class provides its own implementation of a parent's method.

★ MCQ 8. Which type of inheritance has one parent and multiple children?

- a) Single
- b) Multiple
- c) Multilevel
- d) Hierarchical

Answer: d

Explanation: Hierarchical inheritance is when multiple classes inherit from one parent class.

★ MCQ 9. What is polymorphism?

- a) Many classes
- b) Many forms of the same function
- c) Many objects
- d) Many variables

Answer: b

Explanation: Polymorphism allows methods to have different behaviors based on the object calling them.

★ MCQ 10. Which magic method is used for operator overloading of +?

- a) `__plus__()`
- b) `__add__()`
- c) `__sum__()`
- d) `__addition__()`

Answer: b

Explanation: `__add__()` magic method defines custom behavior for the + operator.

★ MCQ 11. What is encapsulation?

- a) Hiding implementation details
- b) Creating objects
- c) Inheriting properties
- d) Overriding methods

Answer: a

Explanation: Encapsulation wraps data and methods together and restricts direct access to internal details.

★ MCQ 12. Which keyword is used to inherit a class?

- a) `extends`
- b) `inherits`
- c) Class name in parentheses
- d) `import`

Answer: c

Explanation: In Python, inheritance is done by passing the parent class name in parentheses: `class Child(Parent):`.

★ MCQ 13. What does MRO stand for?

- a) Method Return Order
- b) Method Resolution Order
- c) Multiple Return Objects
- d) Module Resolution Order

Answer: b

Explanation: MRO determines the order in which Python searches for methods in the inheritance hierarchy.

★ MCQ 14. Which is NOT a type of method in Python?

- a) Instance method
- b) Class method
- c) Static method
- d) Global method

Answer: d

Explanation: Python has instance, class, and static methods. There's no "global method" type.

★ MCQ 15. What is an abstract class?

- a) A class with no methods
- b) A class that cannot be instantiated
- c) A class with only variables
- d) A class without constructor

Answer: b

Explanation: Abstract classes contain abstract methods and cannot be instantiated directly.

★ MCQ 16. Which module is required for creating abstract classes?

- a) abstract
- b) abc

- c) `abstractclass`
- d) `base`

Answer: b

Explanation: The `abc` module provides `ABC` class and `abstractmethod` decorator.

★ MCQ 17. What is composition in OOP?

- a) Combining multiple classes
- b) Strong "has-a" relationship
- c) Inheriting from multiple classes
- d) Overriding methods

Answer: b

Explanation: Composition is when one class contains an object of another class and owns it.

★ MCQ 18. What does `__str__()` method return?

- a) Number
- b) String representation of object
- c) Boolean
- d) List

Answer: b

Explanation: `__str__()` returns a human-readable string representation when object is printed.

★ MCQ 19. Which is true about static methods?

- a) Can access instance variables
- b) Can access class variables using `cls`
- c) Cannot access instance or class variables directly
- d) Must have `self` parameter

Answer: c

Explanation: Static methods don't have access to `self` or `cls`, acting like regular functions.

★ MCQ 20. What is the output?

```
class A:  
    x = 10  
  
print(A.x)
```

- a) Error
- b) 10
- c) None
- d) x

Answer: b

Explanation: Class variables can be accessed using the class name without creating an instance.

★ MCQ 21. Multiple inheritance means:

- a) One child, one parent
- b) One child, multiple parents
- c) Multiple children, one parent
- d) No inheritance

Answer: b

Explanation: Multiple inheritance is when a class inherits from more than one parent class.

★ MCQ 22. What is the output?

```
class Dog:  
    def __init__(self):  
        self.breed = "Labrador"  
  
d = Dog()  
print(d.breed)
```

- a) Error
- b) Labrador
- c) None
- d) Dog

Answer: b

Explanation: The constructor initializes `breed` attribute which is then accessed via the object.

★ MCQ 23. Which is true about instance variables?

- a) Shared among all objects
- b) Unique to each object
- c) Cannot be modified
- d) Must be integers

Answer: b

Explanation: Instance variables are unique to each object and can have different values.

★ MCQ 24. What does `@property` decorator do?

- a) Makes method static
- b) Allows method to be accessed like an attribute
- c) Makes method private
- d) Creates class method

Answer: b

Explanation: `@property` allows calling a method without parentheses, like an attribute.

★ MCQ 25. What is the output?

```
class A:
    pass

class B(A):
    pass

print(isinstance(B(), A))
```

- a) True
- b) False
- c) Error
- d) None

Answer: a

Explanation: `isinstance()` returns `True` because B inherits from A, so B() is an instance of A.

★ MCQ 26. Which method is called when `len(object)` is used?

- a) `__length__()`
- b) `__len__()`
- c) `__size__()`
- d) `__count__()`

Answer: b

Explanation: `__len__()` magic method defines behavior for the `len()` function.

★ MCQ 27. What is aggregation?

- a) Strong ownership
- b) Weak "has-a" relationship
- c) Inheritance
- d) Polymorphism

Answer: b

Explanation: Aggregation is a weak relationship where contained objects can exist independently.

★ MCQ 28. What does `__init__` return?

- a) `self`
- b) Object reference
- c) `None`
- d) `True`

Answer: c

Explanation: Constructors in Python should not return any value (implicitly return `None`).

★ MCQ 29. Which is NOT an OOP principle?

- a) Encapsulation
- b) Inheritance
- c) Compilation
- d) Polymorphism

Answer: c

Explanation: Compilation is not an OOP principle. The four pillars are: Encapsulation, Abstraction, Inheritance, and Polymorphism.

★ MCQ 30. What is the output?

```
class Test:
    count = 0

    def __init__(self):
        Test.count += 1

t1 = Test()
t2 = Test()
print(Test.count)
```

- a) 0
- b) 1
- c) 2
- d) Error

Answer: c

Explanation: Class variable `count` is shared and incremented each time an object is created.

30 MCQs Completed

MODULE 6 — CODING TASKS (20
Exercises with Solutions)

★ Task 1 — Create a Simple Class with Constructor

Problem: Create a class `Person` with attributes `name` and `age`. Initialize them using constructor and display them.

Solution:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

# Test
p = Person("Hemanth", 25)
p.display()
```

Output:

Name: Hemanth, Age: 25

Explanation: Constructor `__init__` initializes instance variables. The `display()` method prints them.

★ Task 2 — Class with Instance and Class Variables

Problem: Create a class `Employee` with a class variable `company` and instance variables `name` and `salary`.

Solution:

```
class Employee:
    company = "TCS" # Class variable

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def display(self):
        print(f"Company: {Employee.company}, Name: {self.name}, Salary: {self.salary}")

# Test
e1 = Employee("Rahul", 50000)
e2 = Employee("Priya", 60000)
e1.display()
```

```
e2.display()
```

Output:

```
Company: TCS, Name: Rahul, Salary: 50000  
Company: TCS, Name: Priya, Salary: 60000
```

Explanation: Class variable `company` is shared among all objects, while instance variables are unique.

★ Task 3 — Implement Class Method and Static Method

Problem: Create a class `Calculator` with a class method to display class info and a static method to add two numbers.

Solution:

```
class Calculator:  
    name = "Scientific Calculator"  
  
    @classmethod  
    def info(cls):  
        print(f"This is {cls.name}")  
  
    @staticmethod  
    def add(a, b):  
        return a + b  
  
# Test  
Calculator.info()  
print("Sum:", Calculator.add(10, 20))
```

Output:

```
This is Scientific Calculator  
Sum: 30
```

Explanation: Class methods use `@classmethod` and access class variables via `cls`. Static methods use `@staticmethod` and don't need class/instance.

★ Task 4 — Encapsulation with Private Variables

Problem: Create a `BankAccount` class with private balance. Provide methods to deposit and withdraw money.

Solution:

```
class BankAccount:
    def __init__(self, balance=0):
        self.__balance = balance

    def deposit(self, amount):
        self.__balance += amount
        print(f"Deposited: {amount}")

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrawn: {amount}")
        else:
            print("Insufficient balance")

    def get_balance(self):
        return self.__balance

# Test
account = BankAccount(1000)
account.deposit(500)
account.withdraw(300)
print(f"Current Balance: {account.get_balance()}")
```

Output:

```
Deposited: 500
Withdrawn: 300
Current Balance: 1200
```

Explanation: Private variable `__balance` is accessed only through methods, ensuring data protection.

★ Task 5 — Single Inheritance

Problem: Create a parent class `Animal` with a method `sound()` and a child class `Dog` that inherits from it.

Solution:

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

# Test
d = Dog()
d.sound()
d.bark()
```

Output:

```
Animal makes a sound  
Dog barks
```

Explanation: Dog inherits from Animal and can use its methods. Dog also has its own method `bark()`.

★ Task 6 — Method Overriding

Problem: Override a parent class method in the child class.

Solution:

```
class Parent:
    def display(self):
        print("This is Parent class")

class Child(Parent):
    def display(self):
        print("This is Child class")

# Test
c = Child()
c.display()
```

Output:

```
This is Child class
```

Explanation: Child class overrides the `display()` method, so calling it on Child object executes Child's version.

★ Task 7 — Using `super()` to Call Parent Constructor

Problem: Use `super()` to call parent class constructor from child class.

Solution:

```
class Parent:
    def __init__(self, name):
        self.name = name
        print(f"Parent constructor: {self.name}")

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name)
```

```
        self.age = age
        print(f"Child constructor: Age = {self.age}")

# Test
c = Child("Hemanth", 25)
```

Output:

```
Parent constructor: Hemanth
Child constructor: Age = 25
```

Explanation: `super().__init__(name)` calls parent's constructor before initializing child's attributes.

★ Task 8 — Multilevel Inheritance

Problem: Create a multilevel inheritance: `Vehicle` → `Car` → `SportsCar`.

Solution:

```
class Vehicle:
    def vehicle_info(self):
        print("This is a vehicle")

class Car(Vehicle):
    def car_info(self):
        print("This is a car")

class SportsCar(Car):
    def sports_car_info(self):
        print("This is a sports car")

# Test
sc = SportsCar()
sc.vehicle_info()
sc.car_info()
sc.sports_car_info()
```

Output:

```
This is a vehicle
This is a car
This is a sports car
```

Explanation: `SportsCar` inherits from `Car`, which inherits from `Vehicle`, forming a chain.

★ Task 9 — Multiple Inheritance

Problem: Create two parent classes and one child class that inherits from both.

Solution:

```
class Father:
    def gardening(self):
        print("Father likes gardening")

class Mother:
    def cooking(self):
        print("Mother likes cooking")

class Child(Father, Mother):
    def playing(self):
        print("Child likes playing")

# Test
c = Child()
c.gardening()
c.cooking()
c.playing()
```

Output:

```
Father likes gardening
Mother likes cooking
Child likes playing
```

Explanation: Child inherits from both Father and Mother, accessing methods from both parents.

★ Task 10 — Polymorphism with Method Overriding

Problem: Create multiple classes with the same method name and call them polymorphically.

Solution:

```
class Dog:
    def sound(self):
        print("Bark")

class Cat:
    def sound(self):
        print("Meow")

class Cow:
    def sound(self):
        print("Moo")

# Test
animals = [Dog(), Cat(), Cow()]

for animal in animals:
```

```
animal.sound()
```

Output:

```
Bark
Meow
Moo
```

Explanation: Same method name `sound()` behaves differently based on the object type.

★ Task 11 — Operator Overloading for +

Problem: Overload the + operator to add two complex numbers.

Solution:

```
class ComplexNumber:
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag

    def __add__(self, other):
        return ComplexNumber(self.real + other.real, self.imag +
other.imag)

    def display(self):
        print(f"{self.real} + {self.imag}i")

# Test
c1 = ComplexNumber(3, 4)
c2 = ComplexNumber(1, 2)
c3 = c1 + c2
c3.display()
```

Output:

```
4 + 6i
```

Explanation: `__add__()` magic method allows using + operator between ComplexNumber objects.

★ Task 12 — Implement `__str__()` Magic Method

Problem: Define `__str__()` to provide a readable string representation of an object.

Solution:

```
class Book:
```



```

def __init__(self, title, author):
    self.title = title
    self.author = author

def __str__(self):
    return f"Book: {self.title} by {self.author}"

# Test
b = Book("Python Programming", "John Doe")
print(b)

```

Output:

Book: Python Programming by John Doe

Explanation: `__str__()` returns a string representation when the object is printed.

★ Task 13 — Abstract Class with Abstract Method

Problem: Create an abstract class `Shape` with abstract method `area()`. Create subclasses `Circle` and `Rectangle`.

Solution:

```

from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

# Test
c = Circle(5)
r = Rectangle(4, 6)
print(f"Circle Area: {c.area()}")
print(f"Rectangle Area: {r.area()}")

```

Output:

Circle Area: 78.5
Rectangle Area: 24

Explanation: Abstract class forces subclasses to implement `area()` method.

★ Task 14 — Composition (HAS-A Relationship)

Problem: Create a `Car` class that contains an `Engine` object.

Solution:

```
class Engine:
    def start(self):
        print("Engine started")

    def stop(self):
        print("Engine stopped")

class Car:
    def __init__(self):
        self.engine = Engine() # Composition

    def drive(self):
        self.engine.start()
        print("Car is driving")

    def park(self):
        print("Car is parked")
        self.engine.stop()

# Test
car = Car()
car.drive()
car.park()
```

Output:

Engine started
Car is driving
Car is parked
Engine stopped

Explanation: Car "has-a" Engine. The Engine object is owned by Car.

★ Task 15 — Aggregation (Weak HAS-A)

Problem: Create a `Teacher` and `Department` class showing aggregation.

Solution:

```
class Teacher:
    def __init__(self, name):
        self.name = name

    def info(self):
        print(f"Teacher: {self.name}")

class Department:
    def __init__(self, name, teachers):
        self.name = name
        self.teachers = teachers # Aggregation

    def show_teachers(self):
        print(f"Department: {self.name}")
        for teacher in self.teachers:
            teacher.info()

# Test
t1 = Teacher("Dr. Smith")
t2 = Teacher("Dr. Johnson")
dept = Department("Computer Science", [t1, t2])
dept.show_teachers()
```

Output:

```
Department: Computer Science
Teacher: Dr. Smith
Teacher: Dr. Johnson
```

Explanation: Department uses teachers but doesn't own them. Teachers can exist independently.

★ Task 16 — Property Decorator (Getter and Setter)

Problem: Use `@property` decorator to create getter and setter for a private variable.

Solution:

```
class Student:
    def __init__(self, marks):
        self._marks = marks

    @property
    def marks(self):
        return self._marks

    @marks.setter
    def marks(self, value):
        if 0 <= value <= 100:
            self._marks = value
        else:
```

```
        print("Invalid marks")

# Test
s = Student(85)
print(f"Marks: {s.marks}")
s.marks = 95
print(f"Updated Marks: {s.marks}")
s.marks = 150 # Invalid
```

Output:

```
Marks: 85
Updated Marks: 95
Invalid marks
```

Explanation: `@property` allows accessing methods like attributes. Setter validates input.

★ Task 17 — Method Resolution Order (MRO)

Problem: Display the MRO for a class with multiple inheritance.

Solution:

```
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass

# Test
print("MRO for D:")
print(D.mro())
```

Output:

```
MRO for D:
[<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>,
<class '__main__.A'>, <class 'object'>]
```

Explanation: MRO shows the order Python searches for methods in inheritance hierarchy using C3 Linearization.

★ Task 18 — Student Database Using OOP

Problem: Create a `Student` class and manage multiple students in a list.

Solution:

```
class Student:
    def __init__(self, name, roll, marks):
        self.name = name
        self.roll = roll
        self.marks = marks

    def display(self):
        print(f"Roll: {self.roll}, Name: {self.name}, Marks: {self.marks}")

# Create students
students = [
    Student("Hemanth", 101, 85),
    Student("Rahul", 102, 90),
    Student("Priya", 103, 88)
]

# Display all students
print("Student Database:")
for student in students:
    student.display()
```

Output:

```
Student Database:
Roll: 101, Name: Hemanth, Marks: 85
Roll: 102, Name: Rahul, Marks: 90
Roll: 103, Name: Priya, Marks: 88
```

Explanation: Objects are stored in a list and accessed using a loop.

★ Task 19 — Bank System with Multiple Accounts

Problem: Create a banking system to manage multiple accounts with deposit and withdrawal.

Solution:

```
class BankAccount:
    def __init__(self, account_number, holder_name, balance=0):
        self.account_number = account_number
        self.holder_name = holder_name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount}. New Balance: {self.balance}")
```

```

def withdraw(self, amount):
    if amount <= self.balance:
        self.balance -= amount
        print(f"Withdrawn {amount}. New Balance: {self.balance}")
    else:
        print("Insufficient balance")

def display(self):
    print(f"Account: {self.account_number}, Holder: {self.holder_name}, Balance: {self.balance}")

# Test
acc1 = BankAccount("A001", "Hemanth", 5000)
acc2 = BankAccount("A002", "Rahul", 3000)

acc1.deposit(1000)
acc1.withdraw(2000)
acc1.display()

acc2.deposit(500)
acc2.display()

```

Output:

```

Deposited 1000. New Balance: 6000
Withdrawn 2000. New Balance: 4000
Account: A001, Holder: Hemanth, Balance: 4000
Deposited 500. New Balance: 3500
Account: A002, Holder: Rahul, Balance: 3500

```

Explanation: Multiple account objects are created and managed independently.

★ Task 20 — Library Management System

Problem: Create a library system with `Book` and `Library` classes.

Solution:

```

class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn

    def display(self):
        print(f"Title: {self.title}, Author: {self.author}, ISBN: {self.isbn}")

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)
        print(f"Added: {book.title}")

```

```

def display_books(self):
    print("\n=== Library Books ===")
    for book in self.books:
        book.display()

def search_book(self, title):
    for book in self.books:
        if book.title.lower() == title.lower():
            print("Book found:")
            book.display()
            return
    print("Book not found")

# Test
library = Library()
library.add_book(Book("Python Programming", "John Doe", "12345"))
library.add_book(Book("Data Science", "Jane Smith", "67890"))
library.display_books()
library.search_book("Python Programming")

```

Output:

```

Added: Python Programming
Added: Data Science

=== Library Books ===
Title: Python Programming, Author: John Doe, ISBN: 12345
Title: Data Science, Author: Jane Smith, ISBN: 67890
Book found:
Title: Python Programming, Author: John Doe, ISBN: 12345

```

Explanation: Library manages multiple Book objects, demonstrating composition and object management.

Module 6 Coding Tasks Completed (20)

MODULE 6 COMPLETE ✓✓

You now have:

- ✓ 6 Detailed Lectures
- ✓ 4000+ word Theory
- ✓ 25 Code Examples
- ✓ 30 MCQs with Explanations
- ✓ 20 Coding Tasks with Solutions

- ✓ Complete LMS-Ready Content
- ✓ Copy-Paste Ready Format

MODULE 7 — FILE HANDLING IN PYTHON

Basic Understanding

Introduction

File handling allows Python programs to read, write, create, and manage files stored on your computer. It is essential for:

- Storing user data
- Saving logs
- Reading input from external files
- Writing outputs
- Handling configuration files
- Working with datasets
- Building backend systems

File handling makes programs persistent, meaning they retain data even after execution ends.

★ 1. What Is a File?

A file is a container on storage (disk) that holds data. Files can be:

Text files (.txt, .py, .csv, .json)

Binary files (.jpg, .png, .exe, .mp4)

★ 2. Opening a File in Python

Python uses the built-in `open()` function:

```
open("filename", "mode")
```

Common Modes:

Mode	Meaning
"r"	Read (default)

Mode	Meaning
"w"	Write (overwrite)
"a"	Append (add to end)
"r+"	Read + Write
"wb"	Write binary
"rb"	Read binary

Example:

```
file = open("data.txt", "r")
```

★ 3. Reading from a File

There are three main ways:

✓ Read entire file

```
data = file.read()
```

✓ Read line by line

```
line = file.readline()
```

✓ Read all lines as list

```
lines = file.readlines()
```

★ 4. Writing to a File

Writing creates or overwrites content:

```
file = open("notes.txt", "w")  
file.write("Hello Python!")
```

Appending adds to the end without deleting the previous content:

```
file = open("notes.txt", "a")  
file.write("\nNew line added")
```

★ 5. Closing a File

Always close the file to release resources:

```
file.close()
```

If you forget, Python may not save the data properly.

★ 6. Using `with` Statement (Best Practice)

Using `with` automatically closes the file:

```
with open("data.txt", "r") as file:
    data = file.read()
```

This is safer and recommended for all file operations.

★ 7. Handling File Errors

Always be cautious of:

- File not found
- Permission issues
- Missing directories
- Trying to read closed files

Basic error handling:

```
try:
    f = open("sample.txt")
    print(f.read())
except FileNotFoundError:
    print("File not found!")
```

★ 8. Difference Between Text & Binary Files

Text file ("**r**" / "**w**"):

- Stores readable characters
- Example: `.txt`, `.py`, `.csv`

Binary file ("**rb**" / "**wb**"):

- Stores raw bytes
- Example: `.jpg`, `.exe`, `.mp4`

Reading an image:

```
with open("photo.jpg", "rb") as f:
    img_data = f.read()
```

★ 9. Why File Handling Is Important?

Because real-world applications must store and retrieve data.

Examples:

- ✓ Saving user profiles
- ✓ Keeping logs in backend systems
- ✓ Reading CSV/JSON datasets
- ✓ Writing reports or outputs
- ✓ Saving program configurations
- ✓ Uploading / downloading files in web apps
- ✓ Processing text/binary files

Without file handling, programs lose data after execution.

MODULE 7 – PART 1 COMPLETE ✓

This section covered:

- What files are
 - Opening files with modes
 - Reading from files (3 methods)
 - Writing and appending to files
 - Closing files properly
 - Using `with` statement
 - Error handling
 - Text vs Binary files
 - Real-world importance
-

What's Next?

- Module 7 Examples (25 snippets)
- Module 7 Lectures (6 detailed lectures)
- Module 7 MCQs (30 questions)
- Module 7 Coding Tasks (20 exercises)

MODULE 7 — EXAMPLES & SNIPPETS (25 Examples)

File Handling in Python

Example 1: Opening and Reading a File

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

Explanation: Opens a file in read mode, reads entire content, and closes it.

Example 2: Using `with` Statement

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

Explanation: Automatically closes the file after the block executes. Best practice.

Example 3: Reading Line by Line

```
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Explanation: Iterates through each line. `strip()` removes extra whitespace.

Example 4: Using `readline()`

```
with open("data.txt", "r") as file:
    line1 = file.readline()
    line2 = file.readline()
    print("First line:", line1)
    print("Second line:", line2)
```

Explanation: Reads one line at a time with each `readline()` call.

Example 5: Using `readlines()`

```
with open("data.txt", "r") as file:
    lines = file.readlines()
    print(lines)
```

Explanation: Returns all lines as a list. Each line is a list element.

Example 6: Writing to a File (Overwrite)

```
with open("output.txt", "w") as file:
    file.write("Hello, Python!\n")
    file.write("File handling is easy.")
```

Explanation: Creates or overwrites the file with new content.

Example 7: Appending to a File

```
with open("output.txt", "a") as file:
    file.write("\nThis line is appended.")
```

Explanation: Adds content to the end without deleting existing data.

Example 8: Writing Multiple Lines

```
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]

with open("output.txt", "w") as file:
    file.writelines(lines)
```

Explanation: `writelines()` writes a list of strings to the file.

Example 9: Checking if File Exists

```
import os

if os.path.exists("data.txt"):
    print("File exists")
else:
    print("File not found")
```

Explanation: Uses `os.path.exists()` to check file existence before operations.

Example 10: Handling File Not Found Error

```
try:
    with open("missing.txt", "r") as file:
        content = file.read()
except FileNotFoundError:
    print("Error: File not found!")
```

Explanation: Catches `FileNotFoundError` to handle missing files gracefully.

Example 11: Reading and Writing in Same File

```
with open("data.txt", "r+") as file:
    content = file.read()
    print("Original:", content)
    file.write("\nNew content added")
```

Explanation: "r+" mode allows both reading and writing.

Example 12: Copying File Content

```
with open("source.txt", "r") as source:
    with open("destination.txt", "w") as destination:
        content = source.read()
        destination.write(content)

print("File copied successfully")
```

Explanation: Reads from one file and writes to another, creating a copy.

Example 13: Counting Lines in a File

```
with open("data.txt", "r") as file:
    lines = file.readlines()
    print(f"Total lines: {len(lines)}")
```

Explanation: Uses `readlines()` to count total number of lines.

Example 14: Counting Words in a File

```
with open("data.txt", "r") as file:
    content = file.read()
    words = content.split()
```

```
print(f"Total words: {len(words)}")
```

Explanation: Splits content by whitespace to count words.

Example 15: Reading Binary File (Image)

```
with open("image.jpg", "rb") as file:  
    data = file.read()  
    print(f"File size: {len(data)} bytes")
```

Explanation: "rb" mode reads binary files like images.

Example 16: Writing Binary File

```
data = b"Binary data content"  
  
with open("output.bin", "wb") as file:  
    file.write(data)  
  
print("Binary file created")
```

Explanation: "wb" mode writes binary data to file.

Example 17: Deleting a File

```
import os  
  
if os.path.exists("temp.txt"):  
    os.remove("temp.txt")  
    print("File deleted")  
else:  
    print("File not found")
```

Explanation: `os.remove()` deletes a file from the system.

Example 18: Renaming a File

```
import os  
  
if os.path.exists("old_name.txt"):  
    os.rename("old_name.txt", "new_name.txt")  
    print("File renamed")
```

Explanation: `os.rename()` changes the file name.

Example 19: Getting File Size

```
import os

if os.path.exists("data.txt"):
    size = os.path.getsize("data.txt")
    print(f"File size: {size} bytes")
```

Explanation: `os.path.getsize()` returns file size in bytes.

Example 20: Reading Specific Lines

```
with open("data.txt", "r") as file:
    lines = file.readlines()
    print("Line 3:", lines[2]) # Index 2 = 3rd line
```

Explanation: Reads all lines and accesses specific line by index.

Example 21: Creating Directory

```
import os

if not os.path.exists("my_folder"):
    os.mkdir("my_folder")
    print("Directory created")
```

Explanation: `os.mkdir()` creates a new directory.

Example 22: Listing Files in Directory

```
import os

files = os.listdir(".")
print("Files in current directory:")
for file in files:
    print(file)
```

Explanation: `os.listdir()` returns list of files in specified directory.

Example 23: File Pointer Position

```
with open("data.txt", "r") as file:
    print("Position:", file.tell()) # Shows current position
    file.read(10)
    print("After reading 10 chars:", file.tell())
```

Explanation: `tell()` returns current position in file.

Example 24: Seeking in File

```
with open("data.txt", "r") as file:
    file.seek(10) # Move to 10th byte
    content = file.read()
    print(content)
```

Explanation: `seek()` moves file pointer to specific position.

Example 25: Working with CSV Files

```
import csv

# Writing CSV
with open("data.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Age", "City"])
    writer.writerow(["Hemanth", "25", "Hyderabad"])

# Reading CSV
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Explanation: Uses `csv` module to read and write CSV files.

MODULE 7 EXAMPLES COMPLETED (25)

Topics Covered:

- Basic file operations
- Reading methods
- Writing and appending
- Error handling
- Binary files
- File management (delete, rename)
- Directory operations
- CSV handling
- File pointers

What's Next?

- Module 7 Lectures (6 detailed lectures)
- Module 7 MCQs (30 questions)
- Module 7 Coding Tasks (20 exercises)

MODULE 7 — LECTURES

File Handling in Python

★ Lecture 1 — Introduction to File Handling & File Modes

Topic

"Understanding Files, File Types, Opening Files, and Different File Modes in Python."

Definition (Detailed & Multi-Line)

File handling is the process of reading, writing, creating, and managing files on storage using Python.

A file is a named location on disk that stores data persistently.

Python provides built-in functions like `open()`, `read()`, `write()`, and `close()` for file operations.

Files can be text files (readable characters) or binary files (raw bytes).

File modes determine how a file is opened: read-only, write, append, or binary.

Understanding file modes is crucial for preventing data loss and ensuring proper file operations.

Syntax

```
open(filename, mode)
```

Common Modes:

- "r" - Read (default)
- "w" - Write (overwrite)
- "a" - Append
- "r+" - Read and Write
- "rb" - Read binary
- "wb" - Write binary

Examples

Example 1 — Opening File in Read Mode

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

Example 2 — Opening File in Write Mode

```
file = open("output.txt", "w")
file.write("Hello World")
file.close()
```

Example 3 — Opening Binary File

```
with open("image.jpg", "rb") as file:
    data = file.read()
```

Key Takeaways

- Files persist data beyond program execution
- Always specify correct mode to avoid data loss
- "w" mode overwrites existing content
- "a" mode adds to the end
- Binary mode is used for images, videos, executables

Practice Section

ready_to_practice: You understand file modes and opening files.

description: Open different files in various modes and observe behavior.

mcqs: Attempt MCQs 1–5.

coding_challenges: Write a program to open and read a text file.

★ Lecture 2 — Reading from Files

Topic

"Different Methods to Read File Content: read(), readline(), readlines(), and Line-by-Line Reading."

Definition (Detailed & Multi-Line)

Reading files allows programs to access stored data and process it.

Python provides multiple methods for reading files based on requirements.

`read()` reads the entire file content as a single string.

`readline()` reads one line at a time, useful for processing large files.

`readlines()` reads all lines and returns them as a list.

You can also iterate through file object directly to read line by line efficiently.

Syntax

```
# Read entire file
content = file.read()

# Read one line
line = file.readline()

# Read all lines as list
lines = file.readlines()

# Iterate line by line
for line in file:
    print(line)
```

Examples

Example 1 — Using read()

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

Example 2 — Using readline()

```
with open("data.txt", "r") as file:
    line1 = file.readline()
    line2 = file.readline()
    print(line1, line2)
```

Example 3 — Using readlines()

```
with open("data.txt", "r") as file:
    lines = file.readlines()
    for line in lines:
        print(line.strip())
```

Example 4 — Line by Line Iteration

```
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Key Takeaways

- `read()` loads entire file into memory
- `readline()` is memory efficient for large files
- `readlines()` returns a list of all lines
- Direct iteration is most Pythonic
- Always use `strip()` to remove extra whitespace

Practice Section

ready_to_practice: You can read files using different methods.

description: Practice reading files and processing line by line.

mcqs: Attempt MCQs 6–10.

coding_challenges: Count lines, words, and characters in a file.

★ Lecture 3 — Writing and Appending to Files

Topic

"Writing Data to Files, Overwriting Content, Appending Data, and Using `write()` and `writelines()` Methods."

Definition (Detailed & Multi-Line)

Writing to files allows programs to save output, logs, and user data persistently.

"w" mode creates a new file or overwrites existing content completely.

"a" mode adds content to the end of file without deleting existing data.

`write()` method writes a single string to the file.

`writelines()` method writes a list of strings to the file.

Always include newline characters (`\n`) manually when needed.

Syntax

```
# Write mode (overwrite)
with open("file.txt", "w") as file:
    file.write("Content")

# Append mode (add to end)
with open("file.txt", "a") as file:
    file.write("\nNew line")

# Write multiple lines
lines = ["Line 1\n", "Line 2\n"]
with open("file.txt", "w") as file:
    file.writelines(lines)
```

Examples

Example 1 — Writing to File

```
with open("output.txt", "w") as file:
    file.write("Hello Python\n")
    file.write("File handling is easy")
```

Example 2 — Appending to File

```
with open("output.txt", "a") as file:
    file.write("\nAppended line")
```

Example 3 — Using `writelines()`

```
lines = ["First line\n", "Second line\n", "Third line\n"]
with open("output.txt", "w") as file:
    file.writelines(lines)
```

Key Takeaways

- `"w"` mode deletes existing content
- `"a"` mode preserves existing content
- Always add `\n` for new lines manually
- `writelines()` doesn't add newlines automatically
- Use `with` statement to ensure file is saved

Practice Section

ready_to_practice: You can write and append to files.

description: Create programs that save user input to files.

mcqs: Attempt MCQs 11–15.

coding_challenges: Write a program to log user activities to a file.

★ Lecture 4 — The `with` Statement and File Context Manager

Topic

"Understanding Context Managers, Automatic File Closing, Best Practices, and Exception Safety with `with` Statement."

Definition (Detailed & Multi-Line)

The `with` statement is a context manager that automatically handles resource management.

It ensures files are properly closed even if errors occur during file operations.

Without `with`, you must manually call `close()` which is error-prone.

Using `with` is considered best practice and makes code cleaner and safer.

Context managers handle both opening and closing of files automatically.

This prevents resource leaks and ensures data is flushed to disk.

Syntax

```
# With context manager (recommended)
with open("file.txt", "r") as file:
    content = file.read()
# File is automatically closed here

# Without context manager (not recommended)
file = open("file.txt", "r")
content = file.read()
file.close() # Must remember to close
```

Examples

Example 1 — Basic `with` Statement

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
# File automatically closed
```


Example 2 — Multiple Files

```
with open("input.txt", "r") as infile, open("output.txt", "w") as
outfile:
    content = infile.read()
    outfile.write(content.upper())
```

Example 3 — Exception Safety

```
try:
    with open("data.txt", "r") as file:
        content = file.read()
        print(content)
except FileNotFoundError:
    print("File not found")
# File is closed even if exception occurs
```

Key Takeaways

- `with` automatically closes files
- Prevents resource leaks
- Works even if exceptions occur
- More readable and Pythonic
- Can handle multiple files simultaneously

Practice Section

ready_to_practice: You understand the importance of `with` statement.

description: Rewrite file operations using `with` statement.

mcqs: Attempt MCQs 16–20.

coding_challenges: Copy file content using `with` statement.

★ Lecture 5 — Error Handling in File Operations

Topic

"Common File Errors, Exception Handling, `FileNotFoundError`, `PermissionError`, and `IOError`."

Definition (Detailed & Multi-Line)

File operations can fail due to various reasons like missing files, permission issues, or disk errors.

`FileNotFoundError` occurs when trying to open a non-existent file in read mode.

`PermissionError` occurs when lacking permissions to read or write a file.

`IOError` is a general input/output error during file operations.

Using try-except blocks prevents program crashes and provides user-friendly error messages.

Always validate file existence and handle exceptions gracefully.

Syntax

```
try:
    with open("file.txt", "r") as file:
        content = file.read()
except FileNotFoundError:
    print("File not found")
except PermissionError:
    print("Permission denied")
except IOError:
    print("I/O error occurred")
```

Examples

Example 1 — Handling `FileNotFoundError`

```
try:
    with open("missing.txt", "r") as file:
        content = file.read()
except FileNotFoundError:
    print("Error: The file does not exist")
```

Example 2 — Checking File Existence

```
import os

filename = "data.txt"
if os.path.exists(filename):
    with open(filename, "r") as file:
        print(file.read())
else:
    print(f"{filename} not found")
```

Example 3 — Multiple Exception Handling

```
try:
    with open("file.txt", "r") as file:
        content = file.read()
        print(content)
except FileNotFoundError:
    print("File not found")
except PermissionError:
    print("No permission to read file")
except Exception as e:
    print(f"Unexpected error: {e}")
```

```
print(f"Error: {e}")
```

Key Takeaways

- Always handle file exceptions
- Check file existence before operations
- Provide meaningful error messages
- Use specific exception types
- Generic Exception catches all errors

Practice Section

ready_to_practice: You can handle file errors properly.

description: Write robust file handling code with error checking.

mcqs: Attempt MCQs 21–25.

coding_challenges: Create a file reader with comprehensive error handling.

★ Lecture 6 — Working with Binary Files and File Management

Topic

"Binary File Operations, Reading Images, os Module Functions, File Management, Copying, Deleting, and Renaming Files."

Definition (Detailed & Multi-Line)

Binary files contain raw bytes and cannot be read as text.

Examples include images, videos, executables, and audio files.

Use "rb" mode to read and "wb" mode to write binary files.

Python's `os` module provides functions for file management operations.

You can check existence, get file size, delete, rename, and list files using `os` module.

Understanding binary operations is essential for multimedia applications.

Syntax

```
# Binary read
with open("image.jpg", "rb") as file:
```

```

    data = file.read()

# Binary write
with open("copy.jpg", "wb") as file:
    file.write(data)

# File management
import os
os.path.exists("file.txt") # Check existence
os.remove("file.txt")      # Delete
os.rename("old.txt", "new.txt") # Rename
os.path.getsize("file.txt") # Get size

```

Examples

Example 1 — Copying an Image

```

with open("original.jpg", "rb") as source:
    with open("copy.jpg", "wb") as destination:
        destination.write(source.read())
print("Image copied successfully")

```

Example 2 — File Management

```

import os

# Check if file exists
if os.path.exists("data.txt"):
    # Get file size
    size = os.path.getsize("data.txt")
    print(f"File size: {size} bytes")

    # Rename file
    os.rename("data.txt", "backup.txt")
    print("File renamed")

```

Example 3 — Deleting Files

```

import os

filename = "temp.txt"
if os.path.exists(filename):
    os.remove(filename)
    print(f"{filename} deleted")
else:
    print("File not found")

```

Example 4 — Listing Directory Files

```

import os

files = os.listdir(".")
print("Files in current directory:")
for file in files:
    print(file)

```

Key Takeaways

- Binary files require "rb" or "wb" mode
- Use `os` module for file management
- Always check file existence before operations
- Binary files cannot be read as text
- File management operations are OS-specific

Practice Section

ready_to_practice: You can work with binary files and manage files.

description: Practice copying images and managing files programmatically.

mcqs: Attempt MCQs 26–30.

coding_challenges: Create a file backup utility.

MODULE 7 LECTURES COMPLETED

You now have:

- ✓ Full Theory
- ✓ 25 Examples
- ✓ 6 Detailed Lectures
- ✓ Ready for MCQs & Coding Tasks

What's Next?

- Module 7 MCQs (30 questions)
- Module 7 Coding Tasks (20 exercises)

MODULE 7 — MCQs (30 Questions with Answers + Explanations)

★ MCQ 1. Which function is used to open a file in Python?

- a) `read()`
- b) `open()`

- c) file()
- d) load()

Answer: b

Explanation: `open()` is the built-in function used to open files in Python.

★ MCQ 2. What is the default mode when opening a file?

- a) "w"
- b) "a"
- c) "r"
- d) "r+"

Answer: c

Explanation: Read mode "r" is the default when no mode is specified.

★ MCQ 3. Which mode overwrites the file content?

- a) "r"
- b) "w"
- c) "a"
- d) "r+"

Answer: b

Explanation: Write mode "w" creates a new file or completely overwrites existing content.

★ MCQ 4. Which mode is used to add content at the end of a file?

- a) "r"
- b) "w"
- c) "a"
- d) "x"

Answer: c

Explanation: Append mode "a" adds content to the end without deleting existing data.

★ MCQ 5. What does `file.close()` do?

- a) Deletes the file
- b) Closes the file and releases resources
- c) Renames the file
- d) Opens the file

Answer: b

Explanation: `close()` properly closes the file and releases system resources.

★ MCQ 6. Which method reads the entire file content?

- a) `readline()`
- b) `read()`
- c) `readall()`
- d) `fetch()`

Answer: b

Explanation: `read()` reads the entire file content as a single string.

★ MCQ 7. What does `readline()` return after reaching EOF (End of File)?

- a) None
- b) Empty string ""
- c) -1
- d) Error

Answer: b

Explanation: `readline()` returns an empty string when it reaches the end of the file.

★ MCQ 8. Which method returns all lines as a list?

- a) `read()`
- b) `readline()`
- c) `readlines()`
- d) `readall()`

Answer: c

Explanation: `readlines()` reads all lines and returns them as a list.

★ MCQ 9. What is the benefit of using `with` statement?

- a) Faster execution
- b) Automatic file closing
- c) Better formatting
- d) Reduces file size

Answer: b

Explanation: `with` statement automatically closes the file, even if errors occur.

★ MCQ 10. Which exception is raised when a file is not found?

- a) `IOError`
- b) `FileError`
- c) `FileNotFoundError`
- d) `ValueError`

Answer: c

Explanation: `FileNotFoundError` is raised when trying to open a non-existent file.

★ MCQ 11. Which mode is used to read binary files?

- a) "r"
- b) "rb"
- c) "br"
- d) "b"

Answer: b

Explanation: "rb" mode is used to read files in binary format.

★ MCQ 12. What is the output?

```
with open("test.txt", "w") as f:  
    f.write("Hello")  
    f.write("World")
```

- a) Hello World
- b) HelloWorld
- c) Hello\nWorld
- d) Error

Answer: b

Explanation: `write()` doesn't add spaces or newlines automatically, so strings are concatenated.

★ MCQ 13. Which module is used for file management operations?

- a) file
- b) sys
- c) os
- d) io

Answer: c

Explanation: The `os` module provides functions for file and directory management.

★ MCQ 14. How do you check if a file exists?

- a) `os.exists("file.txt")`
- b) `os.path.exists("file.txt")`

- c) `file.exists("file.txt")`
- d) `check_file("file.txt")`

Answer: b

Explanation: `os.path.exists()` checks whether a file or directory exists.

★ MCQ 15. Which function deletes a file?

- a) `os.delete()`
- b) `os.remove()`
- c) `file.delete()`
- d) `delete()`

Answer: b

Explanation: `os.remove()` deletes a file from the file system.

★ MCQ 16. How do you rename a file?

- a) `os.rename(old, new)`
- b) `os.change(old, new)`
- c) `file.rename(old, new)`
- d) `rename(old, new)`

Answer: a

Explanation: `os.rename(old_name, new_name)` renames a file.

★ MCQ 17. What does `file.tell()` return?

- a) File size
- b) Current position in file
- c) Number of lines
- d) File name

Answer: b

Explanation: `tell()` returns the current position of the file pointer.

★ MCQ 18. What does `file.seek(0)` do?

- a) Closes the file
- b) Deletes file content
- c) Moves pointer to beginning
- d) Moves pointer to end

Answer: c

Explanation: `seek(0)` moves the file pointer to the beginning of the file.

★ MCQ 19. Which method writes a list of strings to a file?

- a) `write()`
- b) `writelines()`
- c) `writelists()`
- d) `writeall()`

Answer: b

Explanation: `writelines()` writes a list of strings to the file.

★ MCQ 20. What happens if you open a file in "w" mode that doesn't exist?

- a) Error
- b) File is created
- c) Returns None
- d) Opens in read mode

Answer: b

Explanation: Write mode creates a new file if it doesn't exist.

★ MCQ 21. How do you get the size of a file in bytes?

- a) `os.size("file.txt")`
- b) `os.path.getsize("file.txt")`
- c) `file.size()`
- d) `len(file)`

Answer: b

Explanation: `os.path.getsize()` returns file size in bytes.

★ MCQ 22. Which mode allows both reading and writing?

- a) `"rw"`
- b) `"r+"`
- c) `"wr"`
- d) `"a+"`

Answer: b

Explanation: `"r+"` mode opens file for both reading and writing.

★ MCQ 23. What is the correct way to read a file line by line?

- a) `for line in file.read()`
- b) `for line in file:`
- c) `while file.readline()`
- d) `for file in line:`

Answer: b

Explanation: Iterating directly over the file object reads line by line efficiently.

★ MCQ 24. Which function lists all files in a directory?

- a) `os.files()`
- b) `os.list()`
- c) `os.listdir()`
- d) `os.getfiles()`

Answer: c

Explanation: `os.listdir()` returns a list of files and directories.

★ MCQ 25. What does "a+" mode do?

- a) Append only
- b) Read only
- c) Append and read
- d) Write only

Answer: c

Explanation: "a+" mode allows both appending and reading.

★ MCQ 26. How do you create a directory?

- a) `os.createdir("name")`
- b) `os.mkdir("name")`
- c) `os.makedir("name")`
- d) `os.newdir("name")`

Answer: b

Explanation: `os.mkdir()` creates a new directory.

★ MCQ 27. Which is true about binary files?

- a) Can be read as text
- b) Contain readable characters
- c) Store raw bytes
- d) Only for images

Answer: c

Explanation: Binary files store data as raw bytes and cannot be read as plain text.

★ MCQ 28. What is the output?

`with open("test.txt", "w") as f:`

```
f.write("Line 1\n")  
f.write("Line 2\n")
```

How many lines are in the file?

- a) 0
- b) 1
- c) 2
- d) 3

Answer: c

Explanation: Two lines are written with newline characters, resulting in 2 lines.

★ MCQ 29. Which mode is used to write binary data?

- a) "w"
- b) "wb"
- c) "bw"
- d) "write"

Answer: b

Explanation: "wb" mode is used to write binary data to files.

★ MCQ 30. What happens if you forget to close a file?

- a) Program crashes
- b) File gets corrupted
- c) Data may not be saved properly
- d) Nothing happens

Answer: c

Explanation: Not closing files can result in data not being flushed to disk properly, potentially losing data.

30 MCQs Completed

MODULE 7 — CODING TASKS (20 Exercises with Solutions)

★ Task 1 — Create and Write to a File

Problem: Create a text file and write "Hello, Python!" to it.

Solution:

```
with open("hello.txt", "w") as file:
    file.write("Hello, Python!")
print("File created and written successfully")
```

Output:

```
File created and written successfully
```

Explanation: Opens file in write mode, writes content, and automatically closes using `with`.

★ Task 2 — Read and Display File Content

Problem: Read and display the entire content of a file.

Solution:

```
try:
    with open("hello.txt", "r") as file:
        content = file.read()
        print("File content:")
        print(content)
except FileNotFoundError:
    print("File not found!")
```

Output:

```
File content:
Hello, Python!
```

Explanation: Reads entire file using `read()` and includes error handling.

★ Task 3 — Append Text to a File

Problem: Append a new line to an existing file.

Solution:

```
with open("hello.txt", "a") as file:
    file.write("\nNew line appended")
print("Content appended successfully")
```

Explanation: Append mode "a" adds content without deleting existing data.

★ Task 4 — Count Lines in a File

Problem: Count the total number of lines in a file.

Solution:

```
try:
    with open("hello.txt", "r") as file:
        lines = file.readlines()
        print(f"Total lines: {len(lines)}")
except FileNotFoundError:
    print("File not found!")
```

Output:

Total lines: 2

Explanation: `readlines()` returns a list; `len()` gives the count.

★ Task 5 — Count Words in a File

Problem: Count the total number of words in a file.

Solution:

```
try:
    with open("hello.txt", "r") as file:
        content = file.read()
        words = content.split()
        print(f"Total words: {len(words)}")
except FileNotFoundError:
    print("File not found!")
```

Output:

Total words: 5

Explanation: `split()` breaks content into words; `len()` counts them.

★ Task 6 — Count Characters in a File

Problem: Count total characters (including spaces) in a file.

Solution:

```
try:
    with open("hello.txt", "r") as file:
        content = file.read()
        print(f"Total characters: {len(content)}")
except FileNotFoundError:
    print("File not found!")
```

Explanation: `len()` on string returns total character count.

★ Task 7 — Copy File Content to Another File

Problem: Copy content from one file to another.

Solution:

```
try:
    with open("hello.txt", "r") as source:
        with open("copy.txt", "w") as destination:
            content = source.read()
            destination.write(content)
        print("File copied successfully")
except FileNotFoundError:
    print("Source file not found!")
```

Output:

```
File copied successfully
```

Explanation: Reads from source and writes to destination file.

★ Task 8 — Read File Line by Line

Problem: Read and display file content line by line.

Solution:

```
try:
    with open("hello.txt", "r") as file:
        print("File content (line by line):")
        for line in file:
            print(line.strip())
except FileNotFoundError:
    print("File not found!")
```

Output:

```
File content (line by line):
Hello, Python!
New line appended
```

Explanation: Iterating over file object reads line by line. `strip()` removes extra whitespace.

★ Task 9 — Check if File Exists

Problem: Check if a file exists before reading it.

Solution:

```
import os

filename = "hello.txt"

if os.path.exists(filename):
    print(f"{filename} exists")
    with open(filename, "r") as file:
        print(file.read())
else:
    print(f"{filename} does not exist")
```

Explanation: `os.path.exists()` checks file existence before operations.

★ Task 10 — Delete a File

Problem: Delete a file from the system.

Solution:

```
import os

filename = "copy.txt"

if os.path.exists(filename):
```

```
    os.remove(filename)
    print(f"{filename} deleted successfully")
else:
    print(f"{filename} not found")
```

Output:

```
copy.txt deleted successfully
```

Explanation: `os.remove()` deletes the specified file.

★ Task 11 — Rename a File

Problem: Rename a file.

Solution:

```
import os

old_name = "hello.txt"
new_name = "greetings.txt"

if os.path.exists(old_name):
    os.rename(old_name, new_name)
    print(f"File renamed from {old_name} to {new_name}")
else:
    print(f"{old_name} not found")
```

Output:

```
File renamed from hello.txt to greetings.txt
```

Explanation: `os.rename()` changes the file name.

★ Task 12 — Get File Size

Problem: Display the size of a file in bytes.

Solution:

```
import os

filename = "greetings.txt"

if os.path.exists(filename):
    size = os.path.getsize(filename)
    print(f"File size: {size} bytes")
else:
    print(f"{filename} not found")
```

Output:

File size: 33 bytes

Explanation: `os.path.getsize()` returns file size in bytes.

★ Task 13 — Write Multiple Lines Using `writelines()`

Problem: Write a list of lines to a file.

Solution:

```
lines = [
    "Line 1: Python\n",
    "Line 2: File Handling\n",
    "Line 3: Programming\n"
]

with open("lines.txt", "w") as file:
    file.writelines(lines)

print("Multiple lines written successfully")
```

Output:

Multiple lines written successfully

Explanation: `writelines()` writes all strings from a list to the file.

★ Task 14 — Find and Replace Text in File

Problem: Replace a word in a file with another word.

Solution:

```
filename = "greetings.txt"
old_word = "Python"
new_word = "Programming"

try:
    with open(filename, "r") as file:
        content = file.read()

    new_content = content.replace(old_word, new_word)

    with open(filename, "w") as file:
        file.write(new_content)
```

```
    print(f"Replaced '{old_word}' with '{new_word}'")
except FileNotFoundError:
    print("File not found!")
```

Explanation: Reads file, replaces text using `replace()`, then writes back.

★ Task 15 — List All Files in Directory

Problem: Display all files in the current directory.

Solution:

```
import os

print("Files in current directory:")
files = os.listdir(".")

for file in files:
    print(file)
```

Output:

```
Files in current directory:
greetings.txt
lines.txt
script.py
```

Explanation: `os.listdir()` returns list of all files and directories.

★ Task 16 — Create a Directory

Problem: Create a new directory.

Solution:

```
import os

directory = "my_folder"

if not os.path.exists(directory):
    os.mkdir(directory)
    print(f"Directory '{directory}' created")
else:
    print(f"Directory '{directory}' already exists")
```

Output:

```
Directory 'my_folder' created
```

Explanation: `os.mkdir()` creates a new directory if it doesn't exist.

★ Task 17 — Read Specific Line Number

Problem: Read and display a specific line from a file (e.g., line 2).

Solution:

```
try:
    with open("lines.txt", "r") as file:
        lines = file.readlines()
        line_number = 2

        if line_number <= len(lines):
            print(f"Line {line_number}: {lines[line_number - 1].strip()}")
        else:
            print("Line number out of range")
except FileNotFoundError:
    print("File not found!")
```

Output:

Line 2: Line 2: File Handling

Explanation: Reads all lines into a list and accesses by index.

★ Task 18 — Copy Binary File (Image)

Problem: Copy an image file to another location.

Solution:

```
try:
    with open("original.jpg", "rb") as source:
        with open("copy.jpg", "wb") as destination:
            destination.write(source.read())
        print("Image copied successfully")
except FileNotFoundError:
    print("Source image not found!")
```

Explanation: Uses binary mode `"rb"` and `"wb"` to copy image files.

★ Task 19 — Merge Multiple Files

Problem: Merge content of multiple files into one.

Solution:

```
input_files = ["file1.txt", "file2.txt", "file3.txt"]
output_file = "merged.txt"

with open(output_file, "w") as outfile:
    for filename in input_files:
        try:
            with open(filename, "r") as infile:
                outfile.write(infile.read())
                outfile.write("\n") # Separator
        except FileNotFoundError:
            print(f"{filename} not found, skipping...")

print(f"Files merged into {output_file}")
```

Explanation: Reads each file and writes content to output file sequentially.

★ Task 20 — Create a Simple Log System

Problem: Create a logging system that appends timestamped messages to a log file.

Solution:

```
from datetime import datetime

def log_message(message):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    log_entry = f"[{timestamp}] {message}\n"

    with open("app.log", "a") as file:
        file.write(log_entry)
    print(f"Logged: {message}")

# Test the logger
log_message("Application started")
log_message("User logged in")
log_message("Data processed successfully")
log_message("Application closed")
```

Output:

```
Logged: Application started
Logged: User logged in
Logged: Data processed successfully
Logged: Application closed
```

File content (app.log):

```
[2024-01-15 10:30:45] Application started
[2024-01-15 10:30:46] User logged in
[2024-01-15 10:30:47] Data processed successfully
[2024-01-15 10:30:48] Application closed
```

Explanation: Creates a logging function that appends timestamped messages to a file, useful for tracking application events.

Module 7 Coding Tasks Completed (20)

MODULE 7 COMPLETE ✓

Complete Package Includes:

- ✓ **Theory** - Comprehensive file handling concepts
 - ✓ **25 Examples** - Practical code snippets
 - ✓ **6 Lectures** - Detailed explanations
 - ✓ **30 MCQs** - Questions with answers and explanations
 - ✓ **20 Coding Tasks** - Complete solutions with explanations
 - ✓ **LMS-Ready Format** - Copy-paste ready
-

Topics Covered:

- File opening and modes
 - Reading files (multiple methods)
 - Writing and appending
 - The `with` statement
 - Error handling
 - Binary files
 - File management (`os` module)
 - Directory operations
 - File copying and merging
 - Practical applications (logging, backup)
-

MODULE 8 — EXCEPTION HANDLING IN PYTHON

Complete LMS-Ready Content

PART 1: THEORY (Basic Understanding)

Introduction

Exception handling allows your program to deal with unexpected errors without crashing. Errors happen when:

- User enters wrong input
- File is missing
- Data is invalid
- Network fails
- Program performs illegal operations

Without exception handling, your Python program immediately stops when an error occurs.

Exception handling makes programs:

- ✓ Safe
 - ✓ Professional
 - ✓ Debuggable
 - ✓ Reliable
-

★ 1. What Is an Exception?

An exception is an error that happens during program execution.

Example errors:

- `ZeroDivisionError`
- `ValueError`
- `FileNotFoundError`
- `TypeError`
- `IndexError`
- `KeyError`

When these occur, Python stops the program unless you handle them.

★ 2. Why Handle Exceptions?

To prevent the application from:

- ✗ Crashing
- ✗ Losing user data
- ✗ Stopping important operations

And to allow:

- ✓ Graceful failure
 - ✓ Error messages
 - ✓ Logging
 - ✓ Alternative execution flow
-

★ 3. try - except Block

Basic syntax:

```
try:
    # code that may cause error
except:
    # code that runs when error occurs
```

Example:

```
try:
    x = 10 / 0
except:
    print("Cannot divide by zero!")
```

★ 4. Handling Specific Exceptions

```
try:
    n = int("abc")
except ValueError:
    print("Invalid number!")
```

This is better than a generic except.

★ 5. Using else

```
try:
    print("No error")
except:
```

```
    print("Error occurred")
else:
    print("Executed when no error")
```

★ 6. Using finally

Always runs — even if error happens.

```
try:
    f = open("data.txt")
except:
    print("File missing")
finally:
    print("Closing program")
```

★ 7. raise Keyword

Used to create a manually thrown error.

```
age = -1
if age < 0:
    raise ValueError("Age cannot be negative")
```

★ 8. Try - Except - Else - Finally (Complete Structure)

```
try:
    # risky code
except:
    # error handling
else:
    # runs if no error
finally:
    # always executes
```

★ 9. Multiple Except Blocks

```
try:
    x = int("abc")
except ValueError:
    print("Value error")
except TypeError:
    print("Type error")
```

★ 10. Why Exception Handling Matters?

Used in:

- ✓ File Handling
- ✓ Database connections

- ✓ API calls
- ✓ Network programming
- ✓ Input validation
- ✓ Data processing
- ✓ Web development

Real-world apps must handle unexpected errors.

★ 11. Summary

- Exceptions stop program execution
- try-except prevents crashes
- You can handle specific errors
- finally always runs
- raise helps create custom errors

Exception handling makes your code safe, stable, and user-friendly.

PART 2: EXAMPLES & SNIPPETS (25 Examples)

Example 1: Basic try-except

```
try:
    result = 10 / 0
except:
    print("An error occurred!")
```

Output: An error occurred!

Explanation: Catches any error and prevents program crash.

Example 2: Handling ZeroDivisionError

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

Output: Cannot divide by zero!

Explanation: Specifically handles division by zero error.

Example 3: Handling ValueError

```
try:
    number = int("abc")
except ValueError:
    print("Invalid number format!")
```

Output: Invalid number format!

Explanation: Catches error when converting invalid string to integer.

Example 4: Multiple Except Blocks

```
try:
    num = int(input("Enter number: "))
    result = 100 / num
except ValueError:
    print("Please enter a valid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Explanation: Handles different error types separately with specific messages.

Example 5: Using else Clause

```
try:
    number = int("123")
except ValueError:
    print("Error occurred")
else:
    print(f"Successfully converted: {number}")
```

Output: Successfully converted: 123

Explanation: else block executes only when no exception occurs.

Example 6: Using finally

```
try:
    file = open("data.txt", "r")
    content = file.read()
except FileNotFoundError:
    print("File not found")
finally:
    print("Execution completed")
```

Explanation: finally block always executes regardless of error.

Example 7: Complete try-except-else-finally

```
try:
    num = int("42")
    result = num / 2
except ValueError:
    print("Invalid input")
except ZeroDivisionError:
    print("Division by zero")
else:
    print(f"Result: {result}")
finally:
    print("End of operation")
```

Output:

```
Result: 21.0
End of operation
```

Explanation: Shows complete exception handling structure.

Example 8: Raising Custom Exception

```
age = int(input("Enter age: "))

if age < 0:
    raise ValueError("Age cannot be negative!")
elif age > 150:
    raise ValueError("Age seems unrealistic!")
else:
    print(f"Age: {age}")
```

Explanation: Uses `raise` to manually throw exceptions based on conditions.

Example 9: Handling FileNotFoundError

```
try:
    with open("missing.txt", "r") as file:
        content = file.read()
except FileNotFoundError:
    print("The file does not exist")
```

Output: The file does not exist

Explanation: Handles missing file error gracefully.

Example 10: Handling IndexError

```
my_list = [1, 2, 3]

try:
    print(my_list[5])
except IndexError:
    print("Index out of range!")
```

Output: Index out of range!

Explanation: Catches error when accessing non-existent list index.

Example 11: Handling KeyError

```
my_dict = {"name": "John", "age": 25}

try:
    print(my_dict["city"])
except KeyError:
    print("Key not found in dictionary")
```

Output: Key not found in dictionary

Explanation: Handles missing dictionary key error.

Example 12: Handling TypeError

```
try:
    result = "Hello" + 5
except TypeError:
    print("Cannot concatenate string and integer")
```

Output: Cannot concatenate string and integer

Explanation: Catches type mismatch errors.

Example 13: Generic Exception with Message

```
try:
    result = 10 / 0
except Exception as e:
    print(f"Error occurred: {e}")
```

Output: Error occurred: division by zero

Explanation: Catches any exception and displays error message.

Example 14: Nested try-except

```
try:
    try:
        num = int("abc")
    except ValueError:
        print("Inner exception: Invalid number")
        raise # Re-raise the exception
except:
    print("Outer exception: Caught re-raised error")
```

Output:

```
Inner exception: Invalid number
Outer exception: Caught re-raised error
```

Explanation: Shows nested exception handling and re-raising.

Example 15: User Input Validation

```
while True:
    try:
        age = int(input("Enter your age: "))
        if age < 0:
            raise ValueError("Age cannot be negative")
        break
    except ValueError as e:
        print(f"Invalid input: {e}")
        print("Please try again")

print(f"Your age is: {age}")
```

Explanation: Continuously asks for valid input until received.

Example 16: Division Calculator with Error Handling

```
def divide(a, b):
    try:
        result = a / b
        return result
    except ZeroDivisionError:
        return "Cannot divide by zero"
    except TypeError:
        return "Invalid operands"

print(divide(10, 2))    # 5.0
print(divide(10, 0))    # Cannot divide by zero
print(divide(10, "a"))  # Invalid operands
```


Explanation: Function with comprehensive error handling.

Example 17: File Operations with Exception Handling

```
def read_file(filename):
    try:
        with open(filename, "r") as file:
            return file.read()
    except FileNotFoundError:
        return "File not found"
    except PermissionError:
        return "Permission denied"
    except Exception as e:
        return f"Error: {e}"

content = read_file("data.txt")
print(content)
```

Explanation: Handles multiple file-related exceptions.

Example 18: Custom Exception Class

```
class NegativeNumberError(Exception):
    def __init__(self, number):
        self.number = number
        self.message = f"Negative number not allowed: {number}"
        super().__init__(self.message)

try:
    num = -5
    if num < 0:
        raise NegativeNumberError(num)
except NegativeNumberError as e:
    print(e.message)
```

Output: Negative number not allowed: -5

Explanation: Creates and uses custom exception class.

Example 19: Assert Statement

```
def calculate_average(numbers):
    assert len(numbers) > 0, "List cannot be empty"
    return sum(numbers) / len(numbers)

try:
    avg = calculate_average([])
except AssertionError as e:
    print(f"Assertion Error: {e}")
```

Output: Assertion Error: List cannot be empty

Explanation: Uses assert for debugging and validation.

Example 20: Exception in Loop

```
numbers = [10, 0, 5, "abc", 2]

for num in numbers:
    try:
        result = 100 / num
        print(f"100 / {num} = {result}")
    except ZeroDivisionError:
        print(f"Cannot divide by zero (skipping {num})")
    except TypeError:
        print(f"Invalid type (skipping {num})")
```

Output:

```
100 / 10 = 10.0
Cannot divide by zero (skipping 0)
100 / 5 = 20.0
Invalid type (skipping abc)
100 / 2 = 50.0
```

Explanation: Handles exceptions in loop without breaking execution.

Example 21: Database Connection Simulation

```
def connect_database(db_name):
    try:
        if db_name == "":
            raise ValueError("Database name cannot be empty")
        print(f"Connecting to {db_name}...")
        # Simulate connection
        print("Connection successful")
    except ValueError as e:
        print(f"Error: {e}")
    finally:
        print("Closing database resources")

connect_database("mydb")
connect_database("")
```

Explanation: Simulates database connection with proper error handling.

Example 22: Multiple Exception Types in One Block

```
try:
```

```
num = int(input("Enter a number: "))
result = 100 / num
my_list = [1, 2, 3]
print(my_list[num])
except (ValueError, ZeroDivisionError, IndexError) as e:
    print(f"Error occurred: {type(e).__name__}")
    print(f"Message: {e}")
```

Explanation: Handles multiple exception types with single except block.

Example 23: Logging Exceptions

```
import logging

logging.basicConfig(level=logging.ERROR, filename='errors.log')

try:
    result = 10 / 0
except ZeroDivisionError as e:
    logging.error(f"Division error: {e}")
    print("Error logged to file")
```

Explanation: Logs exceptions to file for debugging and monitoring.

Example 24: Exception Handling in Function Calls

```
def risky_function():
    return 10 / 0

def safe_caller():
    try:
        result = risky_function()
        return result
    except ZeroDivisionError:
        print("Handled division by zero in caller")
        return None

result = safe_caller()
print(f"Result: {result}")
```

Output:

```
Handled division by zero in caller
Result: None
```

Explanation: Shows exception handling across function calls.

Example 25: Real-World Input Validator

```
def get_valid_number(prompt):
    while True:
        try:
            value = float(input(prompt))
            if value < 0:
                raise ValueError("Number must be positive")
            return value
        except ValueError as e:
            print(f"Invalid input: {e}")
            print("Please try again\n")

# Usage
age = get_valid_number("Enter your age: ")
print(f"Valid age entered: {age}")
```

Explanation: Robust input validation with retry mechanism.

PART 3: LECTURES (6 Detailed Lectures)

★ Lecture 1 — Introduction to Exceptions

Topic

"Understanding Exceptions, Error Types, and Why Exception Handling is Critical in Python Programming."

Definition (Detailed & Multi-Line)

An exception is a runtime error that disrupts normal program flow.

Exceptions occur due to invalid operations, bad input, missing resources, or logic errors.

Python provides a robust exception handling mechanism using try-except blocks.

Without handling exceptions, programs terminate abruptly when errors occur.

Exception handling makes software reliable, professional, and user-friendly.

Common built-in exceptions include ValueError, TypeError, ZeroDivisionError, FileNotFoundError, and IndexError.

Syntax

```
try:
    # risky code
except ExceptionType:
```

```
# handle error
```

Examples

Example 1 — Unhandled Exception

```
# This will crash
result = 10 / 0 # ZeroDivisionError
```

Example 2 — Handled Exception

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Example 3 — Multiple Error Types

```
try:
    num = int("abc")
except (ValueError, TypeError):
    print("Invalid input")
```

Key Takeaways

- Exceptions are runtime errors
- Unhandled exceptions crash programs
- try-except prevents crashes
- Specific exception handling is better than generic
- Professional apps must handle exceptions

Practice Section

ready_to_practice: You understand what exceptions are.

description: Write programs that handle common exceptions.

mcqs: Attempt MCQs 1–5.

coding_challenges: Create a calculator with exception handling.

★ Lecture 2 — try-except Block

Topic

"Understanding try-except Structure, Catching Specific Exceptions, and Basic Error Handling Patterns."

Definition (Detailed & Multi-Line)

The try block contains code that might raise exceptions.

The except block catches and handles exceptions when they occur.

You can have multiple except blocks for different exception types.

Generic except catches all exceptions but is not recommended.

Specific exception handling provides better error messages and debugging.

Always catch the most specific exception first, then more general ones.

Syntax

```
try:
    # code that may raise exception
except SpecificException:
    # handle specific error
except AnotherException:
    # handle another error
except:
    # catch all other exceptions
```

Examples

Example 1 — Basic Exception Handling

```
try:
    number = int("abc")
except ValueError:
    print("Please enter a valid number")
```

Example 2 — Multiple Exceptions

```
try:
    num = int(input("Enter number: "))
    result = 100 / num
except ValueError:
    print("Invalid number format")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Example 3 — Accessing Exception Object

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
```

Key Takeaways

- try contains risky code
- except handles errors
- Multiple except blocks for different errors
- Use specific exceptions
- Access error details with `as` keyword

Practice Section

ready_to_practice: You can use try-except blocks.

description: Write programs with proper exception handling.

mcqs: Attempt MCQs 6–10.

coding_challenges: Input validator with multiple exception types.

★ Lecture 3 — else and finally Clauses

Topic

"Understanding else Clause for Success Path and finally Clause for Cleanup Operations."

Definition (Detailed & Multi-Line)

The else block executes only when no exception occurs in try block.

The finally block always executes regardless of exceptions.

finally is used for cleanup operations like closing files or releasing resources.

else makes code cleaner by separating success logic from error handling.

finally ensures critical cleanup code runs even if exceptions occur.

This combination provides complete control over execution flow.

Syntax

```
try:
    # risky code
except ExceptionType:
    # error handling
else:
    # runs if no exception
finally:
    # always runs
```

Examples

Example 1 — Using else

```
try:
    num = int("123")
except ValueError:
    print("Invalid number")
else:
    print(f"Successfully converted: {num}")
```

Example 2 — Using finally

```
try:
    file = open("data.txt")
    content = file.read()
except FileNotFoundError:
    print("File not found")
finally:
    print("Cleanup complete")
```

Example 3 — Complete Structure

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Division error")
else:
    print(f"Result: {result}")
finally:
    print("Operation completed")
```

Key Takeaways

- else runs only on success
- finally always runs
- Use finally for cleanup
- Separates success and error logic
- Ensures resource management

Practice Section

ready_to_practice: You understand else and finally.

description: Implement complete exception handling structures.

mcqs: Attempt MCQs 11–15.

coding_challenges: File processor with proper cleanup.

★ Lecture 4 — Raising Exceptions

Topic

"Understanding raise Keyword, Creating Custom Exceptions, and Manual Error Triggering."

Definition (Detailed & Multi-Line)

The raise keyword manually triggers exceptions in your code.

You can raise built-in exceptions or create custom exception classes.

Raising exceptions helps enforce business rules and data validation.

Custom exceptions make error handling more specific to your application.

Re-raising exceptions allows you to log and then propagate errors.

This technique is essential for defensive programming and input validation.

Syntax

```
# Raise built-in exception
raise ValueError("Error message")

# Create custom exception
class CustomError(Exception):
    pass

raise CustomError("Custom error message")

# Re-raise caught exception
try:
    # code
except Exception:
    # log error
    raise # re-raise
```

Examples

Example 1 — Raising ValueError

```
age = -5
if age < 0:
    raise ValueError("Age cannot be negative")
```

Example 2 — Custom Exception

```
class InsufficientFundsError(Exception):
    pass

balance = 100
withdraw = 150
```

```
if withdraw > balance:
    raise InsufficientFundsError("Insufficient balance")
```

Example 3 — Re-raising Exception

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Logging error...")
    raise # Re-raise the exception
```

Key Takeaways

- raise manually triggers exceptions
- Create custom exceptions for specific needs
- Use raise for input validation
- Re-raise to log and propagate errors
- Essential for defensive programming

Practice Section

ready_to_practice: You can raise exceptions.

description: Create validation functions with custom exceptions.

mcqs: Attempt MCQs 16–20.

coding_challenges: Banking system with custom exceptions.

★ Lecture 5 — Common Built-in Exceptions

Topic

"Understanding Python's Built-in Exception Hierarchy, Common Exception Types, and When to Use Each."

Definition (Detailed & Multi-Line)

Python provides a comprehensive hierarchy of built-in exceptions.

BaseException is the base class for all exceptions.

Exception is the base for all non-system-exiting exceptions.

Understanding exception types helps choose appropriate error handling.

Common exceptions include ValueError, TypeError, IndexError, KeyError, FileNotFoundError, and ZeroDivisionError.

Each exception type is designed for specific error scenarios.

Syntax

Common Exception Types:

```
ValueError      # Invalid value
TypeError       # Wrong type
IndexError      # Invalid index
KeyError        # Missing dictionary key
FileNotFoundError # File doesn't exist
ZeroDivisionError # Division by zero
AttributeError  # Invalid attribute
ImportError     # Import fails
RuntimeError    # General runtime error
```

Examples

Example 1 — ValueError

```
try:
    num = int("abc")
except ValueError:
    print("Cannot convert to integer")
```

Example 2 — IndexError

```
try:
    my_list = [1, 2, 3]
    print(my_list[10])
except IndexError:
    print("Index out of range")
```

Example 3 — KeyError

```
try:
    my_dict = {"name": "John"}
    print(my_dict["age"])
except KeyError:
    print("Key not found")
```

Key Takeaways

- Python has hierarchical exception system
- Each exception type has specific purpose
- Use appropriate exception for each scenario
- Catch specific exceptions for better error handling
- Understanding exception types improves debugging

Practice Section

ready_to_practice: You know common exception types.

description: Handle various built-in exceptions appropriately.

mcqs: Attempt MCQs 21–25.

coding_challenges: Program handling multiple exception types.

★ Lecture 6 — Best Practices and Real-World Applications

Topic

"Exception Handling Best Practices, Logging Errors, Production-Ready Code, and Real-World Patterns."

Definition (Detailed & Multi-Line)

Best practices ensure maintainable and reliable exception handling code.

Always catch specific exceptions rather than using generic except.

Log exceptions for debugging and monitoring in production.

Clean up resources using finally or context managers.

Never use exceptions for normal control flow.

Provide meaningful error messages to users.

Real-world applications require comprehensive exception handling strategies.

Syntax

Best Practices:

```
# Good: Specific exception
try:
    num = int(value)
except ValueError:
    handle_error()

# Bad: Generic exception
try:
    num = int(value)
except:
    handle_error()

# Good: Context manager
with open("file.txt") as f:
```

```

        content = f.read()

# Good: Logging
import logging
try:
    risky_operation()
except Exception as e:
    logging.error(f"Error: {e}")

```

Examples

Example 1 — Production-Ready Function

```

import logging

def process_data(data):
    try:
        result = complex_calculation(data)
        return {"status": "success", "result": result}
    except ValueError as e:
        logging.error(f"ValueError: {e}")
        return {"status": "error", "message": "Invalid data"}
    except Exception as e:
        logging.critical(f"Unexpected error: {e}")
        return {"status": "error", "message": "Internal error"}

```

Example 2 — Input Validation Pattern

```

def get_positive_number(prompt):
    while True:
        try:
            value = float(input(prompt))
            if value <= 0:
                raise ValueError("Must be positive")
            return value
        except ValueError as e:
            print(f"Invalid input: {e}. Try again.")

```

Example 3 — Resource Management

```

def process_file(filename):
    try:
        with open(filename, 'r') as file:
            data = file.read()
            return process(data)
    except FileNotFoundError:
        logging.error(f"File not found: {filename}")
        return None
    except Exception as e:
        logging.error(f"Error processing {filename}: {e}")
        return None

```

Key Takeaways

- Use specific exceptions
- Log errors for debugging
- Clean up resources properly

- Provide user-friendly messages
- Never suppress exceptions silently
- Follow consistent error handling patterns

Practice Section

ready_to_practice: You understand best practices.

description: Refactor code with proper exception handling.

mcqs: Attempt MCQs 26–30.

coding_challenges: Build production-ready application with logging.

MODULE 8 LECTURES COMPLETED

PART 4: MCQs (30 Questions with Answers + Explanations)

★ MCQ 1. What is an exception in Python?

- a) A syntax error
- b) A runtime error
- c) A logical error
- d) A warning

Answer: b

Explanation: An exception is a runtime error that disrupts normal program flow during execution.

★ MCQ 2. Which keyword is used to handle exceptions?

- a) catch
- b) except
- c) handle
- d) error

Answer: b

Explanation: Python uses `except` keyword to catch and handle exceptions.

★ MCQ 3. What happens if an exception is not handled?

- a) Program continues normally
- b) Program crashes
- c) Program shows a warning
- d) Nothing

Answer: b

Explanation: Unhandled exceptions cause the program to terminate abruptly.

★ MCQ 4. Which block always executes regardless of exceptions?

- a) try
- b) except
- c) else
- d) finally

Answer: d

Explanation: The `finally` block always executes, whether an exception occurs or not.

★ MCQ 5. What exception is raised for division by zero?

- a) ValueError
- b) ZeroDivisionError
- c) ArithmeticError
- d) DivisionError

Answer: b

Explanation: `ZeroDivisionError` is specifically raised when dividing by zero.

★ MCQ 6. Which exception is raised for invalid type operations?

- a) `TypeError`
- b) `ValueError`
- c) `SyntaxError`
- d) `AttributeError`

Answer: a

Explanation: `TypeError` occurs when an operation is performed on an inappropriate type.

★ MCQ 7. When does the `else` block execute?

- a) Always
- b) When exception occurs
- c) When no exception occurs
- d) Never

Answer: c

Explanation: The `else` block executes only when no exception occurs in the `try` block.

★ MCQ 8. Which keyword is used to manually raise an exception?

- a) `throw`
- b) `raise`
- c) `error`
- d) `except`

Answer: b

Explanation: Python uses the `raise` keyword to manually trigger exceptions.

★ MCQ 9. What is the output?


```
try:
    print("Hello")
except:
    print("Error")
else:
    print("Success")
```

- a) Hello
- b) Error
- c) Hello Success
- d) Success

Answer: c

Explanation: "Hello" prints, no exception occurs, so "Success" also prints.

★ MCQ 10. Which exception is raised when a list index is out of range?

- a) ValueError
- b) IndexError
- c) KeyError
- d) RangeError

Answer: b

Explanation: `IndexError` is raised when accessing an invalid list index.

★ MCQ 11. What is the base class for all exceptions?

- a) Error
- b) Exception
- c) BaseException
- d) RuntimeError

Answer: c

Explanation: `BaseException` is the base class for all built-in exceptions in Python.

★ MCQ 12. Which exception is raised when a dictionary key is not found?

- a) ValueError
- b) IndexError
- c) KeyError
- d) AttributeError

Answer: c

Explanation: `KeyError` is raised when accessing a non-existent dictionary key.

★ MCQ 13. How do you catch multiple exceptions in one except block?

- a) `except ValueError, TypeError:`
- b) `except (ValueError, TypeError):`
- c) `except [ValueError, TypeError]:`
- d) `except ValueError | TypeError:`

Answer: b

Explanation: Use a tuple of exception types in parentheses to catch multiple exceptions.

★ MCQ 14. What does `except Exception as e:` do?

- a) Raises exception
- b) Stores exception in variable `e`
- c) Prints exception
- d) Ignores exception

Answer: b

Explanation: The `as` keyword assigns the exception object to variable `e` for access.

★ MCQ 15. Which is NOT a valid exception handling block?

- a) try
- b) except
- c) catch
- d) finally

Answer: c

Explanation: Python doesn't have a `catch` block; it uses `except` instead.

★ MCQ 16. What is the output?

```
try:
    x = 10 / 0
except:
    print("Error")
finally:
    print("Done")
```

- a) Error
- b) Done
- c) Error Done
- d) Nothing

Answer: c

Explanation: Both `except` and `finally` blocks execute, printing "Error" then "Done".

★ MCQ 17. Which exception is raised when a file is not found?

- a) FileNotFoundError
- b) IOError
- c) FileNotFoundError
- d) OSError

Answer: c

Explanation: `FileNotFoundError` is specifically raised when trying to open a non-existent file.

★ MCQ 18. Can you have multiple except blocks for one try?

- a) No
- b) Yes
- c) Only two
- d) Depends on Python version

Answer: b

Explanation: You can have multiple `except` blocks to handle different exception types.

★ MCQ 19. What is the output?

```
try:
    result = int("abc")
except ValueError:
    print("Invalid")
except TypeError:
    print("Type error")
```

- a) Invalid
- b) Type error
- c) Both
- d) Nothing

Answer: a

Explanation: `int("abc")` raises `ValueError`, so "Invalid" is printed.

★ MCQ 20. Which is used for creating custom exceptions?

- a) Create new class inheriting from `Exception`
- b) Use `raise` keyword only
- c) Use `except` keyword
- d) Not possible

Answer: a

Explanation: Custom exceptions are created by inheriting from the `Exception` class.

★ MCQ 21. What does `raise` without arguments do?

- a) Raises `TypeError`
- b) Re-raises the last exception
- c) Does nothing
- d) Raises `ValueError`

Answer: b

Explanation: `raise` without arguments re-raises the most recent exception.

★ MCQ 22. Which exception is raised for invalid function arguments?

- a) `TypeError`
- b) `ValueError`
- c) Either a or b depending on context
- d) `ArgumentError`

Answer: c

Explanation: `ValueError` for wrong values, `TypeError` for wrong types of arguments.

★ MCQ 23. Is it good practice to use bare `except`?

- a) Yes, always
- b) No, catch specific exceptions
- c) Only in production
- d) Doesn't matter

Answer: b

Explanation: Always catch specific exceptions for better error handling and debugging.

★ MCQ 24. What is the output?

```
try:
```

```
print("Start")
raise ValueError("Error")
print("End")
except ValueError:
    print("Caught")
```

- a) Start End
- b) Start Caught
- c) Start End Caught
- d) Caught

Answer: b

Explanation: "Start" prints, then exception is raised and caught, printing "Caught". "End" never executes.

★ MCQ 25. Which module is used for logging exceptions?

- a) error
- b) logging
- c) log
- d) exception

Answer: b

Explanation: The `logging` module provides facilities for logging exceptions and errors.

★ MCQ 26. Can finally block contain return statement?

- a) Yes
- b) No
- c) Only with try
- d) Only with except

Answer: a

Explanation: `finally` can contain return, but it overrides return from try/except blocks.

★ MCQ 27. What exception is raised for attribute access errors?

- a) ValueError
- b) TypeError
- c) AttributeError
- d) KeyError

Answer: c

Explanation: `AttributeError` is raised when accessing a non-existent attribute.

★ MCQ 28. Which is true about exception handling?

- a) Slows down program always
- b) Makes code more reliable
- c) Should be avoided
- d) Only for file operations

Answer: b

Explanation: Exception handling makes programs more reliable and prevents crashes.

★ MCQ 29. What is the output?

```
try:
    x = [1, 2, 3]
    print(x[5])
except IndexError:
    print("Index error")
finally:
    print("Complete")
```

- a) Index error
- b) Complete
- c) Index error Complete
- d) Error

Answer: c

Explanation: Exception is caught printing "Index error", then `finally` prints "Complete".

★ MCQ 30. Which exception indicates import errors?

- a) ImportError
- b) ModuleError
- c) LoadError
- d) PackageError

Answer: a

Explanation: ImportError or ModuleNotFoundError are raised when imports fail.

30 MCQs Completed

PART 5: CODING TASKS (20 Exercises with Solutions)

★ Task 1 — Basic Exception Handling

Problem: Write a program to divide two numbers with exception handling.

Solution:

```
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))
    result = num1 / num2
    print(f"Result: {result}")
except ZeroDivisionError:
    print("Error: Cannot divide by zero")
except ValueError:
    print("Error: Please enter valid numbers")
```

Explanation: Handles division by zero and invalid input errors.

★ Task 2 — Multiple Exception Handling

Problem: Create a program that handles ValueError, TypeError, and ZeroDivisionError.

Solution:

```
try:
    num = int(input("Enter a number: "))
    result = 100 / num
    my_list = [1, 2, 3]
    print(my_list[num])
except ValueError:
    print("Invalid number format")
except ZeroDivisionError:
    print("Cannot divide by zero")
except IndexError:
    print("Index out of range")
except Exception as e:
    print(f"Unexpected error: {e}")
```

Explanation: Demonstrates handling multiple specific exceptions with a generic fallback.

★ Task 3 — File Reading with Exception Handling

Problem: Read a file with proper exception handling.

Solution:

```
filename = input("Enter filename: ")

try:
    with open(filename, "r") as file:
        content = file.read()
        print("File content:")
        print(content)
except FileNotFoundError:
    print(f"Error: {filename} not found")
except PermissionError:
    print(f"Error: No permission to read {filename}")
except Exception as e:
    print(f"Error: {e}")
```

Explanation: Handles file-related exceptions with specific error messages.

★ Task 4 — Using else and finally

Problem: Demonstrate try-except-else-finally structure.

Solution:

```
try:
    num = int(input("Enter a number: "))
    result = 100 / num
except ValueError:
    print("Invalid input")
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print(f"Result: {result}")
    print("Operation successful")
finally:
    print("Program execution completed")
```

Explanation: Shows complete exception handling structure with all clauses.

★ Task 5 — Input Validation Loop

Problem: Keep asking for valid input until received.

Solution:

```
while True:
    try:
        age = int(input("Enter your age: "))
        if age < 0 or age > 150:
            raise ValueError("Age must be between 0 and 150")
        print(f"Your age is: {age}")
        break
    except ValueError as e:
        print(f"Invalid input: {e}")
        print("Please try again\n")
```

Explanation: Validates input and repeats until valid data is received.

★ Task 6 — Custom Exception

Problem: Create a custom exception for negative numbers.

Solution:

```
class NegativeNumberError(Exception):
    def __init__(self, value):
        self.value = value
        self.message = f"Negative number not allowed: {value}"
        super().__init__(self.message)

try:
    num = int(input("Enter a positive number: "))
```

```

    if num < 0:
        raise NegativeNumberError(num)
    print(f"You entered: {num}")
except NegativeNumberError as e:
    print(e.message)
except ValueError:
    print("Invalid input")

```

Explanation: Creates and uses a custom exception class for specific validation.

★ Task 7 — Calculator with Exception Handling

Problem: Create a calculator with comprehensive error handling.

Solution:

```

def calculator():
    try:
        num1 = float(input("Enter first number: "))
        operator = input("Enter operator (+, -, *, /): ")
        num2 = float(input("Enter second number: "))

        if operator == '+':
            result = num1 + num2
        elif operator == '-':
            result = num1 - num2
        elif operator == '*':
            result = num1 * num2
        elif operator == '/':
            result = num1 / num2
        else:
            raise ValueError("Invalid operator")

        print(f"Result: {result}")

    except ValueError as e:
        print(f"Error: {e}")
    except ZeroDivisionError:
        print("Error: Cannot divide by zero")
    except Exception as e:
        print(f"Unexpected error: {e}")

calculator()

```

Explanation: Complete calculator with validation and error handling.

★ Task 8 — List Operations with Exception Handling

Problem: Perform list operations with error handling.

Solution:

```
my_list = [10, 20, 30, 40, 50]

try:
    index = int(input("Enter index to access: "))
    value = my_list[index]
    print(f"Value at index {index}: {value}")

    new_value = int(input("Enter new value: "))
    my_list[index] = new_value
    print(f"Updated list: {my_list}")

except IndexError:
    print(f"Error: Index {index} is out of range")
    print(f"Valid indices: 0 to {len(my_list)-1}")
except ValueError:
    print("Error: Please enter valid integers")
```

Explanation: Handles list index errors with helpful messages.

★ Task 9 — Dictionary Operations with Exception Handling

Problem: Access dictionary keys with error handling.

Solution:

```
student = {
    "name": "John",
    "age": 20,
    "grade": "A"
}

try:
    key = input("Enter key to access: ")
    value = student[key]
    print(f"{key}: {value}")
except KeyError:
    print(f"Error: Key '{key}' not found")
    print(f"Available keys: {list(student.keys())}")
```

Explanation: Handles missing dictionary keys gracefully.

★ Task 10 — Banking System with Exceptions

Problem: Create a simple banking system with custom exceptions.

Solution:

```
class InsufficientFundsError(Exception):
    pass

class InvalidAmountError(Exception):
    pass

class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        try:
            if amount <= 0:
                raise InvalidAmountError("Amount must be positive")
            self.balance += amount
            print(f"Deposited: ${amount}")
            print(f"New balance: ${self.balance}")
        except InvalidAmountError as e:
            print(f"Error: {e}")

    def withdraw(self, amount):
        try:
            if amount <= 0:
                raise InvalidAmountError("Amount must be positive")
            if amount > self.balance:
                raise InsufficientFundsError("Insufficient funds")
            self.balance -= amount
            print(f"Withdrawn: ${amount}")
            print(f"New balance: ${self.balance}")
        except (InvalidAmountError, InsufficientFundsError) as e:
            print(f"Error: {e}")

# Test
account = BankAccount(1000)
account.deposit(500)
account.withdraw(200)
account.withdraw(2000) # Should fail
```

Explanation: Banking system with custom exceptions for business logic validation.

★ Task 11 — File Copy with Exception Handling

Problem: Copy file content with proper error handling.

Solution:

```
def copy_file(source, destination):
    try:
        with open(source, 'r') as src:
            content = src.read()
```

```

        with open(destination, 'w') as dest:
            dest.write(content)

    print(f"Successfully copied {source} to {destination}")

except FileNotFoundError:
    print(f"Error: {source} not found")
except PermissionError:
    print("Error: Permission denied")
except Exception as e:
    print(f"Error: {e}")

# Test
source_file = input("Enter source filename: ")
dest_file = input("Enter destination filename: ")
copy_file(source_file, dest_file)

```

Explanation: Robust file copying with comprehensive error handling.

★ Task 12 — User Registration Validator

Problem: Validate user registration data with exceptions.

Solution:

```

class InvalidEmailError(Exception):
    pass

class WeakPasswordError(Exception):
    pass

def validate_registration(email, password, age):
    try:
        # Email validation
        if '@' not in email or '.' not in email:
            raise InvalidEmailError("Invalid email format")

        # Password validation
        if len(password) < 8:
            raise WeakPasswordError("Password must be at least 8
characters")

        # Age validation
        age = int(age)
        if age < 18:
            raise ValueError("Must be 18 or older")

        print("Registration successful!")
        return True

    except InvalidEmailError as e:
        print(f"Email Error: {e}")
    except WeakPasswordError as e:
        print(f"Password Error: {e}")
    except ValueError as e:
        print(f"Age Error: {e}")

```

```

    except Exception as e:
        print(f"Error: {e}")

    return False

# Test
email = input("Enter email: ")
password = input("Enter password: ")
age = input("Enter age: ")
validate_registration(email, password, age)

```

Explanation: Comprehensive input validation with custom exceptions.

★ Task 13 — Temperature Converter with Exceptions

Problem: Convert temperature between Celsius and Fahrenheit with error handling.

Solution:

```

def convert_temperature():
    try:
        temp = float(input("Enter temperature: "))
        unit = input("Enter unit (C/F): ").upper()

        if unit == 'C':
            fahrenheit = (temp * 9/5) + 32
            print(f"{temp}° C = {fahrenheit}° F")
        elif unit == 'F':
            celsius = (temp - 32) * 5/9
            print(f"{temp}° F = {celsius}° C")
        else:
            raise ValueError("Unit must be C or F")

    except ValueError as e:
        print(f"Error: {e}")
    except Exception as e:
        print(f"Unexpected error: {e}")

convert_temperature()

```

Explanation: Temperature converter with input validation.

★ Task 14 — Safe Division Function

Problem: Create a division function that never crashes.

Solution:

```
def safe_divide(a, b):
    try:
        result = a / b
        return {"status": "success", "result": result}
    except ZeroDivisionError:
        return {"status": "error", "message": "Cannot divide by zero"}
    except TypeError:
        return {"status": "error", "message": "Both arguments must be numbers"}
    except Exception as e:
        return {"status": "error", "message": str(e)}

# Test cases
print(safe_divide(10, 2))      # Success
print(safe_divide(10, 0))      # Zero division
print(safe_divide(10, "a"))    # Type error
```

Output:

```
{'status': 'success', 'result': 5.0}
{'status': 'error', 'message': 'Cannot divide by zero'}
{'status': 'error', 'message': 'Both arguments must be numbers'}
```

Explanation: Returns dictionary with status instead of raising exceptions.

★ Task 15 — Assert Statements

Problem: Use assert for debugging and validation.

Solution:

```
def calculate_percentage(marks, total):
    try:
        assert total > 0, "Total must be positive"
        assert marks <= total, "Marks cannot exceed total"
        assert marks >= 0, "Marks cannot be negative"

        percentage = (marks / total) * 100
        print(f"Percentage: {percentage}%")
        return percentage

    except AssertionError as e:
        print(f"Assertion Error: {e}")
        return None

# Test
calculate_percentage(85, 100)  # Valid
calculate_percentage(110, 100) # Invalid: marks > total
calculate_percentage(-10, 100) # Invalid: negative marks
```

Explanation: Uses assertions for validation with custom error messages.

★ Task 16 — Exception Logging

Problem: Log exceptions to a file for debugging.

Solution:

```
import logging
from datetime import datetime

# Configure logging
logging.basicConfig(
    filename='errors.log',
    level=logging.ERROR,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

def risky_operation(x, y):
    try:
        result = x / y
        print(f"Result: {result}")
        return result
    except ZeroDivisionError as e:
        logging.error(f"Division by zero: {x}/{y}")
        print("Error logged to errors.log")
    except Exception as e:
        logging.error(f"Unexpected error: {e}")
        print("Error logged to errors.log")

# Test
risky_operation(10, 2)
risky_operation(10, 0)
```

Explanation: Logs errors to file for later analysis and debugging.

★ Task 17 — Nested Exception Handling

Problem: Handle exceptions in nested try blocks.

Solution:

```
def process_data(data):
    try:
        print("Processing data...")
        try:
            # Inner operation
            result = int(data) / 10
            print(f"Inner result: {result}")
        except ValueError:
            print("Inner: Invalid data format")
            raise # Re-raise to outer block
        except ZeroDivisionError:
            print("Inner: Division by zero")
    except Exception as e:
        print(f"Outer: Caught exception - {e}")
```

```
        finally:
            print("Processing complete")

# Test
process_data("50")
process_data("abc")
```

Explanation: Demonstrates nested exception handling and re-raising.

★ Task 18 — Exception Context Manager

Problem: Create a context manager for exception handling.

Solution:

```
class ExceptionHandler:
    def __enter__(self):
        print("Entering protected block")
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        if exc_type is not None:
            print(f"Exception caught: {exc_type.__name__}")
            print(f"Message: {exc_value}")
            return True # Suppress exception
        print("No exception occurred")
        return False

# Usage
with ExceptionHandler():
    print("Doing risky operation...")
    result = 10 / 0 # This will be caught

print("Program continues...")
```

Explanation: Creates custom context manager for automatic exception handling.

★ Task 19 — Multi-File Processor

Problem: Process multiple files with individual error handling.

Solution:

```
def process_files(filenamees):
    results = []

    for filename in filenamees:
        try:
            with open(filename, 'r') as file:
                content = file.read()
                word_count = len(content.split())
```

```

        results.append({
            "file": filename,
            "status": "success",
            "words": word_count
        })
        print(f"✓ Processed {filename}: {word_count} words")
    except FileNotFoundError:
        results.append({
            "file": filename,
            "status": "error",
            "message": "File not found"
        })
        print(f"X {filename}: Not found")
    except Exception as e:
        results.append({
            "file": filename,
            "status": "error",
            "message": str(e)
        })
        print(f"X {filename}: {e}")

    return results

# Test
files = ["file1.txt", "file2.txt", "missing.txt"]
results = process_files(files)

print("\nSummary:")
for result in results:
    print(result)

```

Explanation: Processes multiple files independently, continuing despite individual errors.

★ Task 20 — Complete Error Handling System

Problem: Create a comprehensive error handling system with logging, custom exceptions, and user-friendly messages.

Solution:

```

import logging
from datetime import datetime

# Configure logging
logging.basicConfig(
    filename='app.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

# Custom Exceptions
class ValidationError(Exception):
    pass

```

```

class DatabaseError(Exception):
    pass

# Application class
class Application:
    def __init__(self):
        self.users = {}
        logging.info("Application started")

    def add_user(self, username, age):
        try:
            # Validation
            if not username or len(username) < 3:
                raise ValueError("Username must be at least 3
characters")

            age = int(age)
            if age < 0 or age > 150:
                raise ValueError("Age must be between 0 and 150")

            if username in self.users:
                raise DatabaseError(f"User {username} already exists")

            # Add user
            self.users[username] = age
            logging.info(f"User added: {username}, age {age}")
            print(f"✓ User {username} added successfully")
            return True

        except ValueError as e:
            logging.warning(f"Validation error: {e}")
            print(f"X Validation Error: {e}")
            return False
        except DatabaseError as e:
            logging.error(f"Database error: {e}")
            print(f"X Database Error: {e}")
            return False
        except ValueError:
            logging.error("Invalid age format")
            print("X Error: Age must be a number")
            return False
        except Exception as e:
            logging.critical(f"Unexpected error: {e}")
            print(f"X Unexpected Error: {e}")
            return False

    def get_user(self, username):
        try:
            age = self.users[username]
            print(f"User: {username}, Age: {age}")
            return age
        except KeyError:
            print(f"X User {username} not found")
            return None

    def list_users(self):
        if not self.users:
            print("No users in database")
        else:
            print("\n=== User List ===")

```

```

        for username, age in self.users.items():
            print(f"{username}: {age} years old")

# Test the application
app = Application()

print("Adding users:")
app.add_user("john_doe", "25")
app.add_user("jane", "30")
app.add_user("a", "20")           # Validation error
app.add_user("john_doe", "35")    # Duplicate error
app.add_user("bob", "abc")        # Value error

print("\nRetrieving users:")
app.get_user("john_doe")
app.get_user("nonexistent")

print()
app.list_users()

logging.info("Application finished")

```

Output:

```

Adding users:
✓ User john_doe added successfully
✓ User jane added successfully
X Validation Error: Username must be at least 3 characters
X Database Error: User john_doe already exists
X Error: Age must be a number

Retrieving users:
User: john_doe, Age: 25
X User nonexistent not found

=== User List ===
john_doe: 25 years old
jane: 30 years old

```

Explanation: Production-ready application with custom exceptions, logging, validation, and comprehensive error handling.